arXiv:2006.07733v3 [cs.LG] 10 Sep 2020

# Bootstrap Your Own Latent
# A New Approach to Self-Supervised Learning

**Jean-Bastien Grill**[*,1] **Florian Strub**[*,1] **Florent Altché**[*,1] **Corentin Tallec**[*,1] **Pierre H. Richemond**[*,1,2]
**Elena Buchatskaya**[1] **Carl Doersch**[1] **Bernardo Avila Pires**[1] **Zhaohan Daniel Guo**[1]
**Mohammad Gheshlaghi Azar**[1] **Bilal Piot**[1] **Koray Kavukcuoglu**[1] **Rémi Munos**[1] **Michal Valko**[1]
[1]DeepMind [2]Imperial College
[jbgrill,fstrub,altche,corentint,richemond]@google.com

## Abstract

We introduce **B**ootstrap **Y**our **O**wn **L**atent (BYOL), a new approach to self-supervised image representation learning. BYOL relies on two neural networks, referred to as *online* and *target* networks, that interact and learn from each other. From an augmented view of an image, we train the online network to predict the target network representation of the same image under a different augmented view. At the same time, we update the target network with a slow-moving average of the online network. While state-of-the art methods rely on negative pairs, BYOL achieves a new state of the art *without them*. BYOL reaches 74.3% top-1 classification accuracy on ImageNet using a linear evaluation with a ResNet-50 architecture and 79.6% with a larger ResNet. We show that BYOL performs on par or better than the current state of the art on both transfer and semi-supervised benchmarks. Our implementation and pretrained models are given on GitHub.[3]

## 1 Introduction

Learning good image representations is a key challenge in computer vision [1, 2, 3] as it allows for efficient training on downstream tasks [4, 5, 6, 7]. Many different training approaches have been proposed to learn such representations, usually relying on visual pretext tasks. Among them, state-of-the-art contrastive methods [8, 9, 10, 11, 12] are trained by reducing the distance between representations of different augmented views of the same image ('positive pairs'), and increasing the distance between representations of augmented views from different images ('negative pairs'). These methods need careful treatment of negative pairs [13] by either relying on large batch sizes [8, 12], memory banks [9] or customized mining strategies [14, 15] to retrieve the negative pairs. In addition, their performance critically depends on the choice of image augmentations [8, 12].

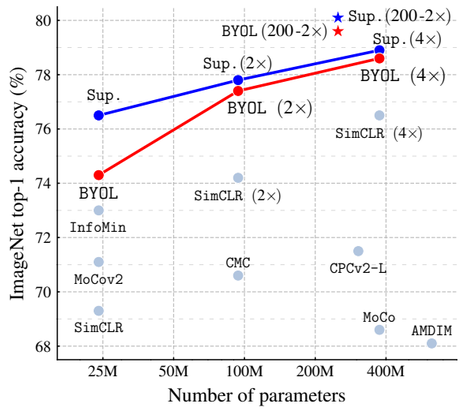



Figure 1: Performance of BYOL on ImageNet (linear evaluation) using ResNet-50 and our best architecture ResNet-200 (2×), compared to other unsupervised and supervised (Sup.) baselines [8].

In this paper, we introduce **B**ootstrap **Y**our **O**wn **L**atent (BYOL), a new algorithm for self-supervised learning of image representations. BYOL achieves higher performance than state-of-the-art contrastive methods

[*]Equal contribution; the order of first authors was randomly selected.
[3]https://github.com/deepmind/deepmind-research/tree/master/byol

arXiv:2006.07733v3 [cs.LG] 10 Sep 2020

# 自举自身的潜在表征
# 一种新的自监督学习方法

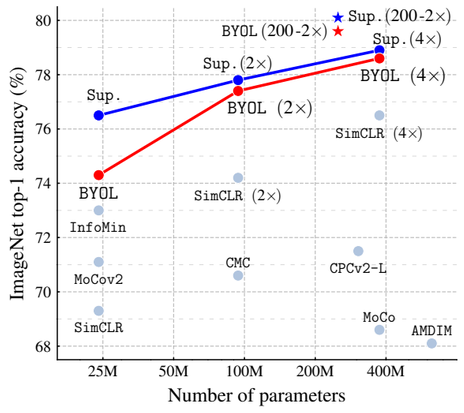
**Jean-Bastien Grill**[*,1] **Florian Strub**[*,1] **Florent Altché**[*,1] **Corentin Tallec**[*,1] **Pierre H.Richemond**[*,1,2] **Elena Buchatskaya**[1] **Carl Doersch**[1] **Bernardo Avila Pires**[1] **Zhaohan Daniel Guo**[1] **Mohammad Gheshlaghi Azar**[1] **Bilal Piot**[1] **Koray Kavukcuoglu**[1] **Rémi Munos**[1] **Michal Valko**[1] [1]DeepMind [2]Imperial College
[jbgrill,fstrub,altche,corentint,richemond]@google.com

## 摘要

我们提出 Bootstrap Your Own Latent（BYOL），一种新的自监督图像表征学习方法。BYOL 使用两个相互作用、相互学习的神经网络，称为在线网络和目标网络。从图像的一个增强视图出发，我们训练在线网络预测目标网络对同一图像另一增强视图的表征。同时，用在线网络的缓慢移动平均更新目标网络。最先进的已有方法依赖负样本对，而 BYOL 无需负样本对就取得新的最优结果。使用 ResNet-50 的线性评估时，BYOL 在 ImageNet 上达到 74.3%的 top-1 分类准确率；使用更大的 ResNet 达到79.6%。我们表明，BYOL 在迁移和半监督基准上均达到或超过当前最优方法。实现和预训练模型已公开于 GitHub。[3]

## 1 引言

学习良好的图像表征是计算机视觉的核心挑战 [1, 2, 3]，因为它能让下游任务的训练更高效 [4, 5, 6, 7]。已有多种训练方法，通常依赖视觉代理任务。其中，最先进的对比方法 [8, 9, 10, 11, 12] 通过缩小同一图像不同增强视图（"正样本对"）的表征距离，并增大不同图像增强视图（"负样本对"）的表征距离来训练。这些方法需要谨慎处理负样本对 [13]，依赖大批量 [8, 12]，、记忆库 [9] 或定制挖掘策略 [14, 15] 获取负样本对。此外，性能还强烈依赖图像增强的选择 [8, 12]。




本文提出 Bootstrap Your Own Latent（BYOL），一种新的自监督图像表征学习算法。BYOL 性能高于最先进的对比方法，

图1：BYOL 采用 ResNet-50 及最佳架构 ResNet-200（2×）时的 ImageNet 线性评估性能，与其他无监督及有监督（Sup.）基线 [8]比较。

[*]同等贡献；共同第一作者顺序随机确定。
[3]https://github.com/deepmind/deepmind-research/tree/master/byol

without using negative pairs. It iteratively bootstraps[4] the outputs of a network to serve as targets for an enhanced representation. Moreover, BYOL is more robust to the choice of image augmentations than contrastive methods; we suspect that not relying on negative pairs is one of the leading reasons for its improved robustness. While previous methods based on bootstrapping have used pseudo-labels [16], cluster indices [17] or a handful of labels [18, 19, 20], we propose to *directly bootstrap the representations*. In particular, BYOL uses two neural networks, referred to as online and target networks, that interact and learn from each other. Starting from an augmented view of an image, BYOL trains its online network to predict the target network's representation of another augmented view of the same image. While this objective admits collapsed solutions, e.g., outputting the same vector for all images, we empirically show that BYOL does not converge to such solutions. We hypothesize (see Section 3.2) that the combination of (i) the addition of a predictor to the online network and (ii) the use of a slow-moving average of the online parameters as the target network encourages encoding more and more information within the online projection and avoids collapsed solutions.

We evaluate the representation learned by BYOL on ImageNet [21] and other vision benchmarks using ResNet architectures [22]. Under the linear evaluation protocol on ImageNet, consisting in training a linear classifier on top of the frozen representation, BYOL reaches 74.3% top-1 accuracy with a standard ResNet-50 and 79.6% top-1 accuracy with a larger ResNet (Figure 1). In the semi-supervised and transfer settings on ImageNet, we obtain results on par or superior to the current state of the art. Our contributions are: (*i*) We introduce BYOL, a self-supervised representation learning method (Section 3) which achieves state-of-the-art results under the linear evaluation protocol on ImageNet without using negative pairs. (*ii*) We show that our learned representation outperforms the state of the art on semi-supervised and transfer benchmarks (Section 4). (*iii*) We show that BYOL is more resilient to changes in the batch size and in the set of image augmentations compared to its contrastive counterparts (Section 5). In particular, BYOL suffers a much smaller performance drop than SimCLR, a strong contrastive baseline, when only using random crops as image augmentations.

## 2 Related work

Most unsupervised methods for representation learning can be categorized as either generative or discriminative [23, 8]. Generative approaches to representation learning build a distribution over data and latent embedding and use the learned embeddings as image representations. Many of these approaches rely either on auto-encoding of images [24, 25, 26] or on adversarial learning [27], jointly modelling data and representation [28, 29, 30, 31]. Generative methods typically operate directly in pixel space. This however is computationally expensive, and the high level of detail required for image generation may not be necessary for representation learning.

Among discriminative methods, contrastive methods [9, 10, 32, 33, 34, 11, 35, 36] currently achieve state-of-the-art performance in self-supervised learning [37, 8, 38, 12]. Contrastive approaches avoid a costly generation step in pixel space by bringing representation of different views of the same image closer ('positive pairs'), and spreading representations of views from different images ('negative pairs') apart [39, 40]. Contrastive methods often require comparing each example with many other examples to work well [9, 8] prompting the question of whether using negative pairs is necessary.

DeepCluster [17] partially answers this question. It uses bootstrapping on previous versions of its representation to produce targets for the next representation; it clusters data points using the prior representation, and uses the cluster index of each sample as a classification target for the new representation. While avoiding the use of negative pairs, this requires a costly clustering phase and specific precautions to avoid collapsing to trivial solutions.

Some self-supervised methods are not contrastive but rely on using auxiliary handcrafted prediction tasks to learn their representation. In particular, relative patch prediction [23, 40], colorizing gray-scale images [41, 42], image inpainting [43], image jigsaw puzzle [44], image super-resolution [45], and geometric transformations [46, 47] have been shown to be useful. Yet, even with suitable architectures [48], these methods are being outperformed by contrastive methods [37, 8, 12].

Our approach has some similarities with *Predictions of Bootstrapped Latents* (PBL, [49]), a self-supervised representation learning technique for reinforcement learning (RL). PBL jointly trains the agent's history representation and an encoding of future observations. The observation encoding is used as a target to train the agent's representation, and the agent's representation as a target to train the observation encoding. Unlike PBL, BYOL uses a slow-moving average of its representation to provide its targets, and does not require a second network.

The idea of using a slow-moving average target network to produce stable targets for the online network was inspired by deep RL [50, 51, 52, 53]. Target networks stabilize the bootstrapping updates provided by the Bellman equation,

[4]Throughout this paper, the term *bootstrap* is used in its idiomatic sense rather than the statistical sense.



而不使用负样本对。它迭代地自举[4] 网络输出，将其作为学习更好表征的目标。此外，相比对比方法，BYOL 对图像增强的选择更稳健；我们认为，不依赖负样本对是其稳健性提高的主要原因之一。已有自举方法使用伪标签 [16], 、聚类索引 [17] 或少量标签 [18, 19, 20],，而我们提出直接自举表征。具体而言，BYOL使用两个相互作用、相互学习的神经网络，称为在线网络和目标网络。从图像的一个增强视图出发，BYOL 训练在线网络去预测目标网络对同一图像另一增强视图的表征。该目标虽允许坍塌解（例如对所有图像输出同一向量），但实验表明 BYOL 不会收敛至此。我们推测（见第3.2节），（i）在在线网络中增加预测器，以及（ii）使用在线参数的缓慢移动平均作为目标网络，两者结合促使在线投影编码越来越多的信息，避免坍塌解。

我们使用 ResNet 架构 [22]，在 ImageNet [21] 及其他视觉基准上评估 BYOL 学习到的表征。在冻结表征上训练线性分类器的 ImageNet 线性评估协议下， BYOL 使用标准 ResNet-50 达到74.3%的 top-1 准确率，使用更大的 ResNet 达到79.6%（图1）。在 ImageNet 半监督及迁移设置下，我们取得与当前最优方法相当或更好的结果。贡献如下：（i）提出自监督表征学习方法 BYOL（第3节），不使用负样本对，就在 ImageNet 线性评估协议下取得最优结果。（ii）所学表征在半监督和迁移基准上超过此前最优方法（第4节）。（iii）相比对比方法， BYOL 对批量大小和图像增强集合的变化更稳健（第5节）。尤其是仅使用随机裁剪作为图像增强时， BYOL 的性能下降远小于强对比基线 SimCLR。

## 2 相关工作

多数无监督表征学习方法可分为生成式或判别式 [23,8]。生成式方法构建数据及潜在嵌入上的分布，并以学到的嵌入作为图像表征。许多方法依赖图像自编码 [24, 25, 26] ，或对数据与表征进行联合建模 [28, 29, 30, 31]的对抗学习 [27], 。生成方法通常直接在像素空间工作，计算代价较高；而图像生成所需的高度细节，未必是表征学习所必需的。

判别方法中，对比方法 [9, 10, 32, 33, 34, 11, 35, 36] 目前在自监督学习上达到最优性能 [37, 8, 38, 12]。它们通过拉近同一图像不同视图的表征（“正样本对”）、推远不同图像视图的表征（“负样本对”）[39, 40]，避免在像素空间进行代价高昂的生成。对比方法通常需要将每个样本与许多其他样本比较，才能取得良好效果 [9, 8] ，由此引出负样本对是否必要的问题。

DeepCluster [17] 部分回答了这一问题。它自举历史版本的表征，产生下一种表征的目标：用先前表征对数据点聚类，再将每个样本的聚类索引作为新表征的分类目标。虽然无需负样本对，但需要昂贵的聚类阶段，还需采取专门措施避免坍塌到平凡解。

一些自监督方法并非对比方法，而是依靠人工设计的辅助预测任务学习表征。例如，相对图像块预测 [23, 40], 、灰度图着色 [41, 42], 、图像修补 [43], 、图像拼图 [44], 、图像超分辨率 [45], 和几何变换 [46, 47]都已证明有用。然而，即便使用合适架构 [48],，这些方法仍逊于对比方法 [37, 8, 12]。

我们的方法与 Predictions of Bootstrapped Latents（PBL， [49]）有相似之处，后者是强化学习（RL）的自监督表征学习技术。 PBL 联合训练智能体历史表征和未来观测编码：用观测编码作为目标训练智能体表征，反过来用智能体表征作为目标训练观测编码。与 PBL不同， BYOL 以自身表征的缓慢移动平均提供目标，不需要第二个网络。

使用缓慢移动平均的目标网络为在线网络产生稳定目标，这一思路受到深度强化学习[50, 51, 52, 53] 的启发。目标网络稳定了贝尔曼方程提供的自举更新，

4 本文中“自举”一词采用日常含义，而非统计学含义。

making them appealing to stabilize the bootstrap mechanism in BYOL. While most RL methods use fixed target networks, BYOL uses a weighted moving average of previous networks (as in [54]) in order to provide smoother changes in the target representation.

In the semi-supervised setting [55, 56], an unsupervised loss is combined with a classification loss over a handful of labels to ground the training [19, 20, 57, 58, 59, 60, 61, 62]. Among these methods, *mean teacher* (MT) [20] also uses a slow-moving average network, called *teacher*, to produce targets for an online network, called *student*. An $\ell_2$ consistency loss between the softmax predictions of the teacher and the student is added to the classification loss. While [20] demonstrates the effectiveness of MT in the semi-supervised learning case, in Section 5 we show that a similar approach collapses when removing the classification loss. In contrast, BYOL introduces an additional predictor on top of the online network, which prevents collapse.

Finally, in self-supervised learning, MoCo [9] uses a slow-moving average network (*momentum encoder*) to maintain consistent representations of negative pairs drawn from a memory bank. Instead, BYOL uses a moving average network to produce prediction targets as a means of stabilizing the bootstrap step. We show in Section 5 that this mere stabilizing effect can also improve existing contrastive methods.

## 3 Method

We start by motivating our method before explaining its details in Section 3.1. Many successful self-supervised learning approaches build upon the cross-view prediction framework introduced in [63]. Typically, these approaches learn representations by predicting different views (e.g., different random crops) of the same image from one another. Many such approaches cast the prediction problem directly in representation space: the representation of an augmented view of an image should be predictive of the representation of another augmented view of the same image. However, predicting directly in representation space can lead to collapsed representations: for instance, a representation that is constant across views is always fully predictive of itself. Contrastive methods circumvent this problem by reformulating the prediction problem into one of discrimination: from the representation of an augmented view, they learn to discriminate between the representation of another augmented view of the same image, and the representations of augmented views of different images. In the vast majority of cases, this prevents the training from finding collapsed representations. Yet, this discriminative approach typically requires comparing each representation of an augmented view with many negative examples, to find ones sufficiently close to make the discrimination task challenging. In this work, we thus tasked ourselves to find out whether these negative examples are indispensable to prevent collapsing while preserving high performance.

To prevent collapse, a straightforward solution is to use a fixed randomly initialized network to produce the targets for our predictions. While avoiding collapse, it empirically does not result in very good representations. Nonetheless, it is interesting to note that the representation obtained using this procedure can already be much better than the initial fixed representation. In our ablation study (Section 5), we apply this procedure by predicting a fixed randomly initialized network and achieve $18.8\%$ top-1 accuracy (Table 5a) on the linear evaluation protocol on ImageNet, whereas the randomly initialized network only achieves $1.4\%$ by itself. This experimental finding is the core motivation for BYOL: from a given representation, referred to as *target*, we can train a new, potentially enhanced representation, referred to as *online*, by predicting the target representation. From there, we can expect to build a sequence of representations of increasing quality by iterating this procedure, using subsequent online networks as new target networks for further training. In practice, BYOL generalizes this bootstrapping procedure by iteratively refining its representation, but using a slowly moving exponential average of the online network as the target network instead of fixed checkpoints.

### 3.1 Description of BYOL

BYOL's goal is to learn a representation $y_\theta$ which can then be used for downstream tasks. As described previously, BYOL uses two neural networks to learn: the *online* and *target* networks. The online network is defined by a set of weights $\theta$ and is comprised of three stages: an *encoder* $f_\theta$, a *projector* $g_\theta$ and a *predictor* $q_\theta$, as shown in Figure 2 and Figure 8. The target network has the same architecture as the online network, but uses a different set of weights $\xi$. The target network provides the regression targets to train the online network, and its parameters $\xi$ are an exponential moving average of the online parameters $\theta$ [54]. More precisely, given a target decay rate $\tau \in [0, 1]$, after each training step we perform the following update,

$$\xi \leftarrow \tau\xi + (1-\tau)\theta. \tag{1}$$

Given a set of images $\mathcal{D}$, an image $x \sim \mathcal{D}$ sampled uniformly from $\mathcal{D}$, and two distributions of image augmentations $\mathcal{T}$ and $\mathcal{T}'$, BYOL produces two augmented views $v \triangleq t(x)$ and $v' \triangleq t'(x)$ from $x$ by applying respectively image



因此适合稳定 BYOL的自举机制。多数 RL 方法使用固定目标网络； BYOL 则采用历史网络的加权移动平均（与 [54]相同），使目标表征变化更平滑。

在半监督设置 [55, 56], 中，无监督损失与少量标签上的分类损失结合，为训练提供锚点 [19, 20, 57, 58, 59, 60, 61, 62]。其中，mean teacher（MT） [20] 也使用称为“教师”的缓慢移动平均网络，为称为“学生”的在线网络产生目标。教师与学生的 softmax 预测之间的$\ell_2$ 一致性损失被加入分类损失。虽然 [20] 展示了 MT 在半监督学习中的有效性，但第5节表明，去除分类损失后，类似方法会坍塌。相比之下， BYOL 在在线网络上额外加入预测器，防止坍塌。

最后，在自监督学习中， MoCo [9] 用缓慢移动平均网络（动量编码器）保持从记忆库抽取的负样本对表征一致。而 BYOL 用移动平均网络产生预测目标，以稳定自举步骤。第5节表明，这种单纯的稳定作用也能改善已有对比方法。

## 3 方法

在第3.1节介绍细节之前，我们先解释方法的动机。许多成功的自监督方法基于 [63]提出的跨视图预测框架：通过同一图像不同视图（如不同随机裁剪）之间的相互预测学习表征。许多方法直接在表征空间中定义预测问题：图像一个增强视图的表征，应能预测另一个增强视图的表征。但直接在表征空间预测可能导致坍塌；例如，不同视图间始终不变的表征，总能完美预测自身。对比方法将预测改写为判别来绕过这一问题：从一个增强视图的表征出发，区分同一图像另一增强视图的表征与其他图像增强视图的表征。在绝大多数情况下，这能阻止训练找到坍塌表征。然而，为找到足够相近、使判别具有挑战性的负样本，这种方法通常需要将每个增强视图的表征与大量负样本比较。因此，我们试图回答：在保持高性能的同时，避免坍塌是否必须使用这些负样本?

为避免坍塌，一种直接方案是用固定、随机初始化的网络产生预测目标。虽然能避免坍塌，但实验上得到的表征并不很好。不过，有趣的是，这种方法学到的表征已经可以显著优于最初的固定表征。在消融研究（第5节）中，我们让模型预测固定、随机初始化网络的输出，在 ImageNet 线性评估协议下达到 18.8%的 top-1 准确率（表5a），而该随机初始化网络自身仅为1.4%。这一发现是 BYOL的核心动机：给定一种称为“目标”的表征，通过预测它，我们可以训练出一种称为“在线”的新表征，其质量可能更高。反复执行这一过程，将后续在线网络作为新的目标网络继续训练，便有望构造质量不断提高的表征序列。实践中， BYOL 推广了这一自举过程，迭代地改进表征，但使用在线网络的缓慢指数移动平均作为目标网络，而不是固定检查点。

### 3.1 BYOL方法说明

BYOL旨在学习表征 $y_\theta$ ，以用于下游任务。如前所述， BYOL 使用在线和目标两个神经网络学习。在线网络由权重 $\theta$ 定义，包含三个阶段：编码器 $f_\theta$、投影器 $g_\theta$ 和预测器 $q_\theta$，见图2及图8。目标网络采用与在线网络相同的架构，但使用另一组权重 $\xi$。目标网络为在线网络训练提供回归目标，其参数 $\xi$ 是在线参数 $\theta$ [54]的指数移动平均。准确地说，给定目标衰减率 $\tau \in [0, 1]$， 每次训练后执行如下更新:

$$\xi \leftarrow \tau\xi + (1-\tau)\theta. \tag{1}$$

给定图像集合 $\mathcal{D}$、从 $\mathcal{D}$中均匀抽取的图像 $x \sim \mathcal{D}$ ，以及两种增强分布$\mathcal{T}$和 $\mathcal{T}'$， BYOL 从 $x$ 生成两个增强视图 $v \triangleq t$（$x$）和 $v' \triangleq t'$（$x$），分别使用图像

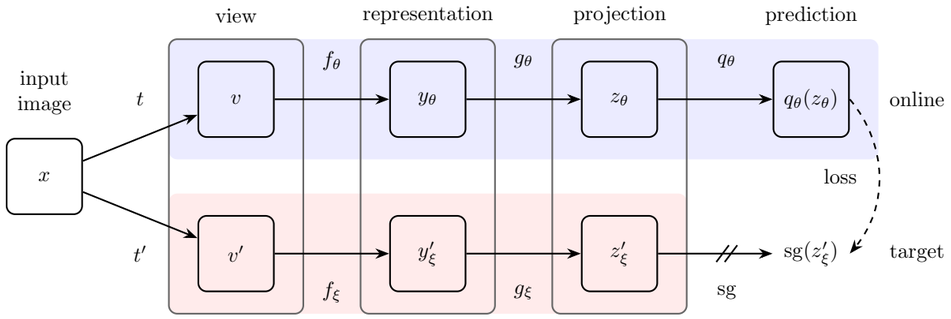



Figure 2: BYOL's architecture. BYOL minimizes a similarity loss between $q_\theta(z_\theta)$ and $\mathrm{sg}(z'_\xi)$, where $\theta$ are the trained weights, $\xi$ are an exponential moving average of $\theta$ and sg means stop-gradient. At the end of training, everything but $f_\theta$ is discarded, and $y_\theta$ is used as the image representation.

augmentations $t \sim \mathcal{T}$ and $t' \sim \mathcal{T}'$. From the first augmented view $v$, the online network outputs a *representation* $y_\theta \triangleq f_\theta(v)$ and a projection $z_\theta \triangleq g_\theta(y)$. The target network outputs $y'_\xi \triangleq f_\xi(v')$ and the *target projection* $z'_\xi \triangleq g_\xi(y')$ from the second augmented view $v'$. We then output a *prediction* $q_\theta(z_\theta)$ of $z'_\xi$ and $\ell_2$-normalize both $q_\theta(z_\theta)$ and $z'_\xi$ to $\overline{q_\theta}(z_\theta) \triangleq q_\theta(z_\theta)/\|q_\theta(z_\theta)\|_2$ and $\overline{z}'_\xi \triangleq z'_\xi/\|z'_\xi\|_2$. Note that this predictor is only applied to the online branch, making the architecture asymmetric between the online and target pipeline. Finally we define the following mean squared error between the normalized predictions and target projections,[5]

$$\mathcal{L}_{\theta,\xi} \triangleq \left\|\overline{q_\theta}(z_\theta) - \overline{z}'_\xi\right\|_2^2 = 2 - 2 \cdot \frac{\langle q_\theta(z_\theta), z'_\xi \rangle}{\|q_\theta(z_\theta)\|_2 \cdot \|z'_\xi\|_2}. \tag{2}$$

We symmetrize the loss $\mathcal{L}_{\theta,\xi}$ in Eq. 2 by separately feeding $v'$ to the online network and $v$ to the target network to compute $\widetilde{\mathcal{L}}_{\theta,\xi}$. At each training step, we perform a stochastic optimization step to minimize $\mathcal{L}^{\texttt{BYOL}}_{\theta,\xi} = \mathcal{L}_{\theta,\xi} + \widetilde{\mathcal{L}}_{\theta,\xi}$ with respect to $\theta$ only, but *not* $\xi$, as depicted by the stop-gradient in Figure 2. BYOL's dynamics are summarized as

$$\theta \leftarrow \mathrm{optimizer}\big(\theta, \nabla_\theta \mathcal{L}^{\texttt{BYOL}}_{\theta,\xi}, \eta\big), \tag{3}$$
$$\xi \leftarrow \tau\xi + (1-\tau)\theta, \tag{1}$$

where optimizer is an optimizer and $\eta$ is a learning rate.

At the end of training, we only keep the encoder $f_\theta$; as in [9]. When comparing to other methods, we consider the number of inference-time weights only in the final representation $f_\theta$. The full training procedure is summarized in Appendix A, and python pseudo-code based on the libraries JAX [64] and Haiku [65] is provided in in Appendix J.

### 3.2 Intuitions on BYOL's behavior

As BYOL does not use an explicit term to prevent collapse (such as negative examples [10]) while minimizing $\mathcal{L}^{\texttt{BYOL}}_{\theta,\xi}$ with respect to $\theta$, it may seem that BYOL should converge to a minimum of this loss with respect to $(\theta, \xi)$ (*e.g.*, a collapsed constant representation). However BYOL's target parameters $\xi$ updates are **not** in the direction of $\nabla_\xi \mathcal{L}^{\texttt{BYOL}}_{\theta,\xi}$. More generally, we hypothesize that there is no loss $L_{\theta,\xi}$ such that BYOL's dynamics is a gradient descent on $L$ jointly over $\theta, \xi$. This is similar to GANs [66], where there is no loss that is jointly minimized w.r.t. both the discriminator and generator parameters. There is therefore no *a priori* reason why BYOL's parameters would converge to a minimum of $\mathcal{L}^{\texttt{BYOL}}_{\theta,\xi}$.

While BYOL's dynamics still admit undesirable equilibria, we did not observe convergence to such equilibria in our experiments. In addition, when assuming BYOL's predictor to be optimal[6] i.e., $q_\theta = q^\star$ with

$$q^\star \triangleq \arg\min_q \mathbb{E}\Big[\big\|q(z_\theta) - z'_\xi\big\|_2^2\Big], \quad \text{where} \quad q^\star(z_\theta) = \mathbb{E}\big[z'_\xi | z_\theta\big], \tag{4}$$

[5]While we could directly predict the representation $y$ and not a projection $z$, previous work [8] have empirically shown that using this projection improves performance.

[6]For simplicity we also consider BYOL without normalization (which performs reasonably close to BYOL, see Appendix F.6) nor symmetrization



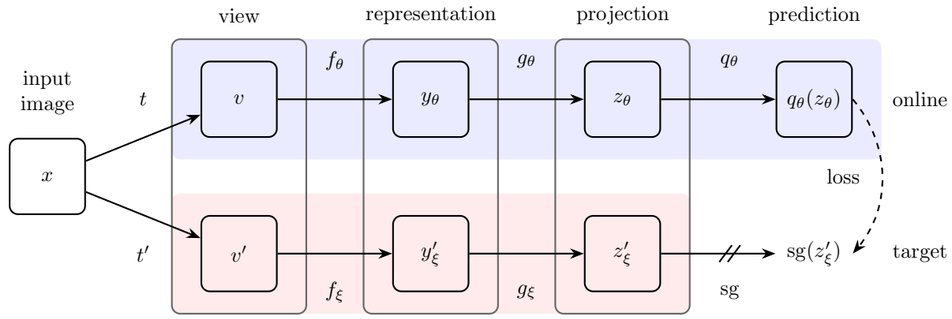



图2： BYOL架构。 BYOL 最小化 $q_\theta$（$z_\theta$）与 sg（$z'_\xi$）之间的相似性损失，其中 $\theta$ 是训练权重， $\xi$ 是 $\theta$ 的指数移动平均，sg 表示停止梯度。训练结束后，仅保留 $f_\theta$ ，并以 $y_\theta$ 作为图像表征。

增强 $t \sim \mathcal{T}$和 $t' \sim \mathcal{T}'$。在线网络从第一个增强视图 $v$输出表征$y_\theta \triangleq f_\theta$（$v$）和投影 $z_\theta \triangleq g_\theta$（$y$）。目标网络从第二个增强视图 $v'$输出 $y'_\xi \triangleq f_\xi$（$v'$）及目标投影$z'_\xi \triangleq g_\xi$（$y'$）。随后输出对 $z'_\xi$ 的预测 $q_\theta$（$z_\theta$），并对$q_\theta$（$z_\theta$）和 $z'_\xi$ 进行 $\ell_2$归一化，得到 $q_\theta$（$z_\theta$）$\triangleq q_\theta$（$z_\theta$）$/\|q_\theta(z_\theta)\|_2$ 及 $z'_\xi \triangleq z'_\xi/\|z'_\xi\|_2$。注意，预测器仅应用于在线分支，使在线和目标处理流程的架构不对称。最后，在归一化预测与目标投影之间定义如下均方误差：[5]

$$\mathcal{L}_{\theta,\xi} \triangleq \left\|\overline{q_\theta}(z_\theta) - \overline{z}'_\xi\right\|_2^2 = 2 - 2 \cdot \frac{\langle q_\theta(z_\theta), z'_\xi \rangle}{\|q_\theta(z_\theta)\|_2 \cdot \|z'_\xi\|_2}. \tag{2}$$

我们分别将 $v'$ 输入在线网络、 $v$ 输入目标网络，以

~ ~计算$\mathcal{L}_{\theta}$、$_{\xi}$，从而对式2中的损失$\mathcal{L}_{\theta}$、$_{\xi}$进行对称化。每个训练步只关于 $\theta$ 做随机优化，最小化$\mathcal{L}^{\texttt{BYOL}}_{\theta}$、$_{\xi}$ $= \mathcal{L}_{\theta}$、$_{\xi} + \mathcal{L}_{\theta}$、$_{\xi}$，不优化 $\xi$，如图2中的停止梯度所示。 BYOL的动力学概括为：

$$\theta \leftarrow \mathrm{optimizer}\big(\theta, \nabla_\theta \mathcal{L}^{\texttt{BYOL}}_{\theta,\xi}, \eta\big), \tag{3}$$
$$\xi \leftarrow \tau\xi + (1-\tau)\theta, \tag{1}$$

其中 optimizer 为优化器， $\eta$ 为学习率。

训练结束后，按照 [9]，仅保留编码器 $f_\theta$。与其他方法比较时，只统计最终表征 $f_\theta$在推理时的权重数。完整训练流程见附录A；附录J给出基于 JAX [64] 和 Haiku [65] 库的 python 伪代码。

### 3.2 理解 BYOL行为的直觉

由于 BYOL 在对 $\theta$最小化$\mathcal{L}^{\texttt{BYOL}}_{\theta}$、$_{\xi}$ 时，并未使用明确防止坍塌的项（如负样本 [10]），看起来 BYOL 应会收敛到该损失关于（$\theta$， $\xi$）的极小值（如坍塌的常量表征）。然而， BYOL的目标参数 $\xi$ 并不沿$\nabla_{\xi}\mathcal{L}^{\texttt{BYOL}}_{\theta}$、$_{\xi}$ 方向更新。更一般地，我们推测，不存在某个损失$L_{\theta}$、$_{\xi}$，使 BYOL的动力学等价于在 $\theta$、 $\xi$上联合对 $L$ 做梯度下降。这与 GAN [66], 类似：不存在一个关于判别器与生成器参数共同最小化的损失。因此，没有先验理由认为 BYOL参数会收敛到$\mathcal{L}^{\texttt{BYOL}}_{\theta}$、$_{\xi}$ 的极小值。

虽然 BYOL的动力学仍允许不理想的均衡，但实验中未观察到收敛至此。此外，假设 BYOL的预测器最优[6]，即 $q_\theta = q^\star$，其中

$$q^\star \triangleq \arg\min_q \mathbb{E}\Big[\big\|q(z_\theta) - z'_\xi\big\|_2^2\Big], \quad \text{where} \quad q^\star(z_\theta) = \mathbb{E}\big[z'_\xi | z_\theta\big], \tag{4}$$

[5]虽然可以直接预测表征 $y$ 而非投影 $z$，但已有工作 [8] 实验表明，使用投影会提高性能。[6]为简便，我们还考虑不使用归一化（性能接近 BYOL，见附录 F.6）和对称化的 BYOL 。

we hypothesize that the undesirable equilibria are unstable. Indeed, in this optimal predictor case, BYOL's updates on $\theta$ follow in expectation the gradient of the expected conditional variance (see Appendix H for details),

$$\nabla_\theta \mathbb{E}\Big[\big\|q^\star(z_\theta) - z'_\xi\big\|_2^2\Big] = \nabla_\theta \mathbb{E}\Big[\big\|\mathbb{E}\big[z'_\xi|z_\theta\big] - z'_\xi\big\|_2^2\Big] = \nabla_\theta \mathbb{E}\left[\sum_i \mathrm{Var}(z'_{\xi,i}|z_\theta)\right], \tag{5}$$

where $z'_{\xi,i}$ is the $i$-th feature of $z'_\xi$.

Note that for any random variables $X$, $Y$, and $Z$, $\mathrm{Var}(X|Y,Z) \leq \mathrm{Var}(X|Y)$. Let $X$ be the target projection, $Y$ the current online projection, and $Z$ an additional variability on top of the online projection induced by stochasticities in the training dynamics: purely discarding information from the online projection cannot decrease the conditional variance.

In particular, BYOL avoids constant features in $z_\theta$ as, for any constant $c$ and random variables $z_\theta$ and $z'_\xi$, $\mathrm{Var}(z'_\xi|z_\theta) \leq \mathrm{Var}(z'_\xi|c)$; hence our hypothesis on these collapsed constant equilibria being unstable. Interestingly, if we were to minimize $\mathbb{E}[\sum_i \mathrm{Var}(z'_{\xi,i}|z_\theta)]$ with respect to $\xi$, we would get a collapsed $z'_\xi$ as the variance is minimized for a constant $z'_\xi$. Instead, BYOL makes $\xi$ closer to $\theta$, incorporating sources of variability captured by the online projection into the target projection.

Furthemore, notice that performing a hard-copy of the online parameters $\theta$ into the target parameters $\xi$ would be enough to propagate new sources of variability. However, sudden changes in the target network might break the assumption of an optimal predictor, in which case BYOL's loss is not guaranteed to be close to the conditional variance. We hypothesize that the main role of BYOL's moving-averaged target network is to ensure the near-optimality of the predictor over training; Section 5 and Appendix I provide some empirical support of this interpretation.

### 3.3 Implementation details

**Image augmentations** BYOL uses the same set of image augmentations as in SimCLR [8]. First, a random patch of the image is selected and resized to $224 \times 224$ with a random horizontal flip, followed by a color distortion, consisting of a random sequence of brightness, contrast, saturation, hue adjustments, and an optional grayscale conversion. Finally Gaussian blur and solarization are applied to the patches. Additional details on the image augmentations are in Appendix B.

**Architecture** We use a convolutional residual network [22] with 50 layers and post-activation (ResNet-50($1\times$) v1) as our base parametric encoders $f_\theta$ and $f_\xi$. We also use deeper (50, 101, 152 and 200 layers) and wider (from $1\times$ to $4\times$) ResNets, as in [67, 48, 8]. Specifically, the representation $y$ corresponds to the output of the final average pooling layer, which has a feature dimension of 2048 (for a width multiplier of $1\times$). As in SimCLR [8], the representation $y$ is projected to a smaller space by a *multi-layer perceptron* (MLP) $g_\theta$, and similarly for the target projection $g_\xi$. This MLP consists in a linear layer with output size 4096 followed by batch normalization [68], rectified linear units (ReLU) [69], and a final linear layer with output dimension 256. Contrary to SimCLR, the output of this MLP is not batch normalized. The predictor $q_\theta$ uses the same architecture as $g_\theta$.

**Optimization** We use the LARS optimizer [70] with a cosine decay learning rate schedule [71], without restarts, over 1000 epochs, with a warm-up period of 10 epochs. We set the base learning rate to 0.2, scaled linearly [72] with the batch size (LearningRate $= 0.2 \times$ BatchSize$/256$). In addition, we use a global weight decay parameter of $1.5 \cdot 10^{-6}$ while excluding the biases and batch normalization parameters from both LARS adaptation and weight decay. For the target network, the exponential moving average parameter $\tau$ starts from $\tau_{\text{base}} = 0.996$ and is increased to one during training. Specifically, we set $\tau \triangleq 1 - (1 - \tau_{\text{base}}) \cdot (\cos(\pi k/K) + 1)/2$ with $k$ the current training step and $K$ the maximum number of training steps. We use a batch size of 4096 split over 512 Cloud TPU v3 cores. With this setup, training takes approximately 8 hours for a ResNet-50($\times 1$). All hyperparameters are summarized in Appendix J; an additional set of hyperparameters for a smaller batch size of 512 is provided in Appendix G.

## 4 Experimental evaluation

We assess the performance of BYOL's representation after self-supervised pretraining on the training set of the ImageNet ILSVRC-2012 dataset [21]. We first evaluate it on ImageNet (IN) in both linear evaluation and semi-supervised setups. We then measure its transfer capabilities on other datasets and tasks, including classification, segmentation, object detection and depth estimation. For comparison, we also report scores for a representation trained using labels from the train ImageNet subset, referred to as Supervised-IN. In Appendix E, we assess the



我们推测，不理想的均衡是不稳定的。事实上，预测器最优时， BYOL对 $\theta$ 的更新在期望上沿着期望条件方差的梯度（详见附录H）：

$$\nabla_\theta \mathbb{E}\Big[\big\|q^\star(z_\theta) - z'_\xi\big\|_2^2\Big] = \nabla_\theta \mathbb{E}\Big[\big\|\mathbb{E}\big[z'_\xi|z_\theta\big] - z'_\xi\big\|_2^2\Big] = \nabla_\theta \mathbb{E}\left[\sum_i \mathrm{Var}(z'_{\xi,i}|z_\theta)\right], \tag{5}$$

其中$z'_{\xi,i}$为 $z'_\xi$的第 $i$个特征。

注意，对任意随机变量 $X$、 $Y$和 $Z$，有 $\mathrm{Var}(X|Y,Z) \leq \mathrm{Var}(X|Y)$。设 $X$ 为目标投影， $Y$为当前在线投影， $Z$ 为训练动力学的随机性在在线投影上引入的额外变异：单纯丢弃在线投影的信息，不能减小条件方差。

具体而言， BYOL 避免了 $z_\theta$ 中的常量特征：对任意常量 $c$ 及随机变量 $z_\theta$、 $z'_\xi$，有 $\mathrm{Var}(z'_\xi|z_\theta) \leq \mathrm{Var}(z'_\xi|c)$。因此，我们推测这些坍塌的常量均衡是不稳定的。有趣的是，如果关于 $\xi$最小化 $\mathbb{E}[\sum_i \mathrm{Var}(z'_{\xi,i}|z_\theta)]$，就会得到坍塌的 $z'_\xi$，因为 $z'_\xi$为常量时方差最小。相反， BYOL 使 $\xi$ 更接近 $\theta$，将在线投影捕获的变异来源纳入目标投影。

此外，直接将在线参数 $\theta$ 复制给目标参数 $\xi$，就足以传播新的变异来源。但目标网络突然变化可能破坏预测器最优的假设，此时无法保证 BYOL损失接近条件方差。我们推测， BYOL移动平均目标网络的主要作用是在训练中保证预测器近似最优；第5节和附录I提供了部分实验证据。

### 3.3 实现细节

图像增强。BYOL 使用与 SimCLR [8]相同的增强集合。首先选取随机图像块并缩放为 $224 \times 224$，随机水平翻转，再进行颜色扰动，包括亮度、对比度、饱和度、色相调整的随机序列，以及可选灰度转换。最后进行高斯模糊和曝光反转。更多细节见附录B。

架构。基础参数化编码器 $f_\theta$ 和 $f_\xi$采用50层、激活后置的卷积残差网络 [22]，即 ResNet-50（$1\times$）v1。我们也按照 [67, 48, 8]使用更深（50、101、152和200层）、更宽（从$1\times$ 至 $4\times$）的 ResNet。具体地，表征 $y$ 为最终平均池化层的输出，宽度倍率为 $1\times$时，特征维数为2048。按照 SimCLR [8]，用多层感知机（MLP） $g_\theta$将表征 $y$ 投影到更小空间；目标投影 $g_\xi$亦然。该 MLP 包含输出维数4096的线性层，随后是批归一化 [68]、修正线性单元（ReLU） [69]，，以及输出维数256的最终线性层。与 SimCLR不同，我们不对该 MLP 的输出做批归一化。预测器 $q_\theta$ 采用与 $g_\theta$相同的架构。

优化。我们使用 LARS 优化器 [70]，采用不重启的余弦衰减学习率日程 [71]，，训练1000轮，前10轮为预热。基础学习率设为0.2，并随批量大小线性缩放 [72]（LearningRate $= 0.2 \times$BatchSize$/256$）。全局权重衰减系数为$1.5 \cdot 10^{-6}$，但偏置和批归一化参数既不参与 LARS 自适应，也不使用权重衰减。目标网络的指数移动平均参数 $\tau$从 $\tau_{\text{base}} = 0.996$ 开始，随训练升至1。具体设置为 $\tau \triangleq 1 - (1 - \tau_{\text{base}}) \cdot (\cos(\pi k/K) + 1)/2$，其中 $k$ 为当前训练步， $K$ 为最大训练步数。批量大小为4096，分布在512个 Cloud TPU v3 核心上。该设置下，ResNet-50（$\times 1$）训练约需8小时。附录J汇总了所有超参数；附录G还提供了适用于较小批量512的超参数。

## 4 实验评估

我们在 ImageNet ILSVRC-2012 [21]训练集上进行自监督预训练后评估 BYOL的表征。首先在 ImageNet（IN）上进行线性和半监督评估，再衡量其迁移到其他数据集及分类、分割、目标检测和深度估计任务的能力。作为比较，我们也报告使用 ImageNet train 子集标签训练的表征，记为 Supervised-IN。附录E评估其

generality of BYOL by pretraining a representation on the Places365-Standard dataset [73] before reproducing this evaluation protocol.

**Linear evaluation on ImageNet** We first evaluate BYOL's representation by training a linear classifier on top of the frozen representation, following the procedure described in [48, 74, 41, 10, 8], and appendix C.1; we report top-1 and top-5 accuracies in % on the test set in Table 1. With a standard ResNet-50 (×1) BYOL obtains 74.3% top-1 accuracy (91.6% top-5 accuracy), which is a 1.3% (resp. 0.5%) improvement over the previous self-supervised state of the art [12]. This tightens the gap with respect to the supervised baseline of [8], 76.5%, but is still significantly below the stronger supervised baseline of [75], 78.9%. With deeper and wider architectures, BYOL consistently outperforms the previous state of the art (Appendix C.2), and obtains a best performance of 79.6% top-1 accuracy, ranking higher than previous self-supervised approaches. On a ResNet-50 (4×) BYOL achieves 78.6%, similar to the 78.9% of the best supervised baseline in [8] for the same architecture.

| Method | Top-1 | Top-5 |
|---|---|---|
| Local Agg. | 60.2 | - |
| PIRL [35] | 63.6 | - |
| CPC v2 [32] | 63.8 | 85.3 |
| CMC [11] | 66.2 | 87.0 |
| SimCLR [8] | 69.3 | 89.0 |
| MoCo v2 [37] | 71.1 | - |
| InfoMin Aug. [12] | 73.0 | 91.1 |
| BYOL (ours) | **74.3** | **91.6** |

(a) ResNet-50 encoder.

| Method | Architecture | Param. | Top-1 | Top-5 |
|---|---|---|---|---|
| SimCLR [8] | ResNet-50 (2×) | 94M | 74.2 | 92.0 |
| CMC [11] | ResNet-50 (2×) | 94M | 70.6 | 89.7 |
| BYOL (ours) | ResNet-50 (2×) | 94M | 77.4 | 93.6 |
| CPC v2 [32] | ResNet-161 | 305M | 71.5 | 90.1 |
| MoCo [9] | ResNet-50 (4×) | 375M | 68.6 | - |
| SimCLR [8] | ResNet-50 (4×) | 375M | 76.5 | 93.2 |
| BYOL (ours) | ResNet-50 (4×) | 375M | 78.6 | 94.2 |
| BYOL (ours) | ResNet-200 (2×) | 250M | **79.6** | **94.8** |

(b) Other ResNet encoder architectures.

Table 1: Top-1 and top-5 accuracies (in %) under linear evaluation on ImageNet.

**Semi-supervised training on ImageNet** Next, we evaluate the performance obtained when fine-tuning BYOL's representation on a classification task with a small subset of ImageNet's train set, this time using label information. We follow the semi-supervised protocol of [74, 76, 8, 32] detailed in Appendix C.1, and use the same fixed splits of respectively 1% and 10% of ImageNet labeled training data as in [8]. We report both top-1 and top-5 accuracies on the test set in Table 2. BYOL consistently outperforms previous approaches across a wide range of architectures. Additionally, as detailed in Appendix C.1, BYOL reaches 77.7% top-1 accuracy with ResNet-50 when fine-tuning over 100% of ImageNet labels.

| Method | Top-1 | | Top-5 | |
|---|---|---|---|---|
| | 1% | 10% | 1% | 10% |
| Supervised [77] | 25.4 | 56.4 | 48.4 | 80.4 |
| InstDisc | - | - | 39.2 | 77.4 |
| PIRL [35] | - | - | 57.2 | 83.8 |
| SimCLR [8] | 48.3 | 65.6 | 75.5 | 87.8 |
| BYOL (ours) | **53.2** | **68.8** | **78.4** | **89.0** |

(a) ResNet-50 encoder.

| Method | Architecture | Param. | Top-1 | | Top-5 | |
|---|---|---|---|---|---|---|
| | | | 1% | 10% | 1% | 10% |
| CPC v2 [32] | ResNet-161 | 305M | - | - | 77.9 | 91.2 |
| SimCLR [8] | ResNet-50 (2×) | 94M | 58.5 | 71.7 | 83.0 | 91.2 |
| BYOL (ours) | ResNet-50 (2×) | 94M | 62.2 | 73.5 | 84.1 | 91.7 |
| SimCLR [8] | ResNet-50 (4×) | 375M | 63.0 | 74.4 | 85.8 | 92.6 |
| BYOL (ours) | ResNet-50 (4×) | 375M | 69.1 | 75.7 | 87.9 | 92.5 |
| BYOL (ours) | ResNet-200 (2×) | 250M | **71.2** | **77.7** | **89.5** | **93.7** |

(b) Other ResNet encoder architectures.

Table 2: Semi-supervised training with a fraction of ImageNet labels.

**Transfer to other classification tasks** We evaluate our representation on other classification datasets to assess whether the features learned on ImageNet (IN) are generic and thus useful across image domains, or if they are ImageNet-specific. We perform linear evaluation and fine-tuning on the same set of classification tasks used in [8, 74], and carefully follow their evaluation protocol, as detailed in Appendix D. Performance is reported using standard metrics for each benchmark, and results are provided on a held-out test set after hyperparameter selection on a validation set. We report results in Table 3, both for linear evaluation and fine-tuning. BYOL outperforms SimCLR on all benchmarks and the Supervised-IN baseline on 7 of the 12 benchmarks, providing only slightly worse performance on the 5 remaining benchmarks. BYOL's representation can be transferred over to small images, e.g., CIFAR [78], landscapes, e.g., SUN397 [79] or VOC2007 [80], and textures, e.g., DTD [81].



通用性：在 Places365-Standard [73] 上预训练 BYOL 的表征，再重复上述评估协议。

ImageNet 线性评估。我们首先按照 [48, 74, 41, 10, 8], 及附录 C.1，在冻结的表征上训练线性分类器，评估 BYOL的表征；表1报告 test 集上的 top-1 和 top-5 准确率（%）。使用标准 ResNet-50（×1）时， BYOL 达到74.3%的 top-1 准确率和91.6%的 top-5 准确率，分别比此前最优的自监督方法 [12]高1.3和0.5个百分点。这缩小了与 [8]中76.5%的有监督基线的差距，但仍明显低于 [75]中78.9%的更强有监督基线。采用更深、更宽的架构时， BYOL 始终超过此前最优结果（附录 C.2），最佳 top-1 准确率达到79.6%，高于已有自监督方法。采用 ResNet-50（4×）时， BYOL 达到78.6%，接近 [8] 中同架构最佳有监督基线的78.9%。

| 方法 | Top-1 | Top-5 |
|---|---|---|
| Local Agg. | 60.2 | - |
| PIRL [35] | 63.6 | - |
| CPC v2 [32] | 63.8 | 85.3 |
| CMC [11] | 66.2 | 87.0 |
| SimCLR [8] | 69.3 | 89.0 |
| MoCov2 [37] | 71.1 | - |
| InfoMin Aug. [12] | 73.0 | 91.1 |
| BYOL （本文） | **74.3** | **91.6** |

（a）ResNet-50 编码器。

| 方法 | 架构 | 参数量 | Top-1 | Top-5 |
|---|---|---|---|---|
| SimCLR [8] | ResNet-50 (2×) | 94M | 74.2 | 92.0 |
| CMC [11] | ResNet-50 (2×) | 94M | 70.6 | 89.7 |
| BYOL （本文） | ResNet-50 (2×) | 94M | 77.4 | 93.6 |
| CPC v2 [32] | ResNet-161 | 305M | 71.5 | 90.1 |
| MoCo [9] | ResNet-50 (4×) | 375M | 68.6 | - |
| SimCLR [8] | ResNet-50 (4×) | 375M | 76.5 | 93.2 |
| BYOL （本文） | ResNet-50 (4×) | 375M | 78.6 | 94.2 |
| BYOL （本文） | ResNet-200 (2×) | 250M | **79.6** | **94.8** |

（b）其他 ResNet 编码器架构。

表1：ImageNet 线性评估的 top-1 与 top-5 准确率（%）。

ImageNet 半监督训练。接下来，利用 ImageNet train 集的一小部分及其标签，在分类任务上微调 BYOL的表征并评估性能。我们遵循 [74, 76, 8, 32] 的半监督协议（详见附录 C.1），采用与 [8]相同的固定划分，分别使用 ImageNet 有标签训练数据的1%和10%。表2报告 test 集的 top-1 和 top-5 准确率。 BYOL 在多种架构上始终优于已有方法。此外，如附录 C.1 所述，使用100%的 ImageNet 标签微调时， BYOL 配合 ResNet-50 达到77.7%的 top-1 准确率。

| Method | Top-1 | | Top-5 | |
|---|---|---|---|---|
| | 1% | 10% | 1% | 10% |
| Supervised [77] | 25.4 | 56.4 | 48.4 | 80.4 |
| InstDisc | - | - | 39.2 | 77.4 |
| PIRL [35] | - | - | 57.2 | 83.8 |
| SimCLR [8] | 48.3 | 65.6 | 75.5 | 87.8 |
| BYOL (ours) | **53.2** | **68.8** | **78.4** | **89.0** |

(a) ResNet-50 encoder.

| Method | Architecture | Param. | Top-1 | | Top-5 | |
|---|---|---|---|---|---|---|
| | | | 1% | 10% | 1% | 10% |
| CPC v2 [32] | ResNet-161 | 305M | - | - | 77.9 | 91.2 |
| SimCLR [8] | ResNet-50 (2×) | 94M | 58.5 | 71.7 | 83.0 | 91.2 |
| BYOL (ours) | ResNet-50 (2×) | 94M | 62.2 | 73.5 | 84.1 | 91.7 |
| SimCLR [8] | ResNet-50 (4×) | 375M | 63.0 | 74.4 | 85.8 | 92.6 |
| BYOL (ours) | ResNet-50 (4×) | 375M | 69.1 | 75.7 | 87.9 | 92.5 |
| BYOL (ours) | ResNet-200 (2×) | 250M | **71.2** | **77.7** | **89.5** | **93.7** |

(b) Other ResNet encoder architectures.

表2：使用部分 ImageNet 标签的半监督训练。

迁移到其他分类任务。我们在其他分类数据集上评估表征，以判断 ImageNet（IN）所学特征是否通用、能跨图像领域使用，还是 ImageNet 特有。我们采用 [8, 74], 使用的同一组分类任务进行线性评估和微调，并严格遵循其评估协议（见附录D）。各基准使用标准指标，在验证集选取超参数后，于留出的 test 集报告结果。表3展示线性评估与微调结果。 BYOL 在所有基准上优于SimCLR ，在12个基准中的7个上优于 Supervised-IN 基线，在其余5个上仅略低。 BYOL的表征可以迁移到小尺寸图像（如 CIFAR [78], ）、场景（如 SUN397 [79] 或 VOC2007 [80], ）以及纹理（如 DTD [81]）。

| Method | Food101 | CIFAR10 | CIFAR100 | Birdsnap | SUN397 | Cars | Aircraft | VOC2007 | DTD | Pets | Caltech-101 | Flowers |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Linear evaluation:* | | | | | | | | | | | | |
| BYOL (ours) | **75.3** | 91.3 | **78.4** | **57.2** | **62.2** | **67.8** | 60.6 | 82.5 | 75.5 | 90.4 | 94.2 | **96.1** |
| SimCLR (repro) | 72.8 | 90.5 | 74.4 | 42.4 | 60.6 | 49.3 | 49.8 | 81.4 | **75.7** | 84.6 | 89.3 | 92.6 |
| SimCLR [8] | 68.4 | 90.6 | 71.6 | 37.4 | 58.8 | 50.3 | 50.3 | 80.5 | 74.5 | 83.6 | 90.3 | 91.2 |
| Supervised-IN [8] | 72.3 | **93.6** | 78.3 | 53.7 | 61.9 | 66.7 | **61.0** | **82.8** | 74.9 | **91.5** | **94.5** | 94.7 |
| *Fine-tuned:* | | | | | | | | | | | | |
| BYOL (ours) | **88.5** | **97.8** | 86.1 | **76.3** | 63.7 | 91.6 | **88.1** | **85.4** | **76.2** | 91.7 | **93.8** | 97.0 |
| SimCLR (repro) | 87.5 | 97.4 | 85.3 | 75.0 | 63.9 | 91.4 | 87.6 | 84.5 | 75.4 | 89.4 | 91.7 | 96.6 |
| SimCLR [8] | 88.2 | 97.7 | 85.9 | 75.9 | 63.5 | 91.3 | 88.1 | 84.1 | 73.2 | 89.2 | 92.1 | 97.0 |
| Supervised-IN [8] | 88.3 | 97.5 | **86.4** | 75.8 | **64.3** | **92.1** | 86.0 | 85.0 | 74.6 | **92.1** | 93.3 | **97.6** |
| Random init [8] | 86.9 | 95.9 | 80.2 | 76.1 | 53.6 | 91.4 | 85.9 | 67.3 | 64.8 | 81.5 | 72.6 | 92.0 |

Table 3: Transfer learning results from ImageNet (IN) with the standard ResNet-50 architecture.

**Transfer to other vision tasks** We evaluate our representation on different tasks relevant to computer vision practitioners, namely semantic segmentation, object detection and depth estimation. With this evaluation, we assess whether BYOL's representation generalizes beyond classification tasks.

We first evaluate BYOL on the VOC2012 semantic segmentation task as detailed in Appendix D.4, where the goal is to classify each pixel in the image [7]. We report the results in Table 4a. BYOL outperforms both the Supervised-IN baseline (+1.9 mIoU) and SimCLR (+1.1 mIoU).

Similarly, we evaluate on object detection by reproducing the setup in [9] using a Faster R-CNN architecture [82], as detailed in Appendix D.5. We fine-tune on trainval2007 and report results on test2007 using the standard $AP_{50}$ metric; BYOL is significantly better than the Supervised-IN baseline (+3.1 $AP_{50}$) and SimCLR (+2.3 $AP_{50}$).

Finally, we evaluate on depth estimation on the NYU v2 dataset, where the depth map of a scene is estimated given a single RGB image. Depth prediction measures how well a network represents geometry, and how well that information can be localized to pixel accuracy [40]. The setup is based on [83] and detailed in Appendix D.6. We evaluate on the commonly used test subset of 654 images and report results using several common metrics in Table 4b: relative (rel) error, root mean squared (rms) error, and the percent of pixels (pct) where the error, $\max(d_{gt}/d_p, d_p/d_{gt})$, is below $1.25^n$ thresholds where $d_p$ is the predicted depth and $d_{gt}$ is the ground truth depth [40]. BYOL is better or on par with other methods for each metric. For instance, the challenging pct.<1.25 measure is respectively improved by +3.5 points and +1.3 points compared to supervised and SimCLR baselines.

| Method | $AP_{50}$ | mIoU |
|---|---|---|
| Supervised-IN [9] | 74.4 | 74.4 |
| MoCo [9] | 74.9 | 72.5 |
| SimCLR (repro) | 75.2 | 75.2 |
| BYOL (ours) | **77.5** | **76.3** |

(a) Transfer results in semantic segmentation and object detection.

| | Higher better | | | Lower better | |
|---|---|---|---|---|---|
| Method | pct.<1.25 | pct.<$1.25^2$ | pct.<$1.25^3$ | rms | rel |
| Supervised-IN [83] | 81.1 | 95.3 | 98.8 | 0.573 | **0.127** |
| SimCLR (repro) | 83.3 | 96.5 | 99.1 | 0.557 | 0.134 |
| BYOL (ours) | **84.6** | **96.7** | **99.1** | **0.541** | 0.129 |

(b) Transfer results on NYU v2 depth estimation.

Table 4: Results on transferring BYOL's representation to other vision tasks.

## 5 Building intuitions with ablations

We present ablations on BYOL to give an intuition of its behavior and performance. For reproducibility, we run each configuration of parameters over three seeds, and report the average performance. We also report the half difference between the best and worst runs when it is larger than 0.25. Although previous works perform ablations at 100 epochs [8, 12], we notice that relative improvements at 100 epochs do not always hold over longer training. For this reason, we run ablations over 300 epochs on 64 TPU v3 cores, which yields consistent results compared to our baseline training of 1000 epochs. For all the experiments in this section, we set the initial learning rate to 0.3 with batch size 4096, the weight decay to $10^{-6}$ as in SimCLR [8] and the base target decay rate $\tau_{\text{base}}$ to 0.99. In this section we report results in top-1 accuracy on ImageNet under the linear evaluation protocol as in Appendix C.1.

**Batch size** Among contrastive methods, the ones that draw negative examples from the minibatch suffer performance drops when their batch size is reduced. BYOL does not use negative examples and we expect it to be more



| 方法 | Food101 | CIFAR10 | CIFAR100 | Birdsnap | SUN397 | Cars | Aircraft | VOC2007 | DTD | Pets | Caltech-101 | Flowers |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 线性评估： | | | | | | | | | | | | |
| BYOL （本文） | **75.3** | 91.3 | **78.4** | **57.2** | **62.2** | **67.8** | 60.6 | 82.5 | 75.5 | 90.4 | 94.2 | **96.1** |
| SimCLR （复现） | 72.8 | 90.5 | 74.4 | 42.4 | 60.6 | 49.3 | 49.8 | 81.4 | **75.7** | 84.6 | 89.3 | 92.6 |
| SimCLR [8] | 68.4 | 90.6 | 71.6 | 37.4 | 58.8 | 50.3 | 50.3 | 80.5 | 74.5 | 83.6 | 90.3 | 91.2 |
| Supervised-IN [8] | 72.3 | **93.6** | 78.3 | 53.7 | 61.9 | 66.7 | **61.0** | **82.8** | 74.9 | **91.5** | **94.5** | 94.7 |
| 微调： | | | | | | | | | | | | |
| BYOL （本文） | **88.5** | **97.8** | 86.1 | **76.3** | 63.7 | 91.6 | **88.1** | **85.4** | **76.2** | 91.7 | **93.8** | 97.0 |
| SimCLR （复现） | 87.5 | 97.4 | 85.3 | 75.0 | 63.9 | 91.4 | 87.6 | 84.5 | 75.4 | 89.4 | 91.7 | 96.6 |
| SimCLR [8] | 88.2 | 97.7 | 85.9 | 75.9 | 63.5 | 91.3 | 88.1 | 84.1 | 73.2 | 89.2 | 92.1 | 97.0 |
| Supervised-IN [8] | 88.3 | 97.5 | **86.4** | 75.8 | **64.3** | **92.1** | 86.0 | 85.0 | 74.6 | **92.1** | 93.3 | **97.6** |
| 随机初始化 [8] | 86.9 | 95.9 | 80.2 | 76.1 | 53.6 | 91.4 | 85.9 | 67.3 | 64.8 | 81.5 | 72.6 | 92.0 |

Table 3：标准 ResNet-50 架构下，从 ImageNet（IN）迁移的结果。

迁移到其他视觉任务。我们在语义分割、目标检测及深度估计这些实际视觉任务上评估表征，以判断BYOL的表征能否泛化到分类之外。

首先在 VOC2012 语义分割任务上评估 BYOL ，详见附录 D.4；任务目标是对图像每个像素分类 [7]。表4a显示， BYOL 优于 Supervised-IN 基线（高+1.9 mIoU）及 SimCLR （高+1.1 mIoU）。

类似地，按照 [9] 复现实验，用 Faster R-CNN [82],评估目标检测，详见附录 D.5。我们在 trainval2007 上微调，在 test2007 上用标准 $AP_{50}$ 指标报告结果； BYOL 明显优于 Supervised-IN 基线（高+3.1 $AP_{50}$）及 SimCLR（高+2.3 $AP_{50}$）。

最后，在 NYU v2 上评估深度估计：给定单张 RGB 图像，估计场景深度图。深度预测衡量网络对几何的表征能力，以及能否将这些信息定位到像素精度 [40]。设置基于 [83]，详见附录 D.6。我们在常用的654张图像的 test 子集上评估，表4b报告相对（rel）误差、均方根（rms）误差，以及误差 max（$d_{gt}/d_p$， $d_p/d_{gt}$）低于 $1.25^n$ 阈值的像素百分比（pct）；其中 $d_p$ 为预测深度， $d_{gt}$ 为真实深度 [40]。BYOL在每项指标上均优于或相当于其他方法。例如，在较难的 pct.<1.25指标上，相比有监督和 SimCLR 基线，分别提高 +3.5 和 +1.3 个百分点。

| 方法 | $AP_{50}$ | mIoU |
|---|---|---|
| Supervised-IN [9] | 74.4 | 74.4 |
| MoCo [9] | 74.9 | 72.5 |
| SimCLR （复现） | 75.2 | 75.2 |
| BYOL （本文） | **77.5** | **76.3** |

（a）语义分割及目标检测迁移结果。

| | 越高越好 | | | 越低越好 | |
|---|---|---|---|---|---|
| 方法 | pct.<1.25 | pct.<$1.25^2$ | pct.<$1.25^3$ | rms | rel |
| Supervised-IN [83] | 81.1 | 95.3 | 98.8 | 0.573 | **0.127** |
| SimCLR （复现） | 83.3 | 96.5 | 99.1 | 0.557 | 0.134 |
| BYOL （本文） | **84.6** | **96.7** | **99.1** | **0.541** | 0.129 |

（b）NYU v2 深度估计迁移结果。

表4：将 BYOL表征迁移到其他视觉任务的结果。

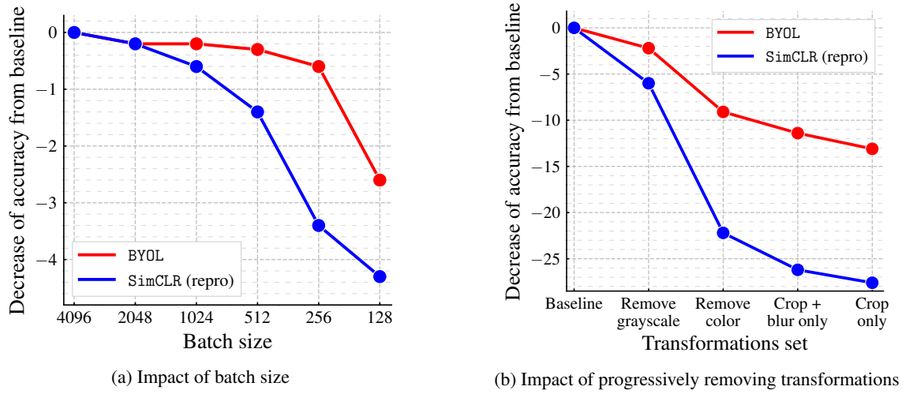
## 5 通过消融建立直觉

我们通过 BYOL 的消融实验直观说明其行为和性能。为保证可复现，每组参数配置运行3个随机种子，报告平均性能；若最好与最差运行结果之差的一半超过0.25，也一并报告。已有研究在100轮时做消融 [8, 12], ，但我们发现，100轮时的相对改进未必在更长训练中保持。因此，我们在64个 TPU v3 核心上训练300轮进行消融，结果与1000轮的基线训练一致。本节所有实验均使用初始学习率0.3、批量大小4096，按 SimCLR [8] 将权重衰减设为 $10^{-6}$，基础目标衰减率 $\tau_{\text{base}}$ 设为0.99。按照附录 C.1 的线性评估协议，报告 ImageNet top-1 准确率。

批量大小。从小批量中获取负样本的对比方法，在批量减小时会损失性能。 BYOL 不用负样本，我们预期它会

robust to smaller batch sizes. To empirically verify this hypothesis, we train both BYOL and SimCLR using different batch sizes from 128 to 4096. To avoid re-tuning other hyperparameters, we average gradients over $N$ consecutive steps before updating the online network when reducing the batch size by a factor $N$. The target network is updated once every $N$ steps, after the update of the online network; we accumulate the $N$-steps in parallel in our runs.

As shown in Figure 3a, the performance of SimCLR rapidly deteriorates with batch size, likely due to the decrease in the number of negative examples. In contrast, the performance of BYOL remains stable over a wide range of batch sizes from 256 to 4096, and only drops for smaller values due to batch normalization layers in the encoder.[7]




(a) Impact of batch size

(b) Impact of progressively removing transformations

Figure 3: Decrease in top-1 accuracy (in % points) of BYOL and our own reproduction of SimCLR at 300 epochs, under linear evaluation on ImageNet.

**Image augmentations** Contrastive methods are sensitive to the choice of image augmentations. For instance, SimCLR does not work well when removing color distortion from its image augmentations. As an explanation, SimCLR shows that crops of the same image mostly share their color histograms. At the same time, color histograms vary across images. Therefore, when a contrastive task only relies on random crops as image augmentations, it can be mostly solved by focusing on color histograms alone. As a result the representation is not incentivized to retain information beyond color histograms. To prevent that, SimCLR adds color distortion to its set of image augmentations. Instead, BYOL is incentivized to keep any information captured by the target representation into its online network, to improve its predictions. Therefore, even if augmented views of a same image share the same color histogram, BYOL is still incentivized to retain additional features in its representation. For that reason, we believe that BYOL is more robust to the choice of image augmentations than contrastive methods.

Results presented in Figure 3b support this hypothesis: the performance of BYOL is much less affected than the performance of SimCLR when removing color distortions from the set of image augmentations ($-9.1$ accuracy points for BYOL, $-22.2$ accuracy points for SimCLR). When image augmentations are reduced to mere random crops, BYOL still displays good performance ($59.4\%$, *i.e.* $-13.1$ points from $72.5\%$ ), while SimCLR loses more than a third of its performance ($40.3\%$, *i.e.* $-27.6$ points from $67.9\%$). We report additional ablations in Appendix F.3.

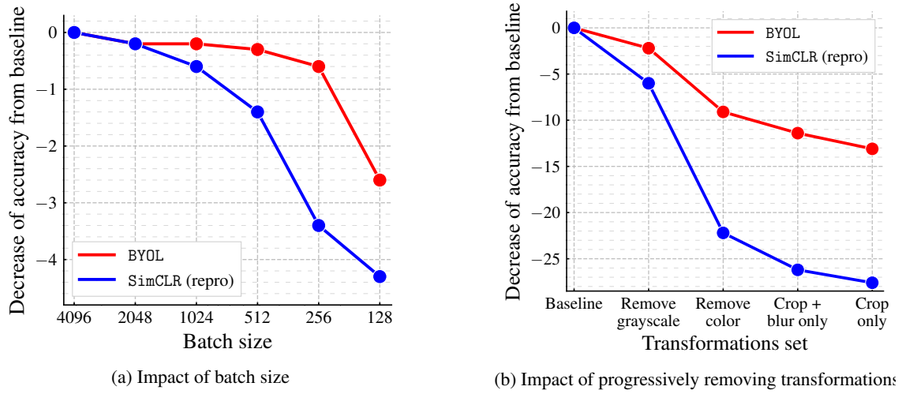
**Bootstrapping** BYOL uses the projected representation of a target network, whose weights are an exponential moving average of the weights of the online network, as target for its predictions. This way, the weights of the target network represent a delayed and more stable version of the weights of the online network. When the target decay rate is 1, the target network is never updated, and remains at a constant value corresponding to its initialization. When the target decay rate is 0, the target network is instantaneously updated to the online network at each step. There is a trade-off between updating the targets too often and updating them too slowly, as illustrated in Table 5a. Instantaneously updating the target network ($\tau = 0$) destabilizes training, yielding very poor performance while never updating the target ($\tau = 1$) makes the training stable but prevents iterative improvement, ending with low-quality final representation. All values of the decay rate between 0.9 and 0.999 yield performance above $68.4\%$ top-1 accuracy at 300 epochs.

[7]The only dependency on batch size in our training pipeline sits within the batch normalization layers.



对较小批量更稳健。为验证这一假设，我们用128至4096的不同批量训练 BYOL 与 SimCLR 。为避免重新调整其他超参数，批量缩小 $N$倍时，先平均连续 $N$ 步梯度，再更新在线网络。在线网络更新后，每 $N$ 步更新一次目标网络；实验中并行累积这 $N$步。

图3a显示， SimCLR 的性能随批量变小迅速下降，可能是因为负样本数减少。相比之下， BYOL 在256至4096的广泛批量范围内性能稳定，仅在更小批量时因编码器中的批归一化层而下降。[7]




(a) Impact of batch size

(b) Impact of progressively removing transformations

图3：训练300轮后， BYOL 与自行复现的 SimCLR 在 ImageNet 线性评估中 top-1 准确率的下降（百分点）。

图像增强。对比方法对增强的选择敏感。例如，去掉颜色扰动后，SimCLR 表现不佳。SimCLR 解释道，同一图像的不同裁剪通常具有相近的颜色直方图，而不同图像的颜色直方图则不同。因此，仅使用随机裁剪时，对比任务主要靠颜色直方图就能解决，表征便没有动力保留颜色直方图之外的信息。为避免这一点， SimCLR 加入颜色扰动。相比之下， BYOL 为提高预测质量，会被驱动在在线网络中保留目标表征捕获的任何信息。因此，即使同一图像的增强视图具有相同颜色直方图， BYOL 仍有动力在表征中保留额外特征。基于此，我们认为 BYOL 对增强选择比对比方法更稳健。

图3b结果支持上述假设：去除颜色扰动时， BYOL的性能受影响远小于 SimCLR ；准确率变化分别为$-9.1$个百分点（ BYOL）和 $-22.2$ 个百分点（ SimCLR）。增强仅剩随机裁剪时，BYOL 仍表现良好（59.4%，相对72.5%的变化为 $-13.1$ 个百分点），而 SimCLR 损失了超过三分之一的性能（40.3%，相对67.9%的变化为 $-27.6$ 个百分点）。附录 F.3 给出更多消融实验。

自举。BYOL 以目标网络的投影表征作为预测目标；目标网络权重是在线网络权重的指数移动平均。因此，目标权重是在线权重的延迟、较稳定版本。目标衰减率为1时，目标网络永不更新，始终保持初始化值；为0时，每一步都立即更新为在线网络。目标更新过于频繁与过于缓慢之间存在权衡，见表5a。即时更新目标网络（$\tau = 0$）会使训练不稳定、性能很差；从不更新目标（$\tau = 1$）虽然训练稳定，却无法迭代改进，最终表征质量较低。训练300轮时，0.9至0.999之间所有衰减率均得到超过68.4%的 top-1 准确率。

7 训练流程对批量大小的依赖仅存在于批归一化层中。

| Target | $\tau_{\text{base}}$ | Top-1 |
|---|---|---|
| Constant random network | 1 | $18.8_{\pm 0.7}$ |
| Moving average of online | 0.999 | 69.8 |
| Moving average of online | 0.99 | **72.5** |
| Moving average of online | 0.9 | 68.4 |
| Stop gradient of online[†] | 0 | 0.3 |

(a) Results for different target modes. [†]In the *stop gradient of online*, $\tau = \tau_{\text{base}} = 0$ is kept constant throughout training.

| Method | Predictor | Target network | $\beta$ | Top-1 |
|---|---|---|---|---|
| BYOL | ✓ | ✓ | 0 | **72.5** |
| – | ✓ | ✓ | 1 | 70.9 |
| – | | ✓ | 1 | 70.7 |
| SimCLR | | | 1 | 69.4 |
| – | ✓ | | 1 | 69.1 |
| – | ✓ | | 0 | 0.3 |
| – | | ✓ | 0 | 0.2 |
| – | | | 0 | 0.1 |

(b) Intermediate variants between BYOL and SimCLR.

Table 5: Ablations with top-1 accuracy (in %) at 300 epochs under linear evaluation on ImageNet.

**Ablation to contrastive methods** In this subsection, we recast SimCLR and BYOL using the same formalism to better understand where the improvement of BYOL over SimCLR comes from. Let us consider the following objective that extends the InfoNCE objective [10, 84] (see Appendix F.4),

$$\text{InfoNCE}_\theta^{\alpha,\beta} \triangleq \frac{2}{B}\sum_{i=1}^{B} S_\theta(v_i, v_i') - \beta \cdot \frac{2\alpha}{B}\sum_{i=1}^{B} \ln\left(\sum_{j\neq i} \exp\frac{S_\theta(v_i, v_j)}{\alpha} + \sum_j \exp\frac{S_\theta(v_i, v_j')}{\alpha}\right), \tag{6}$$

where $\alpha > 0$ is a fixed temperature, $\beta \in [0,1]$ a weighting coefficient, $B$ the batch size, $v$ and $v'$ are batches of augmented views where for any batch index $i$, $v_i$ and $v_i'$ are augmented views from the same image; the real-valued function $S_\theta$ quantifies pairwise similarity between augmented views. For any augmented view $u$ we denote $z_\theta(u) \triangleq f_\theta(g_\theta(u))$ and $z_\xi(u) \triangleq f_\xi(g_\xi(u))$. For given $\phi$ and $\psi$, we consider the normalized dot product

$$S_\theta(u_1, u_2) \triangleq \frac{\langle \phi(u_1), \psi(u_2)\rangle}{\|\phi(u_1)\|_2 \cdot \|\psi(u_2)\|_2}. \tag{7}$$

Up to minor details (cf. Appendix F.5), we recover the SimCLR loss with $\phi(u_1) = z_\theta(u_1)$ (no predictor), $\psi(u_2) = z_\theta(u_2)$ (no target network) and $\beta = 1$. We recover the BYOL loss when using a predictor and a target network, *i.e.*, $\phi(u_1) = p_\theta(z_\theta(u_1))$ and $\psi(u_2) = z_\xi(u_2)$ with $\beta = 0$. To evaluate the influence of the target network, the predictor and the coefficient $\beta$, we perform an ablation over them. Results are presented in Table 5b and more details are given in Appendix F.4.

The only variant that performs well without negative examples (i.e., with $\beta = 0$) is BYOL, using *both* a bootstrap target network *and* a predictor. Adding the negative pairs to BYOL's loss without re-tuning the temperature parameter hurts its performance. In Appendix F.4, we show that we can add back negative pairs and still match the performance of BYOL with proper tuning of the temperature.

Simply adding a target network to SimCLR already improves performance (+1.6 points). This sheds new light on the use of the target network in MoCo [9], where the target network is used to provide more negative examples. Here, we show that by mere stabilization effect, even when using the same number of negative examples, using a target network is beneficial. Finally, we observe that modifying the architecture of $S_\theta$ to include a predictor only mildly affects the performance of SimCLR.

**Network hyperparameters** In Appendix F, we explore how other network parameters may impact BYOL's performance. We iterate over multiple weight decays, learning rates, and projector/encoder architectures to observe that small hyperparameter changes do not drastically alter the final score. We note that removing the weight decay in either BYOL or SimCLR leads to network divergence, emphasizing the need for weight regularization in the self-supervised setting. Furthermore, we observe that changing the scaling factor in the network initialization [85] did not impact the performance (higher than 72% top-1 accuracy).

**Relationship with Mean Teacher** Another semi-supervised approach, Mean Teacher (MT) [20], complements a supervised loss on few labels with an additional consistency loss. In [20], this consistency loss is the $\ell_2$ distance between the logits from a *student* network, and those of a temporally averaged version of the student network, called *teacher*. Removing the predictor in BYOL results in an unsupervised version of MT with no classification loss that



| 目标 | $\tau_{\text{base}}$ | Top-1 |
|---|---|---|
| 固定的随机网络 | 1 | $18.8_{\pm 0.7}$ |
| 在线网络的移动平均 | 0.999 | 69.8 |
| 在线网络的移动平均 | 0.99 | **72.5** |
| 在线网络的移动平均 | 0.9 | 68.4 |
| 在线分支停止梯度[†] | 0 | 0.3 |

（a）不同目标模式的结果。[†]在线分支停止梯度的设置中，整个训练过程的 $\tau = \tau_{\text{base}} = 0$ 保持不变。

| 方法 | 预测器 | 目标网络 | $\beta$ | Top-1 |
|---|---|---|---|---|
| BYOL | ✓ | ✓ | 0 | **72.5** |
| – | ✓ | ✓ | 1 | 70.9 |
| – | | ✓ | 1 | 70.7 |
| SimCLR | | | 1 | 69.4 |
| – | ✓ | | 1 | 69.1 |
| – | ✓ | | 0 | 0.3 |
| – | | ✓ | 0 | 0.2 |
| – | | | 0 | 0.1 |

（b） BYOL 与 SimCLR之间的中间变体。

Table 5：消融实验，训练300轮后 ImageNet 线性评估的 top-1 准确率（%）。 .

对比方法的消融。本小节将 SimCLR 和 BYOL 写成相同形式，以更好理解 BYOL 相对 SimCLR 的改进来源。考虑如下扩展 InfoNCE 目标 [10, 84]的目标函数（见附录 F.4）：

$$\text{InfoNCE}_\theta^{\alpha,\beta} \triangleq \frac{2}{B}\sum_{i=1}^{B} S_\theta(v_i, v_i') - \beta \cdot \frac{2\alpha}{B}\sum_{i=1}^{B} \ln\left(\sum_{j\neq i} \exp\frac{S_\theta(v_i, v_j)}{\alpha} + \sum_j \exp\frac{S_\theta(v_i, v_j')}{\alpha}\right), \tag{6}$$

其中 $\alpha > 0$ 为固定温度， $\beta \in [0,1]$ 为权重系数， $B$ 为批量大小， $v$ 和 $v'$ 为增强视图批次；任意批次索引 $i$对应的 $v_i$ 和 $v_i'$ 均来自同一图像。实值函数 $S_\theta$ 衡量增强视图之间的成对相似度。对任意增强视图 $u$ , 记$z_\theta$（$u$） $\triangleq$ $f_\theta$（$g_\theta$（$u$）） 及 $z_\xi$（$u$） $\triangleq$ $f_\xi$（$g_\xi$（$u$）） 。对给定 $\phi$ 和 $\psi$，考虑归一化点积：

$$S_\theta(u_1, u_2) \triangleq \frac{\langle \phi(u_1), \psi(u_2)\rangle}{\|\phi(u_1)\|_2 \cdot \|\psi(u_2)\|_2}. \tag{7}$$

忽略少量细节（见附录 F.5），设置 $\phi$（$u_1$） $= z_\theta$（$u_1$）（无预测器）、 $\psi$（$u_2$） $=z_\theta$（$u_2$）（无目标网络）及 $\beta = 1$，即可得到 SimCLR 损失。使用预测器与目标网络，即$\phi$（$u_1$） $= p_\theta$（$z_\theta$（$u_1$）） 和 $\psi$（$u_2$） $= z_\xi$（$u_2$），并设置 $\beta = 0$，即可得到 BYOL 损失。为评估目标网络、预测器及系数 $\beta$的影响，我们对它们进行消融。结果见表5b，更多细节见附录 F.4。
不使用负样本（即 $\beta = 0$）时，唯一表现良好的变体是 BYOL，它同时使用自举目标网络和预测器。未重新调温度就给 BYOL损失加入负样本对，会降低性能。附录 F.4 表明，适当调整温度后可以加回负样本对，并达到与 BYOL 相当的性能。

仅给 SimCLR 加入目标网络，性能就提高+1.6 个百分点。这为 MoCo [9]中目标网络的用途提供了新视角：该方法用目标网络提供更多负样本，而这里表明，即使负样本数量相同，目标网络仅凭稳定作用也能带来好处。最后，给 $S_\theta$ 的架构加入预测器，仅轻微影响 SimCLR的性能。

网络超参数。附录F探索其他网络参数对 BYOL性能的影响。我们尝试多种权重衰减、学习率及投影器／编码器架构，发现小幅超参数变化不会大幅改变最终分数。去掉 BYOL 或 SimCLR 中的权重衰减都会导致网络发散，强调了自监督设置中权重正则化的必要性。此外，改变网络初始化 [85]的缩放因子不影响性能（top-1 高于72%）。

与 Mean Teacher 的关系。另一种半监督方法 Mean Teacher （MT） [20], 在少量标签的有监督损失之外，加入一致性损失。 [20], 的一致性损失是学生网络 logits 与其时间平均版本（教师网络）logits 之间的 $\ell_2$ 距离。去除 BYOL 的预测器后，得到无分类损失的无监督 MT 版本，它

uses image augmentations instead of the original architectural noise (e.g., dropout). This variant of BYOL collapses (Row 7 of Table 5) which suggests that the additional predictor is critical to prevent collapse in an unsupervised scenario.

**Importance of a near-optimal predictor** Table 5b already shows the importance of combining a predictor and a target network: the representation does collapse when either is removed. We further found that we can remove the target network without collapse by making the predictor near-optimal, either by (i) using an optimal *linear* predictor (obtained by linear regression on the current batch) before back-propagating the error through the network ($52.5\%$ top-1 accuracy), or (ii) increasing the learning rate of the predictor ($66.5\%$ top-1). By contrast, increasing the learning rates of both projector *and* predictor (without target network) yields poor results ($\approx 25\%$ top-1). See Appendix I for more details. This seems to indicate that keeping the predictor near-optimal at all times is important to preventing collapse, which may be one of the roles of BYOL's target network.

## 6 Conclusion

We introduced BYOL, a new algorithm for self-supervised learning of image representations. BYOL learns its representation by predicting previous versions of its outputs, without using negative pairs. We show that BYOL achieves state-of-the-art results on various benchmarks. In particular, under the linear evaluation protocol on ImageNet with a ResNet-50 ($1\times$), BYOL achieves a new state of the art and bridges most of the remaining gap between self-supervised methods and the supervised learning baseline of [8]. Using a ResNet-200 ($2\times$), BYOL reaches a top-1 accuracy of $79.6\%$ which improves over the previous state of the art ($76.8\%$) while using $30\%$ fewer parameters.

Nevertheless, BYOL remains dependent on existing sets of augmentations that are specific to vision applications. To generalize BYOL to other modalities (e.g., audio, video, text, ...) it is necessary to obtain similarly suitable augmentations for each of them. Designing such augmentations may require significant effort and expertise. Therefore, automating the search for these augmentations would be an important next step to generalize BYOL to other modalities.



以图像增强替代原来的架构噪声（如 dropout）。这一 BYOL 变体会坍塌（表5第7行），表明额外预测器对无监督情形下防止坍塌至关重要。

近似最优预测器的重要性。表5b已经表明，预测器与目标网络的结合十分重要：去掉任一项，表征都会坍塌。我们进一步发现，使预测器近似最优后，即使去掉目标网络也不会坍塌：可以（i）先通过当前批次上的线性回归求得最优线性预测器，再将误差反向传播到网络（top-1 为52.5%）；或（ii）增大预测器学习率（top-1 为66.5%）。相反，不使用目标网络、同时增大投影器和预测器学习率，结果较差（top-1 为≈ 25%）。详见附录I。这似乎表明，始终保持预测器近似最优对防止坍塌很重要，也可能是 BYOL目标网络的作用之一。

## 6 结论

我们提出 BYOL，一种新的自监督图像表征学习算法。 BYOL通过预测自身输出的历史版本学习表征，无需负样本对。 BYOL在多种基准上取得最优结果。尤其是在 ImageNet 线性评估协议下，采用 ResNet-50（1×）时， BYOL 取得新的最优结果，并弥合了自监督方法与 [8]有监督基线之间的大部分剩余差距。使用 ResNet-200（2×）时， BYOL达到79.6%的 top-1 准确率，超过此前最优的76.8%，同时参数量减少30%。

不过， BYOL 仍依赖针对视觉应用的既有增强集合。若要将 BYOL 推广到音频、视频、文本等其他模态，需要为每种模态找到同样合适的增强。这可能需要大量工作和专业知识。因此，自动搜索这些增强，是将 BYOL 推广到其他模态的重要下一步。

## Broader impact

The presented research should be categorized as research in the field of unsupervised learning. This work may inspire new algorithms, theoretical, and experimental investigation. The algorithm presented here can be used for many different vision applications and a particular use may have both positive or negative impacts, which is known as the dual use problem. Besides, as vision datasets could be biased, the representation learned by BYOL could be susceptible to replicate these biases.

## Acknowledgements

The authors would like to thank the following people for their help throughout the process of writing this paper, in alphabetical order: Aaron van den Oord, Andrew Brock, Jason Ramapuram, Jeffrey De Fauw, Karen Simonyan, Katrina McKinney, Nathalie Beauguerlange, Olivier Henaff, Oriol Vinyals, Pauline Luc, Razvan Pascanu, Sander Dieleman, and the DeepMind team. We especially thank Jason Ramapuram and Jeffrey De Fauw, who provided the JAX SimCLR reproduction used throughout the paper.

## References

[1] Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980.

[2] Laurenz Wiskott and Terrence J Sejnowski. Slow feature analysis: Unsupervised learning of invariances. *Neural Computation*, 14(4), 2002.

[3] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7):1527–1554, 2006.

[4] Maxime Oquab, Leon Bottou, Ivan Laptev, and Josef Sivic. Learning and transferring mid-level image representations using convolutional neural networks. In *Computer Vision and Pattern Recognition*, 2014.

[5] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[6] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Computer Vision and Pattern Recognition*, 2014.

[7] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Computer Vision and Pattern Recognition*, 2015.

[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey E. Hinton. A simple framework for contrastive learning of visual representations. *arXiv preprint arXiv:2002.05709*, 2020.

[9] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross B. Girshick. Momentum contrast for unsupervised visual representation learning. *arXiv preprint arXiv:1911.05722*, 2019.

[10] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[11] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. *arXiv preprint arXiv:1906.05849v4*, 2019.

[12] Yonglong Tian, Chen Sun, Ben Poole, Dilip Krishnan, Cordelia Schmid, and Phillip Isola. What makes for good views for contrastive learning. *arXiv preprint arXiv:2005.10243*, 2020.

[13] Nikunj Saunshi, Orestis Plevrakis, Sanjeev Arora, Mikhail Khodak, and Hrishikesh Khandeparkar. A theoretical analysis of contrastive unsupervised representation learning. In *International Conference on Machine Learning*, 2019.

[14] R. Manmatha, Chao-Yuan Wu, Alexander J. Smola, and Philipp Krähenbühl. Sampling matters in deep embedding learning. In *International Conference on Computer Vision*, 2017.

[15] Ben Harwood, Vijay B. G. Kumar, Gustavo Carneiro, Ian Reid, and Tom Drummond. Smart mining for deep metric learning. In *International Conference on Computer Vision*, 2017.

[16] Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *International Conference on Machine Learning*, 2013.

[17] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In *European Conference on Computer Vision*, 2018.

[18] Philip Bachman, Ouais Alsharif, and Doina Precup. Learning with pseudo-ensembles. In *Advances in neural information processing systems*, pages 3365–3373, 2014.

[19] Samuli Laine and Timo Aila. Temporal ensembling for semi-supervised learning. *arXiv preprint arXiv:1610.02242*, 2016.



## 更广泛的影响

本研究属于无监督学习领域，可能启发新的算法以及理论和实验研究。算法可用于多种视觉应用，具体用途可能带来正面或负面影响，即所谓双重用途问题。此外，视觉数据集可能存在偏差，BYOL 所学表征也可能复制这些偏差。

## 致谢

作者感谢以下人员在论文写作过程中的帮助，按字母顺序排列：Aaron van den Oord、Andrew Brock、Jason Ramapuram、Jeffrey De Fauw、Karen Simonyan、Katrina McKinney、Nathalie Beauguerlange、Olivier Henaff、Oriol Vinyals、Pauline Luc、Razvan Pascanu、Sander Dieleman，以及 DeepMind 团队。特别感谢 Jason Ramapuram 和 Jeffrey De Fauw 提供本文使用的 JAX SimCLR 复现。

## 参考文献

[1] Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. Biological Cybernetics, 36(4):193–202, 1980.[2] Laurenz Wiskott and Terrence J Sejnowski. Slow feature analysis: Unsupervised learning of invariances. Neural Computation, 14(4), 2002.[3] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. Neural Computation, 18(7):1527–1554, 2006.[4] Maxime Oquab, Leon Bottou, Ivan Laptev, and Josef Sivic. Learning and transferring mid-level image representations using convolutional neural networks. In Computer Vision and Pattern Recognition, 2014.[5] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556, 2014.[6] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In Computer Vision and Pattern Recognition, 2014.[7] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In Computer Vision and Pattern Recognition, 2015.[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey E. Hinton. A simple framework for contrastive learning of visual representations. arXiv preprint arXiv:2002.05709, 2020.[9] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross B. Girshick. Momentum contrast for unsupervised visual representation learning. arXiv preprint arXiv:1911.05722, 2019.[10] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748, 2018.[11] Yonglong Tian, Dilip Krishnan, a nd Phillip Isola. Contrastive multiview coding. arXiv preprint arXiv:1906 4, 2019 05849. v .[12] Yonglong Tian, Chen Sun, Ben Poole, Dilip Krishnan, Cordelia Schmid, and Phillip Isola. What makes for good views for contrastive learning. arXiv preprint arXiv:2005.10243, 2020.[13] Nikunj Saunshi, Orestis Plevrakis, Sanjeev Arora, Mikhail Khodak, and Hrishikesh Khandeparkar. A theoretical analysis of contrastive unsupervised representation learning. In International Conference on Machine Learning, 2019.[14] R. Manmatha, Chao-Yuan Wu, Alexander J. Smola, and Philipp Krähenbühl. Sampling matters in deep embedding learning. In International Conference on Computer Vision, 2017.[15] Ben Harwood, Vijay B. G. Kumar, Gustavo Carneiro, Ian Reid, and Tom Drummond. Smart mining for deep metric learning. In International Conference on Computer Vision, 2017.[16] Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In International Conference on Machine Learning, 2013.[17] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In European Conference on Computer Vision, 2018.[18] Philip Bachman, Ouais Alsharif, and Doina Precup. Learning with pseudo-ensembles. In Advances in neural information processing systems, pages 3365–3373, 2014.[19] Samuli Laine and Timo Aila. Temporal ensembling for semi-supervised learning. arXiv preprint arXiv:1610.02242, 2016.

[20] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *Advances in neural information processing systems*, pages 1195–1204, 2017.

[21] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Computer Vision and Pattern Recognition*, 2016.

[23] Carl Doersch, Abhinav Gupta, and Alexei A Efros. Unsupervised visual representation learning by context prediction. In *Computer Vision and Pattern Recognition*, 2015.

[24] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *International Conference on Machine Learning*, 2008.

[25] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[26] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic back-propagation and variational inference in deep latent gaussian models. *arXiv preprint arXiv:1401.4082*, 2014.

[27] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Neural Information Processing Systems*, 2014.

[28] Jeff Donahue, Philipp Krähenbühl, and Trevor Darrell. Adversarial feature learning. *arXiv preprint arXiv:1605.09782*, 2016.

[29] Vincent Dumoulin, Ishmael Belghazi, Ben Poole, Alex Lamb, Martín Arjovsky, Olivier Mastropietro, and Aaron C. Courville. Adversarially learned inference. *arXiv preprint arXiv:1606.00704*, 2017.

[30] Jeff Donahue and Karen Simonyan. Large scale adversarial representation learning. In *Neural Information Processing Systems*, 2019.

[31] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. *arXiv preprint arXiv:1809.11096*, 2018.

[32] Olivier J. Hénaff, Aravind Srinivas, Jeffrey De Fauw, Ali Razavi, Carl Doersch, S. M. Ali Eslami, and Aäron van den Oord. Data-efficient image recognition with contrastive predictive coding. In *International Conference on Machine Learning*, 2019.

[33] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Adam Trischler, and Yoshua Bengio. Learning deep representations by mutual information estimation and maximization. *arXiv preprint arXiv:1808.06670*, 2019.

[34] Philip Bachman, R Devon Hjelm, and William Buchwalter. Learning representations by maximizing mutual information across views. In *Neural Information Processing Systems*, 2019.

[35] Ishan Misra and Laurens van der Maaten. Self-supervised learning of pretext-invariant representations. *arXiv preprint arXiv:1912.01991*, 2019.

[36] Junnan Li, Pan Zhou, Caiming Xiong, Richard Socher, and Steven CH Hoi. Prototypical contrastive learning of unsupervised representations. *arXiv preprint arXiv:2005.04966*, 2020.

[37] Rishabh Jain, Haoqi Fan, Ross B. Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. *arXiv preprint arXiv:2003.04297*, 2020.

[38] Ting Chen, Simon Kornblith, Kevin Swersky, Mohammad Norouzi, and Geoffrey Hinton. Big self-supervised models are strong semi-supervised learners. *arXiv preprint arXiv:2006.10029*, 2020.

[39] Zhirong Wu, Yuanjun Xiong, Stella X Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In *Computer Vision and Pattern Recognition*, 2018.

[40] Carl Doersch and Andrew Zisserman. Multi-task self-supervised visual learning. In *International Conference on Computer Vision*, 2017.

[41] Richard Zhang, Phillip Isola, and Alexei A. Efros. Colorful image colorization. In *European Conference on Computer Vision*, 2016.

[42] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. Learning representations for automatic colorization. In *European Conference on Computer Vision*, 2016.

[43] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A. Efros. Context encoders: Feature learning by inpainting. In *Computer Vision and Pattern Recognition*, 2016.

[44] Mehdi Noroozi and Paolo Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *European Conference on Computer Vision*, 2016.

[45] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In *Computer Vision and Pattern Recognition*, 2017.



[20] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In Advances in neural information processing systems, pages 1195–1204, 2017.[21] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. International Journal of Computer Vision, 115(3):211–252, 2015.[22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Computer Vision and Pattern Recognition, 2016.[23] Carl Doersch, Abhinav Gupta, and Alexei A Efros. Unsupervised visual representation learning by context prediction. In Computer Vision and Pattern Recognition, 2015.[24] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In International Conference on Machine Learning, 2008.[25] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114, 2013.[26] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic back-propagation and variational inference in deep latent gaussian models. arXiv preprint arXiv:1401.4082, 2014.[27] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Neural Information Processing Systems, 2014.[28] Jeff Donahue, Philipp Krähenbühl, and Trevor Darrell. Adversarial feature learning. arXiv preprint arXiv:1605 09782, 2016 . .[29] Vincent Dumoulin, Ishmael Belghazi, Ben Poole, Alex Lamb, Martín Arjovsky, Olivier Mastropietro, and Aaron C. Courville. Adversarially learned inference. arXiv preprint arXiv:1606.00704, 2017.[30] Jeff Donahue and Karen Simonyan. Large scale adversarial representation learning. In Neural Information Processing Systems, 2019.[31] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. arXiv preprint arXiv:1809.11096, 2018.[32] Olivier J. Hénaff, Aravind Srinivas, Jeffrey De Fauw, Ali Razavi, Carl Doersch, S. M. Ali Eslami, and Aäron van den Oord. Data-efficient image recognition with contrastive predictive coding. In International Conference on Machine Learning, 2019.[33] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Adam Trischler, and Yoshua Bengio. Learning deep representations by mutual information estimation and maximization. arXiv preprint arXiv:1808.06670, 2019.[34] Philip Bachman, R Devon Hjelm, and William Buchwalter. Learning representations by maximizing mutual information across views. In Neural Information Processing Systems, 2019.[35] Ishan Misra and Laurens van der Maaten. Self-supervised learning of pretext-invariant representations. arXiv preprint arXiv:1912.01991, 2019.[36] Junnan Li, Pan Zhou, Caiming Xiong, Richard Socher, and Steven CH Hoi. Prototypical contrastive learning of unsupervised representations. arXiv preprint arXiv:2005.04966, 2020.[37] Rishabh Jain, Haoqi Fan, Ross B. Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. arXiv preprint arXiv:2003.04297, 2020.[38] Ting Chen, Simon Kornblith, Kevin Swersky, Mohammad Norouzi, and Geoffrey Hinton. Big self-supervised models are strong semi-supervised learners. arXiv preprint arXiv:2006.10029, 2020.[39] Zhirong Wu, Yuanjun Xiong, Stella X Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In Computer Vision and Pattern Recognition, 2018.[40] Carl Doersch and Andrew Zisserman. Multi-task self-supervised visual learning. In International Conference on Computer Vision, 2017.[41] Richard Zhang, Phillip Isola, and Alexei A. Efros. Colorful image colorization. In European Conference on Computer Vision, 2016.[42] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. Learning representations for automatic colorization. In European Conference on Computer Vision, 2016.[43] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A. Efros. Context encoders: Feature learning by inpainting. In Computer Vision and Pattern Recognition, 2016.[44] Mehdi Noroozi and Paolo Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In European Conference on Computer Vision, 2016.[45] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In Computer Vision and Pattern Recognition, 2017.

[46] Alexey Dosovitskiy, Jost Tobias Springenberg, Martin Riedmiller, and Thomas Brox. Discriminative unsupervised feature learning with convolutional neural networks. In *Neural Information Processing Systems*, 2014.

[47] Spyros Gidaris, Praveer Singh, and Nikos Komodakis. Unsupervised representation learning by predicting image rotations. *arXiv preprint arXiv:1803.07728*, 2018.

[48] Alexander Kolesnikov, Xiaohua Zhai, and Lucas Beyer. Revisiting self-supervised visual representation learning. In *Computer Vision and Pattern Recognition*, 2019.

[49] Daniel Guo, Bernardo Avila Pires, Bilal Piot, Jean-Bastien Grill, Florent Altché, Rémi Munos, and Mohammad Gheshlaghi Azar. Bootstrap latent-predictive representations for multitask reinforcement learning. In *International Conference on Machine Learning*, 2020.

[50] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen. King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[51] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, 2016.

[52] Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *AAAI Conference on Artificial Intelligence*, 2018.

[53] Hado Van Hasselt, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. Deep reinforcement learning and the deadly triad. *Deep Reinforcement Learning Workshop NeurIPS*, 2018.

[54] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[55] Olivier Chapelle, Bernhard Scholkopf, and Alexander Zien. Semi-supervised learning. *IEEE Transactions on Neural Networks*, 20(3):542–542, 2009.

[56] Xiaojin Zhu and Andrew B Goldberg. Introduction to semi-supervised learning. *Synthesis lectures on artificial intelligence and machine learning*, 3(1):1–130, 2009.

[57] Durk P Kingma, Shakir Mohamed, Danilo Jimenez Rezende, and Max Welling. Semi-supervised learning with deep generative models. In *Advances in neural information processing systems*, 2014.

[58] Antti Rasmus, Mathias Berglund, Mikko Honkala, Harri Valpola, and Tapani Raiko. Semi-supervised learning with ladder networks. In *Advances in neural information processing systems*, 2015.

[59] David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin A Raffel. Mixmatch: A holistic approach to semi-supervised learning. In *Advances in Neural Information Processing Systems*, 2019.

[60] Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, and Shin Ishii. Virtual adversarial training: a regularization method for supervised and semi-supervised learning. *IEEE transactions on pattern analysis and machine intelligence*, 41(8):1979–1993, 2018.

[61] David Berthelot, N. Carlini, E. D. Cubuk, Alex Kurakin, Kihyuk Sohn, Han Zhang, and Colin Raffel. Remixmatch: Semi-supervised learning with distribution matching and augmentation anchoring. In *ICLR*, 2020.

[62] Kihyuk Sohn, David Berthelot, Chun-Liang Li, Zizhao Zhang, Nicholas Carlini, Ekin D Cubuk, Alex Kurakin, Han Zhang, and Colin Raffel. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *arXiv preprint arXiv:2001.07685*, 2020.

[63] Suzanna Becker and Geoffrey E. Hinton. Self-organizing neural network that discovers surfaces in random-dot stereograms. *Nature*, 355(6356):161–163, 1992.

[64] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, and Skye Wanderman-Milne. JAX: composable transformations of Python+NumPy programs, 2018.

[65] Tom Hennigan, Trevor Cai, Tamara Norman, and Igor Babuschkin. Haiku: Sonnet for JAX, 2020.

[66] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.

[67] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *arXiv preprint arXiv:1605.07146*, 2016.

[68] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, 2015.

[69] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *International Conference on Machine Learning*, 2010.

[70] Yang You, Igor Gitman, and Boris Ginsburg. Scaling SGD batch size to 32k for imagenet training. *arXiv preprint arXiv:1708.03888*, 2017.



[46] Alexey Dosovitskiy, Jost Tobias Springenberg, Martin Riedmiller, and Thomas Brox. Discriminative unsupervised feature learning with convolutional neural networks. In Neural Information Processing Systems, 2014.[47] Spyros Gidaris, Praveer Singh, and Nikos Komodakis. Unsupervised representation learning by predicting image rotations. arXiv preprint arXiv:1803.07728, 2018.[48] Alexander Kolesnikov, Xiaohua Zhai, and Lucas Beyer. Revisiting self-supervised visual representation learning. In Computer Vision and Pattern Recognition, 2019.[49] Daniel Guo, Bernardo Avila Pires, Bilal Piot, Jean-Bastien Grill, Florent Altché, Rémi Munos, and Mohammad Gheshlaghi Azar. Bootstrap latent-predictive representations for multitask reinforcement learning. In International Conference on Machine Learning, 2020.[50] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen. King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. Nature, 518:529–533, 2015.[51] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In International Conference on Machine Learning, 2016.[52] Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In AAAI Conference on Artificial Intelligence, 2018.[53] Hado Van Hasselt, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. Deep reinforcement learning and the deadly triad. Deep Reinforcement Learning Workshop NeurIPS, 2018.[54] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971, 2015.[55] Olivier Chapelle, Bernhard Scholkopf, and Alexander Zien. Semi-supervised learning. IEEE Transactions on Neural Networks, 20(3):542–542, 2009.[56] Xiaojin Zhu and Andrew B Goldberg. Introduction to semi-supervised learning. Synthesis lectures on artificial intelligence and machine learning, 3(1):1–130, 2009.[57] Durk P Kingma, Shakir Mohamed, Danilo Jimenez Rezende, and Max Welling. Semi-supervised learning with deep generative models. In Advances in neural information processing systems, 2014.[58] Antti Rasmus, Mathias Berglund, Mikko Honkala, Harri Valpola, and Tapani Raiko. Semi-supervised learning with ladder networks. In Advances in neural information processing systems, 2015.[59] David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin A Raffel. Mixmatch: A holistic approach to semi-supervised learning. In Advances in Neural Information Processing Systems, 2019.[60] Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, and Shin Ishii. Virtual adversarial training: a regularization method for supervised and semi-supervised learning. IEEE transactions on pattern analysis and machine intelligence, 41(8):1979–1993, 2018.[61] David Berthelot, N. Carlini, E. D. Cubuk, Alex Kurakin, Kihyuk Sohn, Han Zhang, and Colin Raffel. Remixmatch: Semi-supervised learning with distribution matching and augmentation anchoring. In ICLR, 2020.[62] Kihyuk Sohn, David Berthelot, Chun-Liang Li, Zizhao Zhang, Nicholas Carlini, Ekin D Cubuk, Alex Kurakin, Han Zhang, and Colin Raffel. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. arXiv preprint arXiv:2001.07685, 2020.[63] Suzanna Becker and Geoffrey E. Hinton. Self-organizing neural network that discovers surfaces in random-dot stereograms. Nature, 355(6356):161–163, 1992.[64] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, and Skye Wanderman-Milne. JAX: composable transformations of Python+ NumPy programs, 2018.[65] Tom Hennigan, Trevor Cai, Tamara Norman, and Igor Babuschkin. Haiku: Sonnet for JAX, 2020.[66] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Advances in neural information processing systems, pages 2672 2680, 2014 – .[67] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. arXiv preprint arXiv:1605.07146, 2016.[68] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In International Conference on Machine Learning, 2015.[69] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In International Conference on Machine Learning, 2010.[70] Yang You, Igor Gitman, and Boris Ginsburg. Scaling SGD batch size to 32k for imagenet training. arXiv preprint arXiv:1708.03888, 2017.

13

[71] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.

[72] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

[73] Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. *Transactions on Pattern Analysis and Machine Intelligence*, 2017.

[74] Simon Kornblith, Jonathon Shlens, and Quoc V Le. Do better ImageNet models transfer better? In *Computer Cision and Pattern Recognition*, 2019.

[75] Chengyue Gong, Tongzheng Ren, Mao Ye, and Qiang Liu. Maxup: A simple way to improve generalization of neural network training. *arXiv preprint arXiv:2002.09024*, 2020.

[76] Xiaohua Zhai, Joan Puigcerver, Alexander I Kolesnikov, Pierre Ruyssen, Carlos Riquelme, Mario Lucic, Josip Djolonga, André Susano Pinto, Maxim Neumann, Alexey Dosovitskiy, Lucas Beyer, Olivier Bachem, Michael Tschannen, Marcin Michalski, Olivier Bousquet, Sylvain Gelly, and Neil Houlsby. A large-scale study of representation learning with the visual task adaptation benchmark. *arXiv: Computer Vision and Pattern Recognition*, 2019.

[77] Xiaohua Zhai, Avital Oliver, Alexander Kolesnikov, and Lucas Beyer. S4L: Self-supervised semi-supervised learning. In *International Conference on Computer Vision*, 2019.

[78] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[79] Jianxiong Xiao, James Hays, Krista A. Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *Computer Vision and Pattern Recognition*, 2010.

[80] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The Pascal visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

[81] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *Computer Vision and Pattern Recognition*, 2014.

[82] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Neural Information Processing Systems*, 2015.

[83] I. Laina, C. Rupprecht, V. Belagiannis, F. Tombari, and N. Navab. Deeper depth prediction with fully convolutional residual networks. In *International Conference on 3D Vision*, 2016.

[84] Ben Poole, Sherjil Ozair, Aaron van den Oord, Alexander A Alemi, and George Tucker. On variational bounds of mutual information. *arXiv preprint arXiv:1905.06922*, 2019.

[85] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, 2015.

[86] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning*, 2013.

[87] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. *arXiv preprint arXiv:1909.13719*, 2019.

[88] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[89] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *European Conference on Computer Vision*, 2014.

[90] Thomas Berg, Jiongxin Liu, Seung Woo Lee, Michelle L. Alexander, David W. Jacobs, and Peter N. Belhumeur. Birdsnap: Large-scale fine-grained visual categorization of birds. In *Computer Vision and Pattern Recognition*, 2014.

[91] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3D object representations for fine-grained categorization. In *Workshop on 3D Representation and Recognition*, Sydney, Australia, 2013.

[92] Subhransu Maji, Esa Rahtu, Juho Kannala, Matthew B. Blaschko, and Andrea Vedaldi. Fine-grained visual classification of aircraft. *arXiv preprint arXiv:1306.5151*, 2013.

[93] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In *Computer Vision and Pattern Recognition*, 2012.

[94] Li Fei-Fei, Rob Fergus, and Pietro Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. *Computer Vision and Pattern Recognition Workshop*, 2004.

[95] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian Conference on Computer Vision, Graphics and Image Processing*, 2008.

[96] Jeff Donahue, Yangqing Jia, Oriol Vinyals, Judy Hoffman, Ning Zhang, Eric Tzeng, and Trevor Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. In *International Conference on Machine Learning*, 2014.

[97] Art B Owen. A robust hybrid of lasso and ridge regression. *Contemporary Mathematics*, 443(7):59–72, 2007.





[71] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with warm restarts. In International Conference on Learning Representations, 2017.[72] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch s gd: Training imagenet in 1 02677, 2017 1706 hour. arXiv preprint arXiv: . .[73] Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. Transactions on Pattern Analysis and Machine Intelligence, 2017.[74] Simon Kornblith, Jonathon Shlens, and Quoc V Le. Do better ImageNet models transfer better? In Computer Cision and Pattern Recognition, 2019.[75] Chengyue Gong, Tongzheng Ren, Mao Ye, and Qiang Liu. Maxup: A simple way to improve generalization of neural network training. arXiv preprint arXiv:2002.09024, 2020.[76] Xiaohua Zhai, Joan Puigcerver, Alexander I Kolesnikov, Pierre Ruyssen, Carlos Riquelme, Mario Lucic, Josip Djolonga, André Susano Pinto, Maxim Neumann, Alexey Dosovitskiy, Lucas Beyer, Olivier Bachem, Michael Tschannen, Marcin Michalski, Olivier Bousquet, Sylvain Gelly, and Neil Houlsby. A large-scale study of representation learning with the visual task adaptation benchmark. arXiv: Computer Vision and Pattern Recognition, 2019.[77] Xiaohua Zhai, Avital Oliver, Alexander Kolesnikov, and Lucas Beyer. S4L: Self-supervised semi-supervised learning. In International Conference on Computer Vision, 2019.[78] Alex Krizhevsky. Learning multiple layers of features from tiny i mages. Technical report, University of Toronto, 2009.[79] Jianxiong Xiao, James Hays, Krista A. Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In Computer Vision and Pattern Recognition, 2010.[80] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The Pascal visual object classes (VOC) challenge. International Journal of Computer Vision, 88(2):303–338, 2010.[81] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In Computer Vision and Pattern Recognition, 2014.[82] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In Neural Information Processing Systems, 2015.[83] I. Laina, C. Rupprecht, V. Belagiannis, F. Tombari, and N. Navab. Deeper depth prediction with fully convolutional residual networks. In International Conference on 3D Vision, 2016.[84] Ben Poole, Sherjil Ozair, Aaron van den Oord, Alexander A Alemi, and George Tucker. On variational bounds of mutual information. arXiv preprint arXiv:1905.06922, 2019.[85] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In Proceedings of the IEEE international conference on computer vision, 2015.[86] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In International Conference on Machine Learning, 2013.[87] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. arXiv preprint arXiv:1909.13719, 2019.[88] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In Computer Vision and Pattern Recognition, pages 2818–2826, 2016.[89] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In European Conference on Computer Vision, 2014.[90] Thomas Berg, Jiongxin Liu, Seung Woo Lee, Michelle L. Alexander, David W. Jacobs, and Peter N. Belhumeur. Birdsnap: Large-scale fine-grained visual categorization of birds. In Computer Vision and Pattern Recognition, 2014.[91] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3D object representations for fine-grained categorization. In Workshop on 3D Representation and Recognition, Sydney, Australia, 2013.[92] Subhransu Maji, Esa Rahtu, Juho Kannala, Matthew B. Blaschko, and Andrea Vedaldi. Fine-grained visual classification of aircraft. arXiv preprint arXiv:1306.5151, 2013.[93] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In Computer Vision and Pattern Recognition, 2012.[94] Li Fei-Fei, Rob Fergus, and Pietro Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. Computer Vision and Pattern Recognition Workshop, 2004.[95] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In Indian Conference on Computer Vision, Graphics and Image Processing, 2008.[96] Jeff Donahue, Yangqing Jia, Oriol Vinyals, Judy Hoffman, Ning Zhang, Eric Tzeng, and Trevor Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. In International Conference on Machine Learning, 2014.[97] Art B Owen. A robust hybrid of lasso and ridge regression. Contemporary Mathematics, 443(7):59–72, 2007.


14

[98] Chengxu Zhuang, Alex Lin Zhai, and Daniel Yamins. Local aggregation for unsupervised learning of visual embeddings. In *International Conference on Computer Vision*, 2019.

[99] Deepak Pathak, Ross Girshick, Piotr Dollár, Trevor Darrell, and Bharath Hariharan. Learning features by watching objects move. In *Conference on Computer Vision and Pattern Recognition*, 2017.

[100] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision*, 2018.



[98] Chengxu Zhuang, Alex Lin Zhai, and Daniel Yamins. Local aggregation for unsupervised learning of visual embeddings. In International Conference on Computer Vision, 2019.[99] Deepak Pathak, Ross Girshick, Piotr Dollár, Trevor Darrell, and Bharath Hariharan. Learning features by watching objects move. In Conference on Computer Vision and Pattern Recognition, 2017.[100] Yuxin Wu and Kaiming He. Group normalization. In European Conference on Computer Vision, 2018.

15

## A Algorithm

**Algorithm 1:** BYOL: **B**ootstrap **Y**our **O**wn **L**atent

**Inputs :**

| | |
|---|---|
| $\mathcal{D}$, $\mathcal{T}$, and $\mathcal{T}'$ | set of images and distributions of transformations |
| $\theta$, $f_\theta$, $g_\theta$, and $q_\theta$ | initial online parameters, encoder, projector, and predictor |
| $\xi$, $f_\xi$, $g_\xi$ | initial target parameters, target encoder, and target projector |
| optimizer | optimizer, updates online parameters using the loss gradient |
| $K$ and $N$ | total number of optimization steps and batch size |
| $\{\tau_k\}_{k=1}^K$ and $\{\eta_k\}_{k=1}^K$ | target network update schedule and learning rate schedule |

**for** $k = 1$ **to** $K$ **do**
  $\mathcal{B} \leftarrow \{x_i \sim \mathcal{D}\}_{i=1}^N$ // sample a batch of $N$ images
  **for** $x_i \in \mathcal{B}$ **do**
    $t \sim \mathcal{T}$ and $t' \sim \mathcal{T}'$ // sample image transformations
    $z_1 \leftarrow g_\theta(f_\theta(t(x_i)))$ and $z_2 \leftarrow g_\theta(f_\theta(t'(x_i)))$ // compute projections
    $z'_1 \leftarrow g_\xi(f_\xi(t'(x_i)))$ and $z'_2 \leftarrow g_\xi(f_\xi(t(x_i)))$ // compute target projections
    $l_i \leftarrow -2 \cdot \left( \frac{\langle q_\theta(z_1), z'_1 \rangle}{\|q_\theta(z_1)\|_2 \cdot \|z'_1\|_2} + \frac{\langle q_\theta(z_2), z'_2 \rangle}{\|q_\theta(z_2)\|_2 \cdot \|z'_2\|_2} \right)$ // compute the loss for $x_i$
  **end**
  $\delta\theta \leftarrow \frac{1}{N} \sum_{i=1}^N \partial_\theta l_i$ // compute the total loss gradient w.r.t. $\theta$
  $\theta \leftarrow \text{optimizer}(\theta, \delta\theta, \eta_k)$ // update online parameters
  $\xi \leftarrow \tau_k \xi + (1 - \tau_k)\theta$ // update target parameters
**end**

**Output :** encoder $f_\theta$

## B Image augmentations

During self-supervised training, BYOL uses the following image augmentations (which are a subset of the ones presented in [8]):

- random cropping: a random patch of the image is selected, with an area uniformly sampled between 8% and 100% of that of the original image, and an aspect ratio logarithmically sampled between $3/4$ and $4/3$. This patch is then resized to the target size of $224 \times 224$ using bicubic interpolation;
- optional left-right flip;
- color jittering: the brightness, contrast, saturation and hue of the image are shifted by a uniformly random offset applied on all the pixels of the same image. The order in which these shifts are performed is randomly selected for each patch;
- color dropping: an optional conversion to grayscale. When applied, output intensity for a pixel $(r, g, b)$ corresponds to its luma component, computed as $0.2989r + 0.5870g + 0.1140b$;
- Gaussian blurring: for a $224 \times 224$ image, a square Gaussian kernel of size $23 \times 23$ is used, with a standard deviation uniformly sampled over $[0.1, 2.0]$;
- solarization: an optional color transformation $x \mapsto x \cdot \mathbf{1}_{\{x<0.5\}} + (1 - x) \cdot \mathbf{1}_{\{x \geq 0.5\}}$ for pixels with values in $[0, 1]$.

Augmentations from the sets $\mathcal{T}$ and $\mathcal{T}'$ (introduced in Section 3) are compositions of the above image augmentations in the listed order, each applied with a predetermined probability. The image augmentations parameters are listed in Table 6.

During evaluation, we use a center crop similar to [8]: images are resized to 256 pixels along the shorter side using bicubic resampling, after which a $224 \times 224$ center crop is applied. In both training and evaluation, we normalize color channels by subtracting the average color and dividing by the standard deviation, computed on ImageNet, after applying the augmentations.



## A 算法

算法1： BYOL：自举你自己的潜在表征

**输入：**

| | |
|---|---|
| $\mathcal{D}$、 $\mathcal{T}$和 $\mathcal{T}'$ | 图像集合和变换分布 |
| $\theta$、 $f_\theta$、 $g_\theta$和 $q_\theta$ | 初始在线参数、编码器、投影器和预测器 |
| $\xi$, $f_\xi$, $g_\xi$ | 初始目标参数、目标编码器和目标投影器 |
| 优化器 | 优化器，利用损失梯度更新在线参数 |
| $K$ 和 $N$ | 优化总步数和批量大小 |
| $\{\tau_k\}_{k=1}^K$ 和 $\{\eta_k\}_{k=1}^K$ | 目标网络更新日程与学习率日程 |

**1 对** $k = 1$**至** $K$ **循环**
  $\mathcal{B} \leftarrow \{x_i \sim \mathcal{D}\}_{i=1}^N$ // sample a batch of $N$ images
  **对** $x_i \in \mathcal{B}$ **循环**
    $t \sim \mathcal{T}$和 $t' \sim \mathcal{T}'$ // sample image transformations
    $z_1 \leftarrow g_\theta$（$f_\theta$（$t$（$x_i$））） 和 $z_2 \leftarrow g_\theta(f_\theta(t'(x_i)))$ // compute projections
    $z'_1 \leftarrow g_\xi$（$f_\xi$（$t'$（$x_i$））） 和 $z'_2 \leftarrow g_\xi$（$f_\xi$（$t$（$x_i$））） // compute target projections
    $l_i \leftarrow -2 \cdot \left( \frac{\langle q_\theta(z_1)\ z'_1 \rangle}{\|q_\theta(z_1)\|_2 \cdot \|z'_1\|_2} + \frac{\langle q_\theta(z_2)\ z'_2 \rangle}{\|q_\theta(z_2)\|_2 \cdot \|z'_2\|_2} \right)$ // compute the loss for $x_i$
  **end**
  $\delta\theta \leftarrow \frac{1}{N} \sum_{i=1}^N \partial_\theta l_i$ // compute the total loss gradient w.r.t. $\theta$
  $\theta \leftarrow \text{optimizer}(\theta, \delta\theta, \eta_k)$ // update online parameters
  $\xi \leftarrow \tau_k \xi + (1 - \tau_k)\theta$ // update target parameters
**12 结束**

输出：编码器 $f_\theta$

## B 图像增强

自监督训练期间， BYOL 使用以下图像增强（它们是 [8]所列增强的子集）：

- 随机裁剪：选取随机图像块，面积在原图面积的8%至100%之间均匀采样，宽高比在 $3/4$ 至 $4/3$ 之间按对数采样。随后用双三次插值将图像块缩放到目标尺寸 $224 \times 224$；
- 可选的左右翻转；
- 颜色抖动：图像亮度、对比度、饱和度和色相各加上均匀随机偏移，同一图像所有像素使用相同偏移。每个图像块随机决定这些调整的顺序；
- 去除颜色：可选的灰度转换。应用时，像素（$r$， $g$， $b$）的输出强度对应其亮度分量，计算为 $0.2989r + 0.5870g + 0.1140b$；
- 高斯模糊：对 $224 \times 224$ 图像，使用尺寸 $23 \times 23$ 的方形高斯核，标准差在 $[0.1, 2.0]$内均匀采样；
- 曝光反转：可选颜色变换，对数值处于 $[0, 1]$的像素应用 $x \mapsto x \cdot \mathbf{1}_{\{x<0.5\}} +$（$1 - x$） $\cdot \mathbf{1}_{\{x \geq 0.5\}}$。

第3节引入的集合 $\mathcal{T}$和 $\mathcal{T}'$ 中的增强，是按所列顺序组合上述图像增强，每项按预定概率应用。参数见表6。

评估时采用与 [8]类似的中心裁剪：用双三次重采样将短边缩放为256像素，再做 $224 \times 224$ 中心裁剪。训练和评估均在增强后，用 ImageNet 计算的颜色均值与标准差对颜色通道进行减均值、除标准差的归一化。

| Parameter | $\mathcal{T}$ | $\mathcal{T}'$ |
|---|---|---|
| Random crop probability | 1.0 | 1.0 |
| Flip probability | 0.5 | 0.5 |
| Color jittering probability | 0.8 | 0.8 |
| Brightness adjustment max intensity | 0.4 | 0.4 |
| Contrast adjustment max intensity | 0.4 | 0.4 |
| Saturation adjustment max intensity | 0.2 | 0.2 |
| Hue adjustment max intensity | 0.1 | 0.1 |
| Color dropping probability | 0.2 | 0.2 |
| Gaussian blurring probability | 1.0 | 0.1 |
| Solarization probability | 0.0 | 0.2 |

Table 6: Parameters used to generate image augmentations.

# C Evaluation on ImageNet training

## C.1 Self-supervised learning evaluation on ImageNet

**Linear evaluation protocol on ImageNet** As in [48, 74, 8, 37], we use the standard linear evaluation protocol on ImageNet, which consists in training a linear classifier on top of the frozen representation, *i.e.*, without updating the network parameters nor the batch statistics. At training time, we apply spatial augmentations, i.e., random crops with resize to $224 \times 224$ pixels, and random flips. At test time, images are resized to 256 pixels along the shorter side using bicubic resampling, after which a $224 \times 224$ center crop is applied. In both cases, we normalize the color channels by subtracting the average color and dividing by the standard deviation (computed on ImageNet), after applying the augmentations. We optimize the cross-entropy loss using `SGD` with Nesterov momentum over 80 epochs, using a batch size of 1024 and a momentum of 0.9. We do not use any regularization methods such as weight decay, gradient clipping [86], tclip [34], or logits regularization. We finally sweep over 5 learning rates $\{0.4, 0.3, 0.2, 0.1, 0.05\}$ on a local validation set (10009 images from ImageNet `train` set), and report the accuracy of the best validation hyperparameter on the `test` set (which is the public validation set of the original ILSVRC2012 ImageNet dataset).

**Variant on linear evaluation on ImageNet** In this paragraph only, we deviate from the protocol of [8, 37] and propose another way of performing linear evaluation on top of a frozen representation. This method achieves better performance both in top-1 and top-5 accuracy.

- We replace the spatial augmentations (random crops with resize to $224 \times 224$ pixels and random flips) with the pre-train augmentations of Appendix B. This method was already used in [32] with a different subset of pre-train augmentations.
- We regularize the linear classifier as in [34][8] by clipping the logits using a hyperbolic tangent function

$$\text{tclip}(x) \triangleq \alpha \cdot \tanh(x/\alpha),$$

  where $\alpha$ is a positive scalar, and by adding a logit-regularization penalty term in the loss

$$\text{Loss}(x, y) \triangleq \text{cross_entropy}(\text{tclip}(x), y) + \beta \cdot \text{average}(\text{tclip}(x)^2),$$

  where $x$ are the logits, $y$ are the target labels, and $\beta$ is the regularization parameter. We set $\alpha = 20$ and $\beta = 1e{-}2$.

We report in Table 7 the top-1 and top-5 accuracy on ImageNet using this modified protocol. These modifications in the evaluation protocol increase the `BYOL`'s top-1 accuracy from 74.3% to 74.8% with a ResNet-50 ($1\times$).

**Semi-supervised learning on ImageNet** We follow the semi-supervised learning protocol of [8, 77]. We first initialize the network with the parameters of the pretrained representation, and fine-tune it with a subset of ImageNet labels. At training time, we apply spatial augmentations, i.e., random crops with resize to $224 \times 224$ pixels and random flips. At test time, images are resized to 256 pixels along the shorter side using bicubic resampling, after which a $224 \times 224$ center crop is applied. In both cases, we normalize the color channels by subtracting the average color and dividing by the standard deviation (computed on ImageNet), after applying the augmentations. We optimize the cross-entropy loss using `SGD` with Nesterov momentum. We used a batch size of 1024, a momentum of

[8] https://github.com/Philip-Bachman/amdim-public/blob/master/costs.py



| 参数 | $\mathcal{T}$ | $\mathcal{T}'$ |
|---|---|---|
| 随机裁剪概率 | 1.0 | 1.0 |
| 翻转概率 | 0.5 | 0.5 |
| 颜色抖动概率 | 0.8 | 0.8 |
| 亮度调整最大强度 | 0.4 | 0.4 |
| 对比度调整最大强度 | 0.4 | 0.4 |
| 饱和度调整最大强度 | 0.2 | 0.2 |
| 色相调整最大强度 | 0.1 | 0.1 |
| 去除颜色概率 | 0.2 | 0.2 |
| 高斯模糊概率 | 1.0 | 0.1 |
| 曝光反转概率 | 0.0 | 0.2 |

表6：生成图像增强的参数。

# C ImageNet 训练的评估

## C.1 ImageNet 自监督学习评估

ImageNet 线性评估协议。按照 [48, 74, 8, 37], ，我们采用标准 ImageNet 线性评估协议：在冻结的表征上训练线性分类器，即不更新网络参数和批次统计量。训练时采用空间增强，即随机裁剪并缩放为 $224 \times 224$ 像素，以及随机翻转。测试时用双三次重采样将图像短边缩放为256像素，再进行 $224 \times 224$ 中心裁剪。两种情况下都在增强之后，用 ImageNet 的平均颜色和标准差对各颜色通道进行减均值、除标准差的归一化。使用带 Nesterov 动量的 `SGD` 优化交叉熵损失，训练80轮，批量大小1024，动量0.9。不使用权重衰减、梯度裁剪 [86], 、tclip [34], 或 logits 正则化等正则化方法。最后，在本地验证集（ImageNet `train`集中的10009张图像）上搜索5个学习率$\{0.4, 0.3, 0.2, 0.1, 0.05\}$，并在 `test` 集（原始 ILSVRC2012 ImageNet 的公开验证集）上报告最佳验证超参数对应的准确率。

ImageNet 线性评估的变体。仅在本段，我们改变 [8, 37] 的协议，提出另一种在冻结表征上做线性评估的方法，其 top-1 与 top-5 准确率均更高。

- 将空间增强（随机裁剪并缩放为 $224 \times 224$ 像素，以及随机翻转）替换为附录B的预训练增强。该方法已在 [32] 中使用，但采用不同的预训练增强子集。增强。
- 按照 [34][8]，用双曲正切函数裁剪 logits，对线性分类器正则化：

$$\text{tclip}(x) \triangleq \alpha \cdot \tanh(x/\alpha),$$

  其中 $\alpha$ 为正标量，并在损失中加入 logits 正则化惩罚项：

$$\text{Loss}(x, y) \triangleq \text{cross_entropy}(\text{tclip}(x), y) + \beta \cdot \text{average}(\text{tclip}(x)^2),$$

  其中 $x$ 为 logits， $y$ 为目标标签， $\beta$ 为正则化参数。设置 $\alpha = 20$ 及 $\beta = 1e{-}2$。

表7报告修改后协议在 ImageNet 上的 top-1 与 top-5 准确率。对于 ResNet-50（$1\times$），这些评估协议变化使 `BYOL` 的 top-1 从74.3%提高到74.8%。

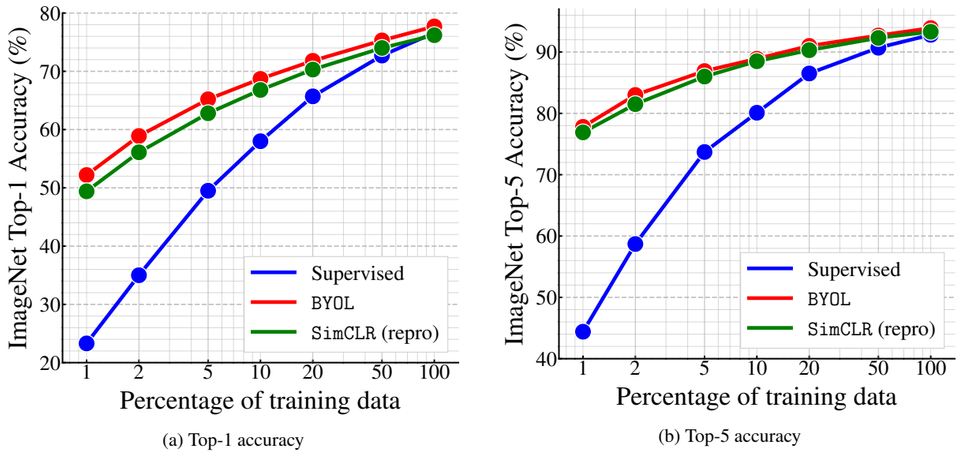
ImageNet 半监督学习。我们遵循 [8, 77]的半监督协议，用预训练表征参数初始化网络，再用 ImageNet 标签子集微调。训练时进行空间增强，即随机裁剪并缩放为 $224 \times 224$ 像素，以及随机翻转。测试时用双三次重采样将短边缩放为256像素，再进行 $224 \times 224$ 中心裁剪。两种情况下都在增强后使用 ImageNet 的均值与标准差归一化颜色通道。用带 Nesterov 动量的 `SGD` 优化交叉熵损失。批量大小1024，动量为

[8] https://github.com/Philip-Bachman/amdim-public/blob/master/costs.py

| Architecture | Pre-train augmentations | Logits regularization | Top-1 | Top-5 |
|---|---|---|---|---|
| ResNet-50 (1×) | | | 74.3 | 91.6 |
| | ✓ | | 74.4 | 91.8 |
| | | ✓ | 74.7 | 91.8 |
| | ✓ | ✓ | **74.8** | **91.8** |
| ResNet-50 (4×) | | | 78.6 | 94.2 |
| | ✓ | | 78.6 | 94.3 |
| | | ✓ | 78.9 | 94.3 |
| | ✓ | ✓ | **79.0** | **94.5** |
| ResNet-200 (2×) | | | 79.6 | 94.8 |
| | ✓ | | 79.6 | 94.8 |
| | | ✓ | 79.8 | 95.0 |
| | ✓ | ✓ | **80.0** | **95.0** |

Table 7: Different linear evaluation protocols on ResNet architectures by either replacing the spatial augmentations with pre-train augmentations, or regularizing the linear classifier. No pre-train augmentations and no logits regularization correspond to the evaluation protocol of the main paper, which is the same as in [8, 37].

0.9. We do not use any regularization methods such as weight decay, gradient clipping [86], tclip [34], or logits rescaling. We sweep over the learning rate $\{0.01, 0.02, 0.05, 0.1, 0.005\}$ and the number of epochs $\{30, 50\}$ and select the hyperparameters achieving the best performance on our local validation set to report test performance.




(a) Top-1 accuracy (b) Top-5 accuracy

Figure 4: Semi-supervised training with a fraction of ImageNet labels on a ResNet-50 (×1).

In Table 2 presented in the main text, we fine-tune the representation over the 1% and 10% ImageNet splits from [8] with various ResNet architectures.

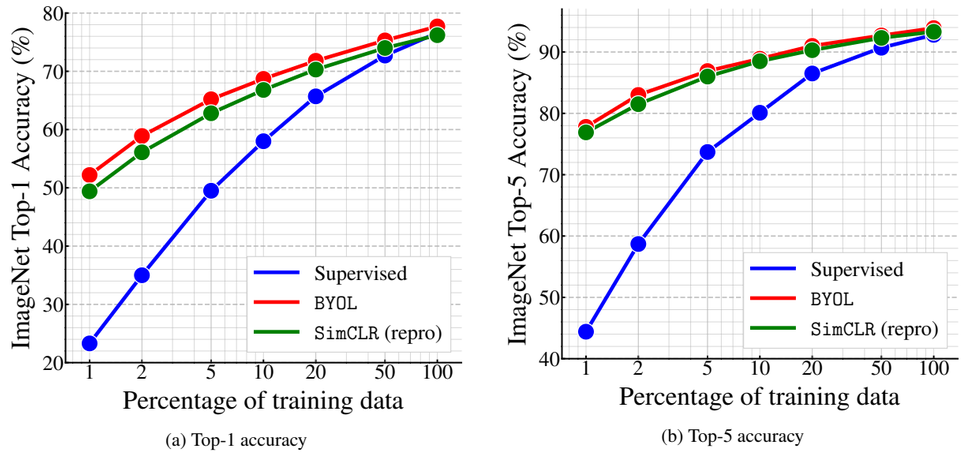
In Figure 4, we fine-tune the representation over 1%, 2%, 5%, 10%, 20%, 50%, and 100% of the ImageNet dataset as in [32] with a ResNet-50 (1×) architecture, and compare them with a supervised baseline and a fine-tuned SimCLR representation. In this case and contrary to Table 2 we don't reuse the splits from SimCLR but we create our own via a balanced selection. In this setting, we observed that tuning a BYOL representation always outperforms a supervised baseline trained from scratch. In Figure 5, we then fine-tune the representation over multiple ResNet architectures. We observe that the largest networks are prone to overfitting as they are outperformed by ResNets with identical depth but smaller scaling factor. This overfitting is further confirmed when looking at the training and evaluation loss: large networks have lower training losses, but higher validation losses than some of their slimmer counterparts. Regularization methods are thus recommended when tuning on large architectures.



| Architecture | Pre-train augmentations | Logits regularization | Top-1 | Top-5 |
|---|---|---|---|---|
| ResNet-50 (1×) | | | 74.3 | 91.6 |
| | ✓ | | 74.4 | 91.8 |
| | | ✓ | 74.7 | 91.8 |
| | ✓ | ✓ | **74.8** | **91.8** |
| ResNet-50 (4×) | | | 78.6 | 94.2 |
| | ✓ | | 78.6 | 94.3 |
| | | ✓ | 78.9 | 94.3 |
| | ✓ | ✓ | **79.0** | **94.5** |
| ResNet-200 (2×) | | | 79.6 | 94.8 |
| | ✓ | | 79.6 | 94.8 |
| | | ✓ | 79.8 | 95.0 |
| | ✓ | ✓ | **80.0** | **95.0** |

表7：ResNet 架构的不同线性评估协议，将空间增强替换为预训练增强，或对线性分类器做正则化。不使用预训练增强和 logits 正则化时，对应正文中的评估协议，与 [8, 37]相同。

0.9。不使用权重衰减、梯度裁剪 [86], 、tclip [34], 或 logits 重缩放等正则化。我们搜索学习率 $\{0.01, 0.02, 0.05, 0.1, 0.005\}$ 与训练轮数 $\{30, 50\}$，选择本地验证集上表现最好的超参数，再报告测试性能。




(a) Top-1 accuracy (b) Top-5 accuracy

图4：ResNet-50（×1）使用部分 ImageNet 标签进行半监督训练。 .

正文表2中，我们使用多种 ResNet 架构，在 [8]的 ImageNet 1%和10%划分上微调表征。

图4中，我们按照 [32]，采用 ResNet-50（1×），在 ImageNet 数据集的1%、2%、5%、10%、20%、50%和100%上微调表征，并与有监督基线及微调后的SimCLR 表征比较。与表2不同，此处没有复用 SimCLR 的数据划分，而是通过均衡选择自行划分。在该设置下，微调 BYOL 表征始终优于从零训练的有监督基线。图5进一步在多种 ResNet 架构上微调表征。最大网络容易过拟合：同样深度、较小宽度倍率的 ResNet 反而表现更好。训练和评估损失也证实了这一点：大型网络训练损失更低，但验证损失高于某些更窄的网络。因此，微调大型架构时建议使用正则化。

| *Supervised:* Method | Top-1 | Top-5 | *Semi-supervised* (100%): Method | Top-1 | Top-5 |
|---|---|---|---|---|---|
| Supervised[8] | 76.5 | – | SimCLR [8] | 76.0 | 93.1 |
| AutoAugment [87] | 77.6 | 93.8 | SimCLR (repro) | 76.5 | 93.5 |
| MaxUp [75] | **78.9** | **94.2** | BYOL | **77.7** | **93.9** |

Table 8: Semi-supervised training with the full ImageNet on a ResNet-50 (×1). We also report other fully supervised methods for extensive comparisons.

Finally, we fine-tune the representation over the full ImageNet dataset. We report the results in Table 8 along with supervised baselines trained on ImageNet. We observe that fine-tuning the SimCLR checkpoint does not yield better results (in our reproduction, which matches the results reported in the original paper [8]) than using a random initialization (76.5 top-1). Instead, BYOL's initialization checkpoint leads to a high final score (77.7 top-1), higher than the vanilla supervised baseline of [8], matching the strong supervised baseline of AutoAugment[87] but still 1.2 points below the stronger supervised baseline [75], which uses advanced supervised learning techniques.

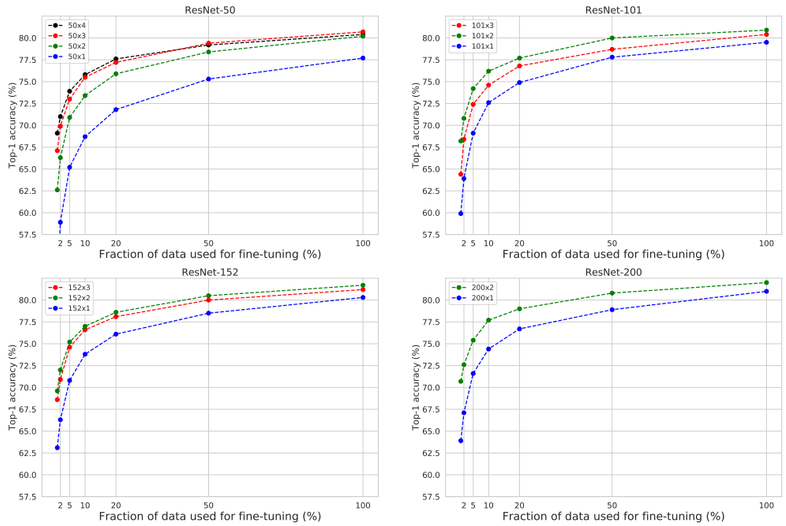



Figure 5: Semi-supervised training with a fraction of ImageNet labels on multiple ResNets architecture pretrained with BYOL. Note that large networks are facing overfitting problems.

### C.2 Linear evaluation on larger architectures and supervised baselines

Here we investigate the performance of BYOL with deeper and wider ResNet architectures. We compare ourselves to the best supervised baselines from [8] when available (rightmost column in table 9), which are also presented in Figure 1. Importantly, we close in on those baselines using the ResNet-50 (2×) and the ResNet-50 (4×) architectures, where we are within 0.4 accuracy points of the supervised performance. To the best of our knowledge, this is the first time that the gap to supervised has been closed to such an extent using a self-supervised method under the linear evaluation protocol. Therefore, in order to ensure fair comparison, and suspecting that the supervised baselines' performance in [8] could be even further improved with appropriate data augmentations, we also report on our own reproduction of strong supervised baselines. We use RandAugment [87] data augmentation for all large



| 有监督: 方法 | Top-1 | Top-5 | 半监督（*100%*）: 方法 | Top-1 | Top-5 |
|---|---|---|---|---|---|
| 有监督[8] | 76.5 | – | SimCLR [8] | 76.0 | 93.1 |
| AutoAugment [87] | 77.6 | 93.8 | SimCLR（复现） | 76.5 | 93.5 |
| MaxUp [75] | **78.9** | **94.2** | BYOL | **77.7** | **93.9** |

表8：ResNet-50（×1）使用完整 ImageNet 的半监督训练。为全面比较，也报告其他完全有监督方法。

最后，我们在完整 ImageNet 上微调表征。表8同时报告微调结果和在 ImageNet 上训练的有监督基线。微调 SimCLR 检查点并未优于随机初始化（top-1 为76.5%；我们的复现与原论文 [8]一致）。而从 BYOL 的检查点初始化得到更高的最终分数（top-1 为77.7），超过 [8], 的普通有监督基线，与 AutoAugment[87] 的强有监督基线相当，但仍比使用更先进有监督技术的更强基线 [75],低1.2个百分点。

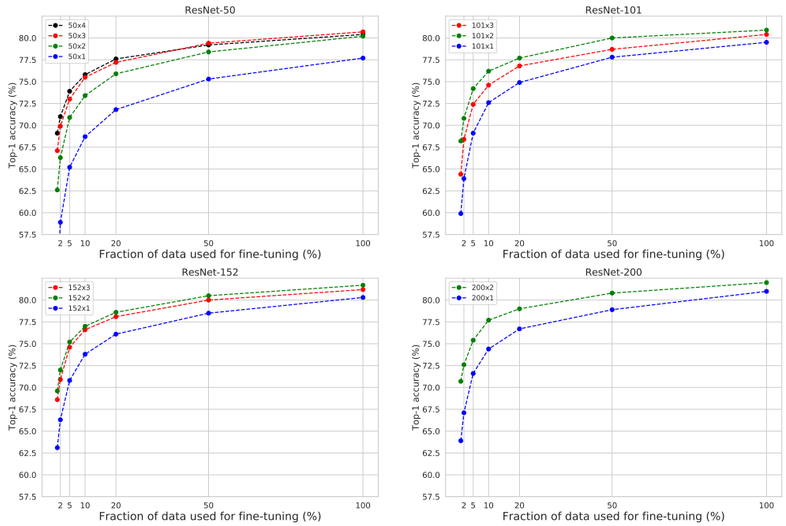



图5：采用 BYOL预训练的多种 ResNet 架构，使用部分 ImageNet 标签进行半监督训练。大型网络出现过拟合问题。

### C.2 更大架构的线性评估及有监督基线

这里研究 BYOL 在更深、更宽 ResNet 上的表现。如有对应结果，我们与 [8] 中的最佳有监督基线比较（表9最右列），这些基线也绘于图1。使用 ResNet-50（2×）和 ResNet-50（4×）时，与有监督性能的差距缩小至0.4个百分点以内。据我们所知，这是自监督方法首次在线性评估协议下将差距缩小到如此程度。因此，为公平比较，且考虑到适当增强可能进一步改善 [8] 的有监督基线，我们也报告自行复现的强有监督基线。我们使用 RandAugment [87] 数据增强，适用于所有大型

| Architecture | Multiplier | Weights | BYOL Top-1 | BYOL Top-5 | Supervised (ours) Top-1 | Supervised (ours) Top-5 | Supervised [8] Top-1 |
|---|---|---|---|---|---|---|---|
| ResNet-50 | 1× | 24M | 74.3 | 91.6 | 76.4 | 92.9 | 76.5 |
| ResNet-101 | 1× | 43M | 76.4 | 93.0 | 78.0 | 94.0 | - |
| ResNet-152 | 1× | 58M | 77.3 | 93.7 | 79.1 | 94.5 | - |
| ResNet-200 | 1× | 63M | 77.8 | 93.9 | 79.3 | 94.6 | - |
| ResNet-50 | 2× | 94M | 77.4 | 93.6 | 79.9 | 95.0 | 77.8 |
| ResNet-101 | 2× | 170M | 78.7 | 94.3 | 80.3 | 95.0 | - |
| ResNet-50 | 3× | 211M | 78.2 | 93.9 | 80.2 | 95.0 | - |
| ResNet-152 | 2× | 232M | 79.0 | 94.6 | 80.6 | 95.3 | - |
| ResNet-200 | 2× | 250M | **79.6** | **94.9** | 80.1 | 95.2 | - |
| ResNet-50 | 4× | 375M | 78.6 | 94.2 | 80.7 | **95.3** | 78.9 |
| ResNet-101 | 3× | 382M | 78.4 | 94.2 | 80.7 | **95.3** | - |
| ResNet-152 | 3× | 522M | 79.5 | 94.6 | **80.9** | 95.2 | - |

Table 9: Linear evaluation of BYOL on ImageNet using larger encoders. Top-1 and top-5 accuracies are reported in %.

ResNet architectures (which are all version 1, as per [22]). We train our supervised baselines for up to 200 epochs, using SGD with a Nesterov momentum value of 0.9, a cosine-annealed learning rate after a 5 epochs linear warmup period, weight decay with a value of $1e-4$, and a label smoothing [88] value of 0.1. Results are presented in Figure 6.



| 架构 | 倍率 | 权重 | BYOL Top-1 | BYOL Top-5 | 有监督（本文） Top-1 | 有监督（本文） Top-5 | 有监督 [8] Top-1 |
|---|---|---|---|---|---|---|---|
| ResNet-50 | 1× | 24M | 74.3 | 91.6 | 76.4 | 92.9 | 76.5 |
| ResNet-101 | 1× | 43M | 76.4 | 93.0 | 78.0 | 94.0 | - |
| ResNet-152 | 1× | 58M | 77.3 | 93.7 | 79.1 | 94.5 | - |
| ResNet-200 | 1× | 63M | 77.8 | 93.9 | 79.3 | 94.6 | - |
| ResNet-50 | 2× | 94M | 77.4 | 93.6 | 79.9 | 95.0 | 77.8 |
| ResNet-101 | 2× | 170M | 78.7 | 94.3 | 80.3 | 95.0 | - |
| ResNet-50 | 3× | 211M | 78.2 | 93.9 | 80.2 | 95.0 | - |
| ResNet-152 | 2× | 232M | 79.0 | 94.6 | 80.6 | 95.3 | - |
| ResNet-200 | 2× | 250M | **79.6** | **94.9** | 80.1 | 95.2 | - |
| ResNet-50 | 4× | 375M | 78.6 | 94.2 | 80.7 | **95.3** | 78.9 |
| ResNet-101 | 3× | 382M | 78.4 | 94.2 | 80.7 | **95.3** | - |
| ResNet-152 | 3× | 522M | 79.5 | 94.6 | **80.9** | 95.2 | - |

表9：更大编码器下， BYOL 在 ImageNet 的线性评估。top-1 与 top-5 准确率以百分比报告。

ResNet 架构（均按 [22]使用版本1）。有监督基线训练最多200轮，使用 Nesterov 动量0.9的 SGD、5轮线性预热后的余弦退火学习率，权重衰减 $1e-4$及0.1的标签平滑 [88] 。结果见图6。

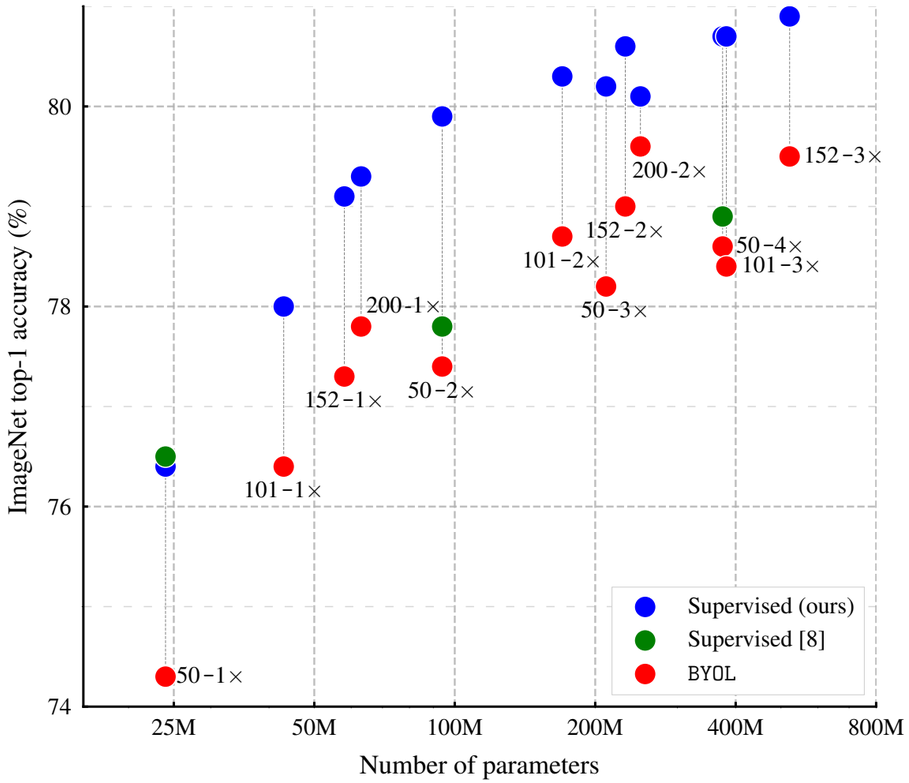



Figure 6: Results for linear evaluation of BYOL compared to fully supervised baselines with various ResNet architectures. Our supervised baselines are ran with RandAugment [87] augmentations.



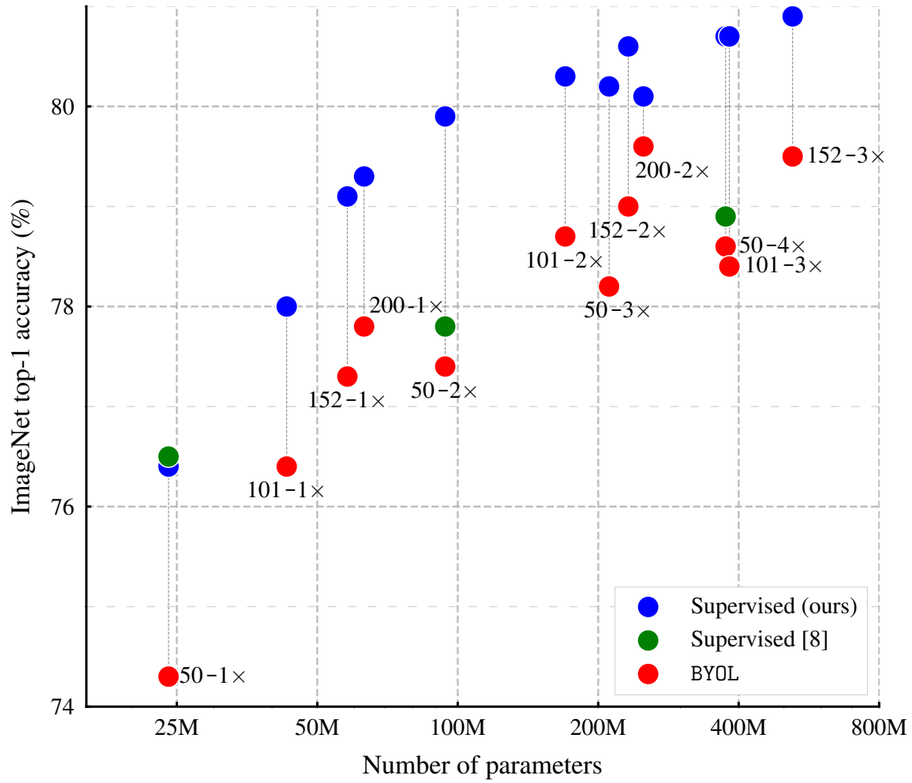



图6：多种 ResNet 架构下， BYOL 的线性评估与完全有监督基线的比较。我们的有监督基线使用 RandAugment [87] 增强。

# D Transfer to other datasets

## D.1 Datasets

| Dataset | Classes | Original train examples | Train examples | Valid. examples | Test examples | Accuracy measure | Test provided |
|---|---|---|---|---|---|---|---|
| ImageNet [21] | 1000 | 1281167 | 1271158 | 10009 | 50000 | Top-1 accuracy | - |
| Food101 [89] | 101 | 75750 | 68175 | 7575 | 25250 | Top-1 accuracy | - |
| CIFAR-10 [78] | 10 | 50000 | 45000 | 5000 | 10000 | Top-1 accuracy | - |
| CIFAR-100 [78] | 100 | 50000 | 44933 | 5067 | 10000 | Top-1 accuracy | - |
| Birdsnap [90] | 500 | 47386 | 42405 | 4981 | 2443 | Top-1 accuracy | - |
| Sun397 (split 1) [79] | 397 | 19850 | 15880 | 3970 | 19850 | Top-1 accuracy | - |
| Cars [91] | 196 | 8144 | 6494 | 1650 | 8041 | Top-1 accuracy | - |
| Aircraft [92] | 100 | 3334 | 3334 | 3333 | 3333 | Mean per-class accuracy | Yes |
| PASCAL-VOC2007 [80] | 20 | 5011 | 2501 | 2510 | 4952 | 11-point mAP / AP50 | - |
| PASCAL-VOC2012 [80] | 21 | 10582 | – | 2119 | 1449 | Mean IoU | - |
| DTD (split 1) [81] | 47 | 1880 | 1880 | 1880 | 1880 | Top-1 accuracy | Yes |
| Pets [93] | 37 | 3680 | 2940 | 740 | 3669 | Mean per-class accuracy | - |
| Caltech-101 [94] | 101 | 3060 | 2550 | 510 | 6084 | Mean per-class accuracy | - |
| Places365 [73] | 365 | 1803460 | 1803460 | – | 36500 | Top-1 accuracy | - |
| Flowers [95] | 102 | 1020 | 1020 | 1020 | 6149 | Mean per-class accuracy | Yes |

Table 10: Characteristics of image datasets used in transfer learning. When an official test split with labels is not publicly available, we use the official validation split as test set, and create a held-out validation set from the training examples.

We perform transfer via linear classification and fine-tuning on the same set of datasets as in [8], namely Food-101 dataset [89], CIFAR-10 [78] and CIFAR-100 [78], Birdsnap [90], the SUN397 scene dataset [79], Stanford Cars [91], FGVC Aircraft [92], the PASCAL VOC 2007 classification task [80], the Describable Textures Dataset (DTD) [81], Oxford-IIIT Pets [93], Caltech-101 [94], and Oxford 102 Flowers [95]. As in [8], we used the validation sets specified by the dataset creators to select hyperparameters for FGVC Aircraft, PASCAL VOC 2007, DTD, and Oxford 102 Flowers. On other datasets, we use the validation examples as test set, and hold out a subset of the training examples that we use as validation set. We use standard metrics for each datasets:

- *Top*-1: We compute the proportion of correctly classified examples.
- *Mean per class*: We compute the top-1 accuracy for each class separately and then compute the empirical mean over the classes.
- *Point 11-mAP*: We compute the empirical mean *average precision* as defined in [80].
- *Mean IoU:* We compute the empirical mean Intersection-Over-Union as defined in [80].
- *AP50:* We compute the Average Precision as defined in [80].

We detail the validation procedures for some specific datasets:

- For Sun397 [79], the original dataset specifies 10 train/test splits, all of which contain 50 examples/images of 397 different classes. We use the first train/test split. The original dataset specifies no validation split and therefore, the training images have been further subdivided into 40 images per class for the train split and 10 images per class for the valid split.
- For Birdsnap [90], we use a random selection of valid images with the same number of images per category as the test split.
- For DTD [81], the original dataset specifies 10 train/validation/test splits, we only use the first split.
- For Caltech-101 [94], the original does not dataset specifies any train/test splits. We have followed the approach used in [96]: This file defines datasets for 5 random splits of 25 training images per category, with 5 validation images per category and the remaining images used for testing.
- For ImageNet, we took the last 10009 last images of the official tensorflow ImageNet split.
- For Oxford-IIIT Pets, the valid set consists of 20 randomly selected images per class.

Information about the dataset are summarized in Table 10.

## D.2 Transfer via linear classification

We follow the linear evaluation protocol of [48, 74, 8] that we detail next for completeness. We train a regularized multinomial logistic regression classifier on top of the frozen representation, i.e., with frozen pretrained parameters



# D 迁移到其他数据集

## D.1 数据集

| 数据集 | 类别数 | 原始训练样本数 | 训练样本数 | 验证样本数 | 测试样本数 | 准确率指标 | 是否提供测试集 |
|---|---|---|---|---|---|---|---|
| ImageNet [21] | 1000 | 1281167 | 1271158 | 10009 | 50000 | Top-1 准确率 | - |
| Food101 [89] | 101 | 75750 | 68175 | 7575 | 25250 | Top-1 准确率 | - |
| CIFAR-10 [78] | 10 | 50000 | 45000 | 5000 | 10000 | Top-1 准确率 | - |
| CIFAR-100 [78] | 100 | 50000 | 44933 | 5067 | 10000 | Top-1 准确率 | - |
| Birdsnap [90] | 500 | 47386 | 42405 | 4981 | 2443 | Top-1 准确率 | - |
| SUN397（划分1） [79] | 397 | 19850 | 15880 | 3970 | 19850 | Top-1 准确率 | - |
| Cars [91] | 196 | 8144 | 6494 | 1650 | 8041 | Top-1 准确率 | - |
| Aircraft [92] | 100 | 3334 | 3334 | 3333 | 3333 | 类别平均准确率 | Yes |
| PASCAL-VOC2007 [80] | 20 | 5011 | 2501 | 2510 | 4952 | 11点 mAP／AP50 | - |
| PASCAL-VOC2012 [80] | 21 | 10582 | – | 2119 | 1449 | 平均 IoU | - |
| DTD（划分1） [81] | 47 | 1880 | 1880 | 1880 | 1880 | Top-1 准确率 | Yes |
| Pets [93] | 37 | 3680 | 2940 | 740 | 3669 | 类别平均准确率 | - |
| Caltech-101 [94] | 101 | 3060 | 2550 | 510 | 6084 | 类别平均准确率 | - |
| Places365 [73] | 365 | 1803460 | 1803460 | – | 36500 | Top-1 准确率 | - |
| Flowers [95] | 102 | 1020 | 1020 | 1020 | 6149 | 类别平均准确率 | Yes |

表10：迁移学习使用的图像数据集特征。若没有公开的带标签官方测试划分，则将官方验证划分用作测试集，并从训练样本中留出验证集。

我们在与 [8], 相同的数据集上进行线性分类和微调迁移：Food-101 [89], 、CIFAR-10 [78] 和 CIFAR-100 [78], 、Birdsnap [90], 、SUN397 场景数据集 [79], 、Stanford Cars [91],、FGVC Aircraft [92], 、PASCAL VOC 2007 分类任务 [80], 、可描述纹理数据集（DTD） [81],、Oxford-IIIT Pets [93], 、Caltech-101 [94], 和 Oxford 102 Flowers [95]。按照 [8], ，对于 FGVC Aircraft、PASCAL VOC 2007、DTD 和 Oxford 102 Flowers，使用数据集作者指定的验证集选择超参数。其他数据集将原验证样本作为测试集，并从训练样本中留出一部分作为验证集。各数据集使用标准指标：

- Top-1：计算正确分类样本的比例。
- 类别平均：分别计算每类 top-1 准确率，再对类别求经验均值。
- 11点 mAP：按 [80]的定义，计算经验平均精度均值。
- 平均 IoU：按 [80]的定义，计算交并比的经验均值。
- AP50：按 [80]的定义计算平均精度。

下面详述部分数据集的验证程序：

- 对于 SUN397 [79], ，原数据集指定10组训练／测试划分，每组均有397类，每类50张图像。我们使用第一组划分。原数据集未指定验证划分，因此再将训练图像分为每类40张训练图像和10张验证图像。
- 对于 Birdsnap [90], ，随机选取验证图像，使每类图像数与测试划分相同。
- 对于 DTD [81], ，原数据集指定10组训练／验证／测试划分，我们仅使用第一组。
- 对于 Caltech-101 [94], ，原数据集未指定训练／测试划分。我们按照 [96]，采用5组随机划分：每类25张训练图像、5张验证图像，其余用于测试。
- 对于 ImageNet，使用 TensorFlow 官方 ImageNet 划分的最后10009张图像。
- 对于 Oxford-IIIT Pets，验证集为每类随机选取的20张图像。

数据集信息汇总于表10。

## D.2 通过线性分类迁移

我们遵循 [48, 74, 8] 的线性评估协议，为完整起见详述如下。在冻结表征上训练正则化多项逻辑回归分类器，即冻结预训练参数

and without re-computing batch-normalization statistics. In training and testing, we do not perform any image augmentations; images are resized to 224 pixels along the shorter side using bicubic resampling and then normalized with ImageNet statistics. Finally, we minimize the cross-entropy objective using `LBFGS` with $\ell_2$-regularization, where we select the regularization parameters from a range of 45 logarithmically-spaced values between $10^{-6}$ and $10^5$. After choosing the best-performing hyperparameters on the validation set, the model is retrained on combined training and validation images together, using the chosen parameters. The final accuracy is reported on the test set.

### D.3 Transfer via fine-tuning

We follow the same fine-tuning protocol as in [32, 48, 76, 8] that we also detail for completeness. Specifically, we initialize the network with the parameters of the pretrained representation. At training time, we apply spatial transformation, i.e., random crops with resize to $224 \times 224$ pixels and random flips. At test time, images are resized to 256 pixels along the shorter side using bicubic resampling, after which a $224 \times 224$ center crop is extracted. In both cases, we normalize the color channels by subtracting the average color and dividing by the standard deviation (computed on ImageNet), after applying the augmentations. We optimize the loss using `SGD` with Nesterov momentum for 20000 steps with a batch size of 256 and with a momentum of 0.9. We set the momentum parameter for the batch normalization statistics to $\max(1-10/s, 0.9)$ where $s$ is the number of steps per epoch. The learning rate and weight decay are selected respectively with a grid of seven logarithmically spaced learning rates between $0.0001$ and $0.1$, and 7 logarithmically-spaced values of weight decay between $10^{-6}$ and $10^{-3}$, as well as no weight decay. These values of weight decay are divided by the learning rate. After choosing the best-performing hyperparameters on the validation set, the model is retrained on combined training and validation images together, using the chosen parameters. The final accuracy is reported on the test set.

### D.4 Implementation details for semantic segmentation

We use the same fully-convolutional network (FCN)-based [7] architecture as [9]. The backbone consists of the convolutional layers in ResNet-50. The $3 \times 3$ convolutions in the conv5 blocks use dilation 2 and stride 1. This is followed by two extra $3 \times 3$ convolutions with 256 channels, each followed by batch normalization and ReLU activations, and a $1 \times 1$ convolution for per-pixel classification. The dilation is set to 6 in the two extra $3 \times 3$ convolutions. The total stride is 16 (FCN-16s [7]).

We train on the `train_aug2012` set and report results on `val2012`. Hyperparameters are selected on a 2119 images held-out validation set. We use a standard per-pixel softmax cross-entropy loss to train the FCN. Training is done with random scaling (by a ratio in $[0.5, 2.0]$), cropping, and horizontal flipping. The crop size is 513. Inference is performed on the $[513, 513]$ central crop. For training we use a batch size of 16 and weight decay of $0.0001$. We select the base learning rate by sweeping across 5 logarithmically spaced values between $10^{-3}$ and $10^{-1}$. The learning rate is multiplied by $0.1$ at the 70-th and 90-th percentile of training. We train for 30000 iterations, and average the results on 5 seeds.

### D.5 Implementation details for object detection

For object detection, we follow prior work on Pascal detection transfer [40, 23] wherever possible. We use a Faster R-CNN [82] detector with a R50-C4 backbone with a frozen representation. The R50-C4 backbone ends with the conv4 stage of a ResNet-50, and the box prediction head consists of the conv5 stage (including global pooling). We preprocess the images by applying multi-scale augmentation (rescaling the image so its longest edge is between 480 and 1024 pixels) but no other augmentation. We use an asynchronous `SGD` optimizer with 9 workers and train for 1.5M steps. We used an initial learning rate of $10^{-3}$, which is reduced to $10^{-4}$ at 1M steps and to $10^{-5}$ at 1.2M steps.

### D.6 Implementation details for depth estimation

For depth estimation, we follow the same protocol as in [83], and report its core components for completeness. We use a standard ResNet-50 backbone and feed the conv5 features into 4 fast up-projection blocks with respective filter sizes 512, 256, 128, and 64. We use a reverse Huber loss function for training [83, 97].

The original NYU Depth v2 frames of size $[640, 480]$ are down-sampled by a factor $0.5$ and center-cropped to $[304, 228]$ pixels. Input images are randomly horizontally flipped and the following color transformations are applied:

- *Grayscale* with an application probability of 0.3.



且不重新计算批归一化统计量。训练与测试均不做图像增强；用双三次重采样将短边缩放为224像素，再用 ImageNet 统计量归一化。最后使用 `LBFGS` 最小化带 $\ell_2$正则化的交叉熵目标，正则化参数在 $10^{-6}$ 至$10^5$之间的45个对数等距值中选择。在验证集选出最佳超参数后，合并训练与验证图像，用选定参数重新训练，并在测试集报告最终准确率。

### D.3 通过微调迁移

我们遵循 [32, 48, 76, 8] 的微调协议，为完整起见也在此详述。首先用预训练表征的参数初始化网络。训练时进行空间变换，即随机裁剪、缩放为 $224 \times 224$ 像素并随机翻转。测试时用双三次重采样将短边缩放为256像素，再提取 $224 \times 224$ 中心裁剪。两种情况下，都在增强后使用 ImageNet 均值和标准差归一化颜色通道。使用带 Nesterov 动量的 `SGD` 优化损失，训练20000步，批量大小256，动量0.9。批归一化统计量的动量设为 $\max(1-10/s, 0.9)$，其中 $s$ 为每轮步数。学习率在0.0001至0.1之间的7个对数等距值中选择；权重衰减在 $10^{-6}$ 至 $10^{-3}$之间的7个对数等距值以及零衰减中选择。上述权重衰减值除以学习率。在验证集选出最佳超参数后，合并训练和验证图像，使用选定参数重新训练，并在测试集报告最终准确率。

### D.4 语义分割实现细节

我们采用与 [9]相同的基于全卷积网络（FCN） [7] 的架构。骨干为 ResNet-50 的卷积层，conv5 模块中的 $3 \times 3$ 卷积采用膨胀率2、步幅1。之后接两个额外的256通道 $3 \times 3$ 卷积，每个后接批归一化和 ReLU，再用 $1 \times 1$ 卷积进行逐像素分类。两个额外的 $3 \times 3$卷积膨胀率为6。总步幅为16（FCN-16s [7]）。

我们在 `train_aug2012` 集上训练，在 `val2012`上报告结果，使用留出的2119张验证图像选择超参数。FCN 采用标准逐像素 softmax 交叉熵损失。训练增强包括随机缩放（比例为 $[0.5, 2.0]$）、裁剪和水平翻转，裁剪尺寸为513。推理使用 $[513, 513]$ 中心裁剪。训练批量为16，权重衰减为0.0001；基础学习率在 $10^{-3}$ 至 $10^{-1}$之间的5个对数等距值中选择。在训练进度70%和90%处，学习率各乘0.1。训练30000次迭代，对5个随机种子取平均。

### D.5 目标检测实现细节

目标检测尽可能遵循此前 Pascal 检测迁移研究 [40, 23] 。我们使用 Faster R-CNN [82] 检测器，配合冻结表征的 R50-C4 骨干。该骨干止于 ResNet-50 的 conv4 阶段，边界框预测头由 conv5 阶段（含全局池化）构成。预处理仅使用多尺度增强（将图像最长边缩放到480至1024像素），无其他增强。使用9个工作进程的异步 `SGD` 优化器，训练150万步。初始学习率为 $10^{-3}$，在100万步时降为 $10^{-4}$，120万步时降为 $10^{-5}$ 。

### D.6 深度估计实现细节

深度估计遵循 [83], 的协议，为完整起见介绍核心部分。使用标准 ResNet-50 骨干，将 conv5 特征输入4个快速上投影模块，滤波器数分别为512、256、128和64。训练采用逆 Huber 损失 [83, 97]。

NYU Depth v2 原始帧尺寸为 $[640, 480]$ ，先以0.5倍下采样，再中心裁剪为$[304, 228]$ 像素。输入随机水平翻转，并施加以下颜色变换：

- 灰度化，应用概率为 0.3.

- *Brightness* with a maximum brightness difference of $0.1255$.
- *Saturation* with a saturation factor randomly picked in the interval $[0.5, 1.5]$.
- *Hue* with a hue adjustment factor randomly picked in the interval $[-0.2, 0.2]$.

We train for 7500 steps with batch size 256, weight decay $0.0005$, and learning rate $0.16$ (scaled linearly from the setup of [83] to account for the bigger batch size).

### D.7 Further comparisons on PASCAL and NYU v2 Depth

For completeness, Table 11 and 12 extends Table 4 with other published baselines which use comparable networks. We see that in almost all settings, BYOL outperforms these baselines, even when those baselines use more data or deeper models. One notable exception is RMS error for NYU Depth prediction, which is a metric that's sensitive to outliers. The reason for this is unclear, but one possibility is that the network is producing higher-variance predictions due to being more confident about a test-set scene's similarities with those in the training set.

| Method | $AP_{50}$ | mIoU |
|---|---|---|
| Supervised-IN [9] | 74.4 | 74.4 |
| RelPos [23], by [40]* | 66.8 | - |
| Multi-task [40]* | 70.5 | - |
| LocalAgg [98] | 69.1 | - |
| MoCo [9] | 74.9 | 72.5 |
| MoCo + IG-1B [9] | 75.6 | 73.6 |
| CPC[32]** | 76.6 | - |
| SimCLR (repro) | 75.2 | 75.2 |
| BYOL (ours) | **77.5** | **76.3** |

Table 11: Transfer results in semantic segmentation and object detection.

* uses a larger model (ResNet-101). ** uses an even larger model (ResNet-161).

| Method | Higher better pct.$<1.25$ | pct.$<1.25^2$ | pct.$<1.25^3$ | Lower better rms | rel |
|---|---|---|---|---|---|
| Supervised-IN [83] | 81.1 | 95.3 | 98.8 | 0.573 | **0.127** |
| RelPos [23], by [40]* | 80.6 | 94.7 | 98.3 | **0.399** | 0.146 |
| Color [41], by [40]* | 76.8 | 93.5 | 97.7 | 0.444 | 0.164 |
| Exemplar [46, 40]* | 71.3 | 90.6 | 96.5 | 0.513 | 0.191 |
| Mot. Seg. [99], by [40]* | 74.2 | 92.4 | 97.4 | 0.473 | 0.177 |
| Multi-task [40]* | 79.3 | 94.2 | 98.1 | 0.422 | 0.152 |
| SimCLR (repro) | 83.3 | 96.5 | 99.1 | 0.557 | 0.134 |
| BYOL (ours) | **84.6** | **96.7** | **99.1** | 0.541 | 0.129 |

Table 12: Transfer results on NYU v2 depth estimation.

## E Pretraining on Places 365

To ascertain that BYOL learns good representations on other datasets, we applied our representation learning protocol on the scene recognition dataset Places365-Standard [73] before performing linear evaluation. This dataset contains 1.80 million training images and 36500 validation images with labels, making it roughly similar to ImageNet in scale. We reuse the *exact* same parameters as in Section 4 and train the representation for 1000 epochs, using BYOL and our SimCLR reproduction. Results for the linear evaluation setup (using the protocol of Appendix C.1 for ImageNet and Places365, and that of Appendix D on other datasets) are reported in Table 13.

Interestingly, the representation trained by using BYOL on Places365 (BYOL-PL) consistently outperforms that of SimCLR on the same dataset, but underperforms the BYOL representation trained on ImageNet (BYOL-IN) on all tasks except Places365 and SUN397 [79], another scene understanding dataset. Interestingly, all three unsupervised representation learning methods achieve a relatively high performance on the Places365 task; for comparison,



- 亮度：最大亮度差为0.1255。
- 饱和度：饱和度因子从区间 $[0.5, 1.5]$随机选取。
- 色相：色相调整因子从区间 $[-0.2, 0.2]$随机选取。

训练7500步，批量256、权重衰减0.0005、学习率0.16（从 [83]的设置线性缩放，以适应更大批量）。

### D.7 PASCAL 与 NYU v2 深度的进一步比较

为完整起见，表11、12在表4基础上加入了使用相近网络的其他已发表基线。几乎所有设置中， BYOL 均超过这些基线，即使它们使用更多数据或更深模型。一个明显例外是 NYU 深度预测的 RMS 误差，该指标对异常值敏感。原因尚不明确，一种可能是：网络对测试场景与训练场景的相似性更有信心，导致预测方差更大。

| 方法 | $AP_{50}$ | mIoU |
|---|---|---|
| Supervised-IN [9] | 74.4 | 74.4 |
| RelPos [23]，，来自 [40]* | 66.8 | - |
| 多任务 [40]* | 70.5 | - |
| LocalAgg [98] | 69.1 | - |
| MoCo [9] | 74.9 | 72.5 |
| MoCo +IG-1B [9] | 75.6 | 73.6 |
| CPC[32]** | 76.6 | - |
| SimCLR （复现） | 75.2 | 75.2 |
| BYOL （本文） | **77.5** | **76.3** |

表11：语义分割及目标检测迁移结果。

* 使用更大模型（ResNet-101）。 ** 使用更大的模型（ResNet-161）。

| 方法 | 越高越好 pct.$<1.25$ | pct.$<1.25^2$ | pct.$<1.25^3$ | 越低越好 rms | rel |
|---|---|---|---|---|---|
| Supervised-IN [83] | 81.1 | 95.3 | 98.8 | 0.573 | **0.127** |
| RelPos [23]，，来自 [40]* | 80.6 | 94.7 | 98.3 | **0.399** | 0.146 |
| 着色 [41]，，来自 [40]* | 76.8 | 93.5 | 97.7 | 0.444 | 0.164 |
| Exemplar [46, 40]* | 71.3 | 90.6 | 96.5 | 0.513 | 0.191 |
| 运动分割 [99]，，来自 [40]* | 74.2 | 92.4 | 97.4 | 0.473 | 0.177 |
| 多任务 [40]* | 79.3 | 94.2 | 98.1 | 0.422 | 0.152 |
| SimCLR （复现） | 83.3 | 96.5 | 99.1 | 0.557 | 0.134 |
| BYOL （本文） | **84.6** | **96.7** | **99.1** | 0.541 | 0.129 |

表12：NYU v2 深度估计迁移结果。

## E 在 Places365 上预训练

为确认 BYOL 在其他数据集也能学习良好表征，我们先在场景识别数据集 Places365-Standard [73] 上应用表征学习协议，再做线性评估。该数据集有180万训练图像及36500张带标签验证图像，规模与 ImageNet 大致相近。复用第4节完全相同的参数，使用BYOL 及自行复现的 SimCLR 训练1000轮。线性评估结果见表13：ImageNet 和 Places365 使用附录 C.1 协议，其他数据集使用附录D协议。

有趣的是，在 Places365 上用 BYOL 训练的表征（BYOL-PL）始终优于同数据集的SimCLR ，但除了 Places365 和另一场景理解数据集 SUN397 [79]，，在其他任务上均低于 ImageNet 上训练的 BYOL 表征（BYOL-IN）。三种无监督表征学习方法在 Places365 任务上均表现较好；作为比较，

reference [73] (in its linked repository) reports a top-1 accuracy of 55.2% for a ResNet-50v2 trained from scratch using labels on this dataset.

| Method | Places365 | ImageNet | Food101 | CIFAR10 | CIFAR100 | Birdsnap | SUN397 | Cars | Aircraft | DTD | Pets | Caltech-101 | Flowers |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| BYOL-IN | 51.0 | 74.3 | 75.3 | 91.3 | 78.4 | 57.3 | 62.6 | 67.2 | 60.6 | 76.5 | 90.4 | 94.3 | 96.1 |
| BYOL-PL | **53.2** | **58.5** | **64.7** | **84.5** | **66.1** | **28.8** | **64.2** | **55.6** | **55.9** | **68.5** | **66.1** | **84.3** | **90.0** |
| SimCLR-PL | 53.0 | 56.5 | 61.7 | 80.8 | 61.1 | 21.2 | 62.5 | 40.1 | 44.3 | 64.3 | 59.4 | 77.1 | 85.9 |

Table 13: Transfer learning results (linear evaluation, ResNet-50) from Places365 (PL). For comparison purposes, we also report the results from BYOL trained on ImageNet (BYOL-IN).

## F Additional ablation results

To extend on the above results, we provide additional ablations obtained using the same experimental setup as in Section 5, *i.e.,* 300 epochs, averaged over 3 seeds with the initial learning rate set to 0.3, the batch size to 4096, the weight decay to $10^{-6}$ and the base target decay rate $\tau_{\text{base}}$ to 0.99 unless specified otherwise. Confidence intervals correspond to the half-difference between the maximum and minimum score of these seeds; we omit them for half-differences lower than 0.25 accuracy points.

### F.1 Architecture settings

Table 14 shows the influence of projector and predictor architecture on BYOL. We examine the effect of different depths for both the projector and predictor, as well as the effect of the projection size. We do not apply a ReLU activation nor a batch normalization on the final linear layer of our MLPs such that a depth of 1 corresponds to a linear layer. Using the default projector and predictor of depth 2 yields the best performance.

| Proj. $g_\theta$ depth | Pred. $q_\theta$ depth | Top-1 | Top-5 |
|---|---|---|---|
| 1 | 1 | 61.9 | 86.0 |
| | 2 | 65.0 | 86.8 |
| | 3 | 65.7 | 86.8 |
| 2 | 1 | 71.5 | 90.7 |
| | 2 | **72.5** | **90.8** |
| | 3 | 71.4 | 90.4 |
| 3 | 1 | 71.4 | 90.4 |
| | 2 | 72.1 | 90.5 |
| | 3 | 72.1 | 90.5 |

(a) Projector and predictor depth (i.e. the number of Linear layers).

| Projector $g_\theta$ output dim | Top-1 | Top-5 |
|---|---|---|
| 16 | $69.9_{\pm 0.3}$ | 89.9 |
| 32 | 71.3 | 90.6 |
| 64 | 72.2 | 90.9 |
| 128 | 72.5 | 91.0 |
| 256 | 72.5 | 90.8 |
| 512 | **72.6** | **91.0** |

(b) Projection dimension.

Table 14: Effect of architectural settings where top-1 and top-5 accuracies are reported in %.

Table 15a shows the influence of the initial learning rate on BYOL. Note that the optimal value depends on the number of training epochs. Table 15b displays the influence of the weight decay on BYOL.

### F.2 Batch size

We run a sweep over the batch size for both BYOL and our reproduction of SimCLR. As explained in Section 5, when reducing the batch size by a factor $N$, we average gradients over $N$ consecutive steps and update the target network once every $N$ steps. We report in Table 16, the performance of both our reproduction of SimCLR and BYOL for batch sizes between 4096 (BYOL and SimCLR default) down to 64. We observe that the performance of SimCLR deteriorates faster than the one of BYOL which stays mostly constant for batch sizes larger than 256. We believe that the performance at batch size 256 could match the performance of the large 4096 batch size with proper parameter tuning when accumulating the gradient. We think that the drop in performance at batch size 64 in table 16 is mainly related to the ill behaviour of batch normalization at low batch sizes [100].



文献 [73] 链接的仓库报告，在该数据集上使用标签从零训练的 ResNet-50v2，top-1 准确率为55.2%。

| 方法 | Places365 | ImageNet | Food101 | CIFAR10 | CIFAR100 | Birdsnap | SUN397 | Cars | Aircraft | DTD | Pets | Caltech-101 | Flowers |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| BYOL-IN | 51.0 | 74.3 | 75.3 | 91.3 | 78.4 | 57.3 | 62.6 | 67.2 | 60.6 | 76.5 | 90.4 | 94.3 | 96.1 |
| BYOL-PL | **53.2** | **58.5** | **64.7** | **84.5** | **66.1** | **28.8** | **64.2** | **55.6** | | | | **84.3** | **90.0** |
| SimCLR-PL | 53.0 | 56.5 | 61.7 | 80.8 | 61.1 | 21.2 | 62.5 | 40.1 | 44.3 | 64.3 | 59.4 | 77.1 | 85.9 |

表13：从 Places365（PL）迁移的结果（线性评估，ResNet-50）。作为比较，也报告 ImageNet 上训练的 BYOL （BYOL-IN）。

## F 更多消融结果

为扩展上述结果，我们提供更多消融，使用与第5节相同的设置：训练300轮，取3个随机种子的平均，初始学习率0.3、批量4096、权重衰减 $10^{-6}$；除非另有说明，基础目标衰减率 $\tau_{\text{base}}$ 为0.99。置信区间为这些种子最高与最低分数之差的一半；小于0.25个百分点时省略。

### F.1 架构设置

表14展示投影器与预测器架构对 BYOL的影响。我们考察二者不同深度及投影维数的作用。MLP 的最终线性层不使用 ReLU 或批归一化，因此深度1对应单个线性层。默认深度2的投影器与预测器得到最佳性能。

| 投影器 $g_\theta$ 深度 | 预测器 $q_\theta$ 深度 | Top-1 | Top-5 |
|---|---|---|---|
| 1 | 1 | 61.9 | 86.0 |
| | 2 | 65.0 | 86.8 |
| | 3 | 65.7 | 86.8 |
| 2 | 1 | 71.5 | 90.7 |
| | 2 | **72.5** | **90.8** |
| | 3 | 71.4 | 90.4 |
| 3 | 1 | 71.4 | 90.4 |
| | 2 | 72.1 | 90.5 |
| | 3 | 72.1 | 90.5 |

（a）投影器与预测器深度（即线性层数）。

| 投影器 $g_\theta$ 输出维数 | Top-1 | Top-5 |
|---|---|---|
| 16 | $69.9_{\pm 0.3}$ | 89.9 |
| 32 | 71.3 | 90.6 |
| 64 | 72.2 | 90.9 |
| 128 | 72.5 | 91.0 |
| 256 | 72.5 | 90.8 |
| 512 | **72.6** | **91.0** |

（b）投影维数。

表14：架构设置的影响，top-1 与 top-5 准确率以百分比报告。

表15a展示初始学习率对 BYOL的影响。最优值取决于训练轮数。表15b展示权重衰减对 BYOL的影响。

### F.2 批量大小

我们对 BYOL 和自行复现的 SimCLR进行批量大小搜索。如第5节所述，批量缩小 $N$倍时，对连续 $N$ 步的梯度取平均，每 $N$ 步更新一次目标网络。表16报告复现的 SimCLR 与 BYOL 在4096（BYOL 和 SimCLR 的默认值）至64的批量范围内的性能。 SimCLR的性能下降快于 BYOL；后者在批量大于256时基本不变。我们认为，在累积梯度时适当调参，批量256的性能可能达到大批量4096的水平。表16中批量64的性能下降，主要可能与小批量下批归一化的不良行为有关 [100]。

| Learning rate | Top-1 | Top-5 |
|---|---|---|
| 0.01 | $34.8_{\pm 3.0}$ | $60.8_{\pm 3.2}$ |
| 0.1 | 65.0 | 87.0 |
| 0.2 | 71.7 | 90.6 |
| 0.3 | **72.5** | **90.8** |
| 0.4 | 72.3 | 90.6 |
| 0.5 | 71.5 | 90.1 |
| 1 | 69.4 | 89.2 |

(a) Base learning rate.

| Weight decay coefficient | Top-1 | Top-5 |
|---|---|---|
| $1 \cdot 10^{-7}$ | 72.1 | 90.4 |
| $5 \cdot 10^{-7}$ | **72.6** | **91.0** |
| $1 \cdot 10^{-6}$ | 72.5 | 90.8 |
| $5 \cdot 10^{-6}$ | $71.0_{\pm 0.3}$ | 90.0 |
| $1 \cdot 10^{-5}$ | $69.6_{\pm 0.4}$ | 89.3 |

(b) Weight decay.

Table 15: Effect of learning rate and weight decay. We note that BYOL's performance is quite robust within a range of hyperparameters. We also observe that setting the weight decay to zero may lead to unstable results (as in SimCLR).

| Batch size | Top-1 | | Top-5 | |
|---|---|---|---|---|
| | BYOL (ours) | SimCLR (repro) | BYOL (ours) | SimCLR (repro) |
| 4096 | **72.5** | 67.9 | **90.8** | 88.5 |
| 2048 | 72.4 | 67.8 | 90.7 | 88.5 |
| 1024 | 72.2 | 67.4 | 90.7 | 88.1 |
| 512 | 72.2 | 66.5 | 90.8 | 87.6 |
| 256 | 71.8 | $64.3_{\pm 2.1}$ | 90.7 | $86.3_{\pm 1.0}$ |
| 128 | $69.6_{\pm 0.5}$ | 63.6 | 89.6 | 85.9 |
| 64 | $59.7_{\pm 1.5}$ | $59.2_{\pm 2.9}$ | $83.2_{\pm 1.2}$ | $83.0_{\pm 1.9}$ |

Table 16: Influence of the batch size.

### F.3 Image augmentations

Table 17 compares the impact of individual image transformations on BYOL and SimCLR. BYOL is more resilient to changes of image augmentations across the board. For completeness, we also include an ablation with symmetric parameters across both views; for this ablation, we use a Gaussian blurring w.p. of 0.5 and a solarization w.p. of 0.2 for both $\mathcal{T}$ and $\mathcal{T}'$, and recover very similar results compared to our baseline choice of parameters.

| Image augmentation | Top-1 | | Top-5 | |
|---|---|---|---|---|
| | BYOL (ours) | SimCLR (repro) | BYOL (ours) | SimCLR (repro) |
| Baseline | **72.5** | 67.9 | **90.8** | 88.5 |
| Remove flip | 71.9 | 67.3 | 90.6 | 88.2 |
| Remove blur | 71.2 | 65.2 | 90.3 | 86.6 |
| Remove color (jittering and grayscale) | $63.4_{\pm 0.7}$ | 45.7 | $85.3_{\pm 0.5}$ | 70.6 |
| Remove color jittering | 71.8 | 63.7 | 90.7 | 85.9 |
| Remove grayscale | 70.3 | 61.9 | 89.8 | 84.1 |
| Remove blur in $\mathcal{T}'$ | 72.4 | 67.5 | 90.8 | 88.4 |
| Remove solarize in $\mathcal{T}'$ | 72.3 | 67.7 | 90.8 | 88.2 |
| Remove blur and solarize in $\mathcal{T}'$ | 72.2 | 67.4 | 90.8 | 88.1 |
| Symmetric blurring/solarization | 72.5 | 68.1 | 90.8 | 88.4 |
| Crop only | $59.4_{\pm 0.3}$ | $40.3_{\pm 0.3}$ | 82.4 | $64.8_{\pm 0.4}$ |
| Crop and flip only | $60.1_{\pm 0.3}$ | 40.2 | $83.0_{\pm 0.3}$ | 64.8 |
| Crop and color only | 70.7 | 64.2 | 90.0 | 86.2 |
| Crop and blur only | $61.1_{\pm 0.3}$ | 41.7 | 83.9 | 66.4 |

Table 17: Ablation on image transformations.



| 学习 rate | Top-1 | Top-5 |
|---|---|---|
| 0.01 | $34.8_{\pm 3.0}$ | $60.8_{\pm 3.2}$ |
| 0.1 | 65.0 | 87.0 |
| 0.2 | 71.7 | 90.6 |
| 0.3 | **72.5** | **90.8** |
| 0.4 | 72.3 | 90.6 |
| 0.5 | 71.5 | 90.1 |
| 1 | 69.4 | 89.2 |

（a）基础学习率。

| 权重衰减系数 | Top-1 | Top-5 |
|---|---|---|
| $1 \cdot 10^{-7}$ | 72.1 | 90.4 |
| $5 \cdot 10^{-7}$ | **72.6** | **91.0** |
| $1 \cdot 10^{-6}$ | 72.5 | 90.8 |
| $5 \cdot 10^{-6}$ | $71.0_{\pm 0.3}$ | 90.0 |
| $1 \cdot 10^{-5}$ | $69.6_{\pm 0.4}$ | 89.3 |

（b）权重衰减。

表15：学习率与权重衰减的影响。BYOL在一定超参数范围内相当稳健。将权重衰减设为零可能导致不稳定结果（与SimCLR类似）。

| 批次 size | Top-1 | | Top-5 | |
|---|---|---|---|---|
| | BYOL（本文） | SimCLR（复现） | BYOL（本文） | SimCLR（复现） |
| 4096 | **72.5** | 67.9 | **90.8** | 88.5 |
| 2048 | 72.4 | 67.8 | 90.7 | 88.5 |
| 1024 | 72.2 | 67.4 | 90.7 | 88.1 |
| 512 | 72.2 | 66.5 | 90.8 | 87.6 |
| 256 | 71.8 | $64.3_{\pm 2.1}$ | 90.7 | $86.3_{\pm 1.0}$ |
| 128 | $69.6_{\pm 0.5}$ | 63.6 | 89.6 | 85.9 |
| 64 | $59.7_{\pm 1.5}$ | $59.2_{\pm 2.9}$ | $83.2_{\pm 1.2}$ | $83.0_{\pm 1.9}$ |

表16：批量大小的影响。

### F.3 图像增强

表17比较单独图像变换对 BYOL 与 SimCLR的影响。总体上， BYOL 对增强变化更稳健。为完整起见，还给出两个视图参数对称的消融：对 $\mathcal{T}$和 $\mathcal{T}'$均以0.5概率高斯模糊、0.2概率曝光反转，结果与默认参数非常接近。

| 图像增强 | Top-1 | | Top-5 | |
|---|---|---|---|---|
| | BYOL（本文） | SimCLR（复现） | BYOL（本文） | SimCLR（复现） |
| 基线 | **72.5** | 67.9 | **90.8** | 88.5 |
| 去除翻转 | 71.9 | 67.3 | 90.6 | 88.2 |
| 去除模糊 | 71.2 | 65.2 | 90.3 | 86.6 |
| 去除颜色变换（抖动及灰度化） | $63.4_{\pm 0.7}$ | 45.7 | $85.3_{\pm 0.5}$ | 70.6 |
| 去除颜色抖动 | 71.8 | 63.7 | 90.7 | 85.9 |
| 去除灰度化 | 70.3 | 61.9 | 89.8 | 84.1 |
| 在 $\mathcal{T}'$中去除模糊 | 72.4 | 67.5 | 90.8 | 88.4 |
| 在 $\mathcal{T}'$中去除曝光反转 | 72.3 | 67.7 | 90.8 | 88.2 |
| 在 $\mathcal{T}'$中去除模糊和曝光反转 | 72.2 | 67.4 | 90.8 | 88.1 |
| 对称的模糊／曝光反转 | 72.5 | 68.1 | 90.8 | 88.4 |
| 仅裁剪 | $59.4_{\pm 0.3}$ | $40.3_{\pm 0.3}$ | 82.4 | $64.8_{\pm 0.4}$ |
| 仅裁剪与翻转 | $60.1_{\pm 0.3}$ | 40.2 | $83.0_{\pm 0.3}$ | 64.8 |
| 仅裁剪与颜色变换 | 70.7 | 64.2 | 90.0 | 86.2 |
| 仅裁剪与模糊 | $61.1_{\pm 0.3}$ | 41.7 | 83.9 | 66.4 |

表17：图像变换的消融。

| Loss weight $\beta$ | Temperature $\alpha$ | Top-1 | Top-5 |
|---|---|---|---|
| 0 | 0.1 | 72.5 | 90.8 |
| 0.1 | 0.01 | 72.2 | 90.7 |
| | 0.1 | 72.4 | 90.9 |
| | 0.3 | **72.7** | 91.0 |
| | 1 | 72.6 | 90.9 |
| | 3 | 72.5 | 90.9 |
| | 10 | 72.5 | 90.9 |
| 0.5 | 0.01 | 70.9 | 90.2 |
| | 0.1 | 72.0 | 90.8 |
| | 0.3 | **72.7** | **91.2** |
| | 1 | 72.7 | 91.1 |
| | 3 | 72.6 | 91.1 |
| | 10 | 72.5 | 91.0 |
| 1 | 0.01 | 53.9$\pm$0.5 | 77.5$\pm$0.5 |
| | 0.1 | 70.9 | 90.3 |
| | 0.3 | **72.7** | 91.1 |
| | 1 | **72.7** | 91.1 |
| | 3 | 72.6 | 91.0 |
| | 10 | 72.6 | 91.1 |

Table 18: Top-1 accuracy in % under linear evaluation protocol at 300 epochs of sweep over the temperature $\alpha$ and the dispersion term weight $\beta$ when using a predictor and a target network.

### F.4 Details on the relation to contrastive methods

As mentioned in Section 5, the BYOL loss Eq. 2 can be derived from the InfoNCE loss

$$\text{InfoNCE}_\theta^{\alpha,\beta} \triangleq \frac{2}{B}\sum_{i=1}^{B} S_\theta(v_i, v_i') - \frac{2\alpha\cdot\beta}{B}\sum_{i=1}^{B}\ln\left(\sum_{j\neq i}\exp\frac{S_\theta(v_i,v_j)}{\alpha} + \sum_j \exp\frac{S_\theta(v_i,v_j')}{\alpha}\right), \tag{8}$$

with

$$S_\theta(u_1,u_2) \triangleq \frac{\langle\phi(u_1),\psi(u_2)\rangle}{\|\phi(u_1)\|_2\cdot\|\psi(u_2)\|_2}. \tag{9}$$

The InfoNCE loss, introduced in [10], can be found in factored form in [84] as

$$\text{InfoNCE}_\theta \triangleq \frac{1}{B}\sum_{i=1}^{B}\ln\frac{f(v_i,v_i')}{\frac{1}{B}\sum_j \exp f(v_i,v_j')}. \tag{10}$$

As in SimCLR [8] we also use negative examples given by $(v_i, v_j)_{j\neq i}$ to get

$$\frac{1}{B}\sum_{i=1}^{B}\ln\frac{\exp f(v_i,v_i')}{\frac{1}{B}\sum_{j\neq i}\exp f(v_i,v_j) + \frac{1}{B}\sum_j \exp f(v_i,v_j')} \tag{11}$$

$$= \ln B + \frac{1}{B}\sum_{i=1}^{B} f(v_i,v_i') - \frac{1}{B}\sum_{i=1}^{B}\ln\left(\sum_{j\neq i}\exp f(v_i,v_j) + \sum_j \exp f(v_i,v_j')\right). \tag{12}$$

To obtain Eq. 8 from Eq. 12, we subtract $\ln B$ (which is independent of $\theta$), multiply by $2\alpha$, take $f(x,y) = S_\theta(x,y)/\alpha$ and finally multiply the second (negative examples) term by $\beta$. Using $\beta = 1$ and dividing by $2\alpha$ gets us back to the usual InfoNCE loss as used by SimCLR.

In our ablation in Table 5b, we set the temperature $\alpha$ to its best value in the SimCLR setting (i.e., $\alpha = 0.1$). With this value, setting $\beta$ to 1 (which adds negative examples), in the BYOL setting (i.e., with both a predictor and a target network) hurts the performances. In Table 18, we report results of a sweep over both the temperature $\alpha$ and the



| 损失权重 $\beta$ | 温度 $\alpha$ | Top-1 | Top-5 |
|---|---|---|---|
| 0 | 0.1 | 72.5 | 90.8 |
| 0.1 | 0.01 | 72.2 | 90.7 |
| | 0.1 | 72.4 | 90.9 |
| | 0.3 | **72.7** | 91.0 |
| | 1 | 72.6 | 90.9 |
| | 3 | 72.5 | 90.9 |
| | 10 | 72.5 | 90.9 |
| 0.5 | 0.01 | 70.9 | 90.2 |
| | 0.1 | 72.0 | 90.8 |
| | 0.3 | **72.7** | **91.2** |
| | 1 | 72.7 | 91.1 |
| | 3 | 72.6 | 91.1 |
| | 10 | 72.5 | 91.0 |
| 1 | 0.01 | 53.9$\pm$0.5 | 77.5$\pm$0.5 |
| | 0.1 | 70.9 | 90.3 |
| | 0.3 | **72.7** | 91.1 |
| | 1 | **72.7** | 91.1 |
| | 3 | 72.6 | 91.0 |
| | 10 | 72.6 | 91.1 |

表18：使用预测器与目标网络、同时搜索温度 $\alpha$ 和分散项权重 $\beta$ 时，300轮线性评估的 top-1 准确率（%）。

### F.4 与对比方法关系的细节

如第5节所述，式2的 BYOL 损失可由 InfoNCE 损失导出。

$$\text{InfoNCE}_\theta^{\alpha,\beta} \triangleq \frac{2}{B}\sum_{i=1}^{B} S_\theta(v_i, v_i') - \frac{2\alpha\cdot\beta}{B}\sum_{i=1}^{B}\ln\left(\sum_{j\neq i}\exp\frac{S_\theta(v_i,v_j)}{\alpha} + \sum_j \exp\frac{S_\theta(v_i,v_j')}{\alpha}\right), \tag{8}$$

with

$$S_\theta(u_1,u_2) \triangleq \frac{\langle\phi(u_1),\psi(u_2)\rangle}{\|\phi(u_1)\|_2\cdot\|\psi(u_2)\|_2}. \tag{9}$$

[10], 提出的 InfoNCE 损失可写为 [84] 中的分解形式：

$$\text{InfoNCE}_\theta \triangleq \frac{1}{B}\sum_{i=1}^{B}\ln\frac{f(v_i,v_i')}{\frac{1}{B}\sum_j \exp f(v_i,v_j')}. \tag{10}$$

按照 SimCLR [8]，我们还使用（$v_i$，$v_j$）$_{j\neq i}$ 给出的负样本，得到

$$\frac{1}{B}\sum_{i=1}^{B}\ln\frac{\exp f(v_i,v_i')}{\frac{1}{B}\sum_{j\neq i}\exp f(v_i,v_j) + \frac{1}{B}\sum_j \exp f(v_i,v_j')} \tag{11}$$

$$= \ln B + \frac{1}{B}\sum_{i=1}^{B} f(v_i,v_i') - \frac{1}{B}\sum_{i=1}^{B}\ln\left(\sum_{j\neq i}\exp f(v_i,v_j) + \sum_j \exp f(v_i,v_j')\right). \tag{12}$$

由式12得到式8：减去 $\ln B$（它与 $\theta$无关），乘以 $2\alpha$，令 $f$（$x$，$y$）$= S_\theta$（$x$，$y$）$/\alpha$，最后将第二项（负样本项）乘以 $\beta$。取 $\beta = 1$ 并除以 $2\alpha$，便得到 SimCLR使用的常规 InfoNCE 损失。

表5b消融将温度 $\alpha$ 设为 SimCLR 设置下的最佳值（即 $\alpha = 0.1$）。在此温度下，BYOL 设置（同时有预测器和目标网络）中将 $\beta$ 设为1、加入负样本，会降低性能。表18报告同时搜索温度 $\alpha$ 及

| Method | Predictor | Target parameters | $\beta$ | Top-1 |
|---|---|---|---|---|
| BYOL | ✓ | $\xi$ | 0 | **72.5** |
| | ✓ | $\xi$ | 1 | 70.9 |
| | | $\xi$ | 1 | 70.7 |
| | ✓ | $\text{sg}(\theta)$ | 1 | 70.2 |
| SimCLR | | $\theta$ | 1 | 69.4 |
| | ✓ | $\text{sg}(\theta)$ | 1 | 70.1 |
| | | $\text{sg}(\theta)$ | 1 | 69.2 |
| | ✓ | $\theta$ | 1 | 69.0 |
| | ✓ | $\text{sg}(\theta)$ | 0 | 5.5 |
| | ✓ | $\theta$ | 0 | 0.3 |
| | | $\xi$ | 0 | 0.2 |
| | | $\text{sg}(\theta)$ | 0 | 0.1 |
| | | $\theta$ | 0 | 0.1 |

Table 19: Top-1 accuracy in %, under linear evaluation protocol at 300 epochs, of intermediate variants between BYOL and SimCLR (with caveats discussed in Appendix F.5). sg means stop gradient.

weight parameter $\beta$ with a predictor and a target network where BYOL corresponds to $\beta = 0$. No run significantly outperforms BYOL and some values of $\alpha$ and $\beta$ hurt the performance. While the best temperature for SimCLR (without the target network and a predictor) is 0.1, after adding a predictor and a target network the best temperature $\alpha$ is higher than 0.3.

Using a target network in the loss has two effects: stopping the gradient through the prediction targets and stabilizing the targets with averaging. Stopping the gradient through the target change the objective while averaging makes the target stable and stale. In Table 5b we only shows results of the ablation when either using the online network as the prediction target (and flowing the gradient through it) or with a target network (both stopping the gradient into the prediction targets and computing the prediction targets with a moving average of the online network). We shown in Table 5b that using a target network is beneficial but it has two distinct effects we would like to understand from which effect the improvement comes from. We report in Table 19 the results already in Table 5b but also when the prediction target is computed with a stop gradient of the online network (the gradient does not flow into the prediction targets). This shows that making the prediction targets stable and stale is the main cause of the improvement rather than the change in the objective due to the stop gradient.

### F.5 SimCLR baseline of Section 5

The SimCLR baseline in Section 5 ($\beta = 1$, without predictor nor target network) is slightly different from the original one in [8]. First we multiply the original loss by $2\alpha$. For comparaison here is the original SimCLR loss,

$$\text{InfoNCE}_\theta \triangleq \frac{1}{B}\sum_{i=1}^{B}\frac{S_\theta(v_i, v_i')}{\alpha} - \frac{1}{B}\sum_{i=1}^{B}\ln\left(\sum_{j\neq i}\exp\frac{S_\theta(v_i, v_j)}{\alpha} + \sum_j \exp\frac{S_\theta(v_i, v_j')}{\alpha}\right). \tag{13}$$

Note that this multiplication by $2\alpha$ matters as the LARS optimizer is not completely invariant with respect to the scale of the loss. Indeed, LARS applies a preconditioning to gradient updates on all weights, except for biases and batch normalization parameters. Updates on preconditioned weights are invariant by multiplicative scaling of the loss. However, the bias and batch normalization parameter updates remain sensitive to multiplicative scaling of the loss.

We also increase the original SimCLR hidden and output size of the projector to respectively 4096 and 256. In our reproduction of SimCLR, these three combined changes improves the top-1 accuracy at 300 epochs from 67.9% (without the changes) to 69.2% (with the changes).

### F.6 Ablation on the normalization in the loss function

BYOL minimizes a squared error between the $\ell_2$-normalized prediction and target. We report results of BYOL at 300 epochs using different normalization function and no normalization at all. More precisely, given batch of prediction and targets in $\mathbb{R}^d$, $(p_i, t_i)_{i\leq B}$ with $B$ the batch size, BYOL uses the loss function $\frac{1}{B}\sum_{i=1}^{B}\|n_{\ell_2}(p_i) - n_{\ell_2}(z_i)\|_2^2$ with $n_{\ell_2} : x \to x/\|x\|_2$. We run BYOL with other normalization functions: non-trainable batch-normalization



| 方法 | 预测器 | 目标参数 | $\beta$ | Top-1 |
|---|---|---|---|---|
| BYOL | ✓ | $\xi$ | 0 | **72.5** |
| | ✓ | $\xi$ | 1 | 70.9 |
| | | $\xi$ | 1 | 70.7 |
| | ✓ | $\text{sg}(\theta)$ | 1 | 70.2 |
| SimCLR | | $\theta$ | 1 | 69.4 |
| | ✓ | $\text{sg}(\theta)$ | 1 | 70.1 |
| | | $\text{sg}(\theta)$ | 1 | 69.2 |
| | ✓ | $\theta$ | 1 | 69.0 |
| | ✓ | $\text{sg}(\theta)$ | 0 | 5.5 |
| | ✓ | $\theta$ | 0 | 0.3 |
| | | $\xi$ | 0 | 0.2 |
| | | $\text{sg}(\theta)$ | 0 | 0.1 |
| | | $\theta$ | 0 | 0.1 |

表19： BYOL与 SimCLR 之间各中间变体在300轮时的线性评估 top-1 准确率（%），相关说明见附录 F.5。sg 表示停止梯度。

权重参数 $\beta$，同时使用预测器与目标网络； BYOL 对应 $\beta = 0$。没有实验显著优于 BYOL ，而某些 $\alpha$ 和 $\beta$ 值会降低性能。 SimCLR（无目标网络与预测器）的最佳温度为0.1；加入预测器和目标网络后，最佳温度$\alpha$ 大于0.3。

在损失中使用目标网络有两种作用：阻止梯度流经预测目标，以及通过平均稳定目标。停止目标分支的梯度会改变优化目标，而平均使目标稳定且有所滞后。表5b仅比较了两种情况：直接以在线网络为预测目标（梯度也流经它），或使用目标网络（既停止预测目标的梯度，又通过在线网络的移动平均计算目标）。表5b表明目标网络有益，但我们希望区分这两个作用各自的贡献。表19除重复表5b结果外，还报告了将在线网络输出停止梯度后作为预测目标的情况（梯度不流入目标）。这说明，改进主要来自预测目标的稳定与滞后，而不是停止梯度造成的优化目标变化。

### F.5 SimCLR 第5节的基线

第5节的 SimCLR 基线（$\beta = 1$，无预测器及目标网络）与 [8]中的原版略有不同。首先，将原损失乘以 $2\alpha$。作为比较，原始 SimCLR 损失如下：

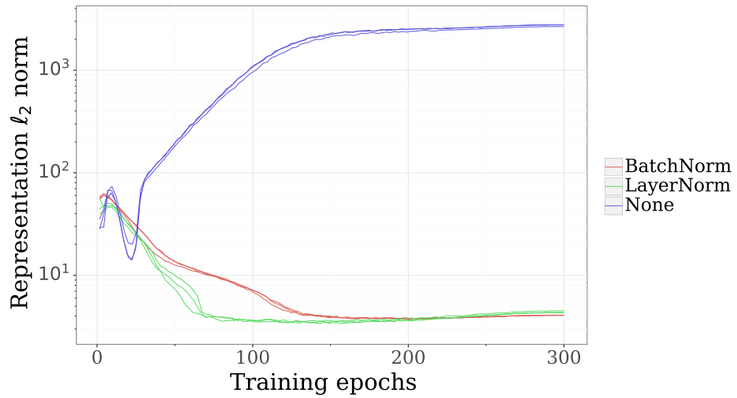
$$\text{InfoNCE}_\theta \triangleq \frac{1}{B}\sum_{i=1}^{B}\frac{S_\theta(v_i, v_i')}{\alpha} - \frac{1}{B}\sum_{i=1}^{B}\ln\left(\sum_{j\neq i}\exp\frac{S_\theta(v_i, v_j)}{\alpha} + \sum_j \exp\frac{S_\theta(v_i, v_j')}{\alpha}\right). \tag{13}$$

乘以 $2\alpha$ 这一操作有影响，因为 LARS 优化器对损失尺度并非完全不变。 LARS 对除偏置与批归一化参数之外的所有权重的梯度更新做预条件处理。经过预条件处理的权重更新对损失的乘法缩放不变；但偏置和批归一化参数的更新仍对这种缩放敏感。

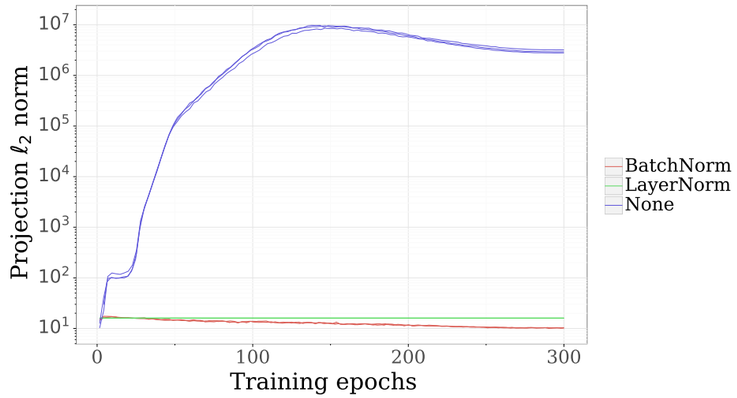
我们还将原始 SimCLR投影器的隐藏与输出维数分别增至4096和256。复现 SimCLR时，这三项变化共同使300轮的 top-1 准确率从67.9%（修改前）提高到69.2%（修改后）。

### F.6 损失函数归一化的消融

BYOL 最小化 $\ell_2$归一化的预测与目标之间的平方误差。我们报告训练300轮、使用不同归一化函数以及完全不归一化时的 BYOL 结果。更准确地，给定 $\mathbb{R}^d$中的预测和目标批次 $(p_i,\ t_i)_{i\leq B}$，批量大小为 $B$， BYOL 使用损失函数 $\frac{1}{B}\sum_{i=1}^{B}\|n_{\ell_2}(p_i) - n_{\ell_2}(z_i)\|_2^2$，其中 $n_{\ell_2}$： $x \to x/\|x\|_2$。我们也使用其他归一化函数运行 BYOL： 不可训练的批归一化




(a) Representation $\ell_2$-norm




(b) Projection $\ell_2$-norm

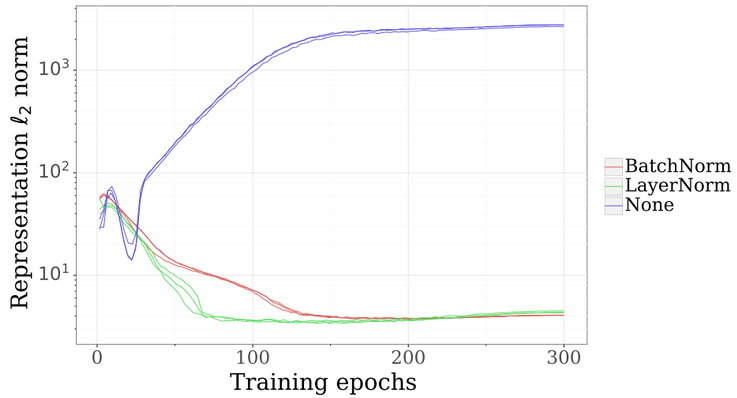
Figure 7: Effect of normalization on the $\ell_2$ norm of network outputs.

| Normalization | Top-1 | Top-5 |
|---|---|---|
| $\ell_2$-norm | **72.5** | **90.8** |
| LayerNorm | 72.5±0.4 | 90.1 |
| No normalization | 67.4 | 87.1 |
| BatchNorm | 65.3 | 85.3 |

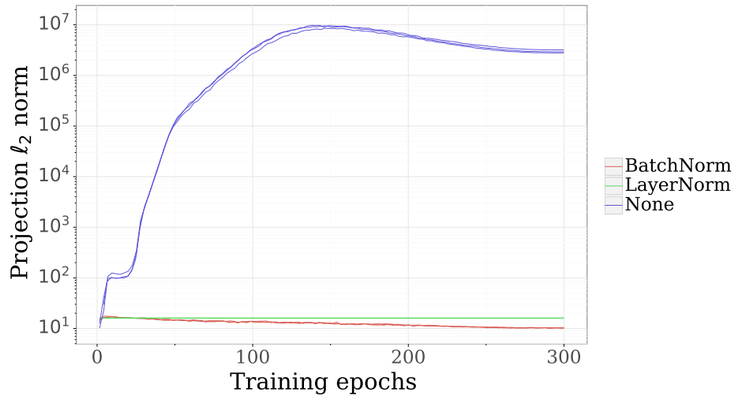
Table 20: Top-1 accuracy in % under linear evaluation protocol at 300 epochs for different normalizations in the loss.

and layer-normalization and no normalization. We divide the batch normalization and layer normalization by $\sqrt{d}$ to have a consistent scale with the $\ell_2$-normalization. We report results in Table 20 where $\ell_2$, LayerNorm, no normalization and BatchNorm respectively denote using $n_{\ell_2}$, $n_{\text{BN}}$, $n_{\text{LN}}$ and $n_{\text{ID}}$ with






(a) Representation $\ell_2$-norm




(b) Projection $\ell_2$-norm

图7：归一化对网络输出 $\ell_2$ 范数的影响。

| 归一化 | Top-1 | Top-5 |
|---|---|---|
| $\ell_2$范数 | **72.5** | **90.8** |
| LAYERNORM | 72.5±0.4 | 90.1 |
| 无归一化 | 67.4 | 87.1 |
| BATCHNORM | 65.3 | 85.3 |

表20：损失采用不同归一化时，300轮线性评估的 top-1 准确率（%）。

、层归一化，以及无归一化。为与 $\ell_2$归一化的尺度一致，将批归一化和层归一化结果除以 $\sqrt{d}$。表20中，$\ell_2$、LAYERNORM、无归一化和 BATCHNORM 分别表示使用 $n_{\ell_2}$、 $n_{\text{BN}}$、 $n_{\text{LN}}$ 和 $n_{\text{ID}}$ ， 其中

$$n^j_{\text{BN}_i} : x \to \frac{x^j_i - \mu^j_{\text{BN}}(x)}{\sigma^j_{\text{BN}}(x) \cdot \sqrt{d}}, \quad n^j_{\text{LN}_i} : x \to \frac{x^j_i - \mu_{\text{LN}_i}(x)}{\sigma_{\text{LN}_i}(x) \cdot \sqrt{d}}, \quad n_{\text{Id}} : x \to x,$$
$$\mu^j_{\text{BN}} : x \to \frac{1}{B}\sum_{i=1}^{B} x^j_i, \quad \sigma^j_{\text{BN}} : x \to \sqrt{\frac{1}{B}\sum_{i=1}^{B}\left(x^j_i\right)^2 - \mu^j_{\text{BN}}(x)^2},$$
$$\mu_{\text{LN}_i} : x \to \frac{1}{d}\sum_{j=1}^{d} x^j_i, \quad \sigma_{\text{LN}_i} : x \to \frac{\|x_i - \mu_{\text{LN}_i}(x)\|_2}{\sqrt{d}}$$

When using no normalization at all, the projection $\ell_2$ norm rapidly increases during the first 100 epochs and stabilizes at around $3 \cdot 10^6$ as shown in Figure 7. Despite this behaviour, using no normalization still performs reasonably well $(67.4\%)$. The $\ell_2$ normalization performs the best.

## G Training with smaller batch sizes

The results described in Section 4 were obtained using a batch size of 4096 split over 512 TPU cores. Due to its increased robustness, BYOL can also be trained using smaller batch sizes without significantly decreasing performance. Using the same linear evaluation setup, BYOL achieves $73.7\%$ top-1 accuracy when trained over 1000 epochs with a batch size of 512 split over 64 TPU cores (approximately 4 days of training). For this setup, we reuse the same setting as in Section 3, but use a base learning rate of 0.4 (appropriately scaled by the batch size) and $\tau_{\text{base}} = 0.9995$ with the same weight decay coefficient of $1.5 \cdot 10^{-6}$.

## H Details on Equation 5 in Section 3.2

In this section we clarify why BYOL's update is related to Eq. 5 from Section 3.2,

$$\nabla_\theta \mathbb{E}\left[\left\|q^\star(z_\theta) - z'_\xi\right\|_2^2\right] = \nabla_\theta \mathbb{E}\left[\left\|\mathbb{E}\left[z'_\xi | z_\theta\right] - z'_\xi\right\|_2^2\right] = \nabla_\theta \mathbb{E}\left[\sum_i \text{Var}(z'_{\xi,i} | z_\theta)\right]. \tag{5}$$

Recall that $q^\star$ is defined as

$$q^\star \triangleq \arg\min_q \mathbb{E}\left[\left\|q(z_\theta) - z'_\xi\right\|_2^2\right], \quad \text{where} \quad q^\star(z_\theta) = \mathbb{E}\left[z'_\xi | z_\theta\right], \tag{4}$$

and implicitly depends on $\theta$ and $\xi$; therefore, it should be denoted as $q^\star(\theta, \xi)$ instead of just $q^\star$. For simplicity we write $q^\star(\theta, \xi)(z_\theta)$ as $q^\star(\theta, \xi, z_\theta)$ the output of the optimal predictor for any parameters $\theta$ and $\xi$ and input $z_\theta$.

BYOL updates its online parameters following the gradient of Eq. 5, but considering only the gradients of $q$ with respect to its third argument $z$ when applying the chain rule. If we rewrite

$$\mathbb{E}\left[\left\|q^\star(\theta, \xi, z_\theta) - z'_\xi\right\|_2^2\right] = \mathbb{E}\left[L(q^\star(\theta, \xi, z_\theta), z'_\xi)\right], \tag{14}$$

the gradient of this quantity w.r.t. $\theta$ is

$$\frac{\partial}{\partial \theta}\mathbb{E}\left[L(q^\star(\theta, \xi, z_\theta), z'_\xi)\right] = \mathbb{E}\left[\frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial \theta} + \frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial z} \cdot \frac{\partial z_\theta}{\partial \theta}\right], \tag{15}$$

where $\frac{\partial q^\star}{\partial \theta}$ and $\frac{\partial q^\star}{\partial z}$ are the gradients of $q^\star$ with respect to its first and last argument. Using the envelope theorem, and thanks to the optimality condition of the predictor, the term $\mathbb{E}\left[\frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial \theta}\right] = 0$. Therefore, the remaining term $\mathbb{E}\left[\frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial z} \cdot \frac{\partial z_\theta}{\partial \theta}\right]$ where gradients are only back-propagated through the predictor's input is exactly the direction followed by BYOL.

## I Importance of a near-optimal predictor

In this part we build upon the intuitions of Section 3.2 on the importance of keeping the predictor near-optimal. Specifically, we show that it is possible to remove the exponential moving average in BYOL's target network (*i.e.*, simply copy weights of the online network into the target) without causing the representation to collapse, provided the predictor remains sufficiently good.



$$n^j_{\text{BN}_i} : x \to \frac{x^j_i - \mu^j_{\text{BN}}(x)}{\sigma^j_{\text{BN}}(x) \cdot \sqrt{d}}, \quad n^j_{\text{LN}_i} : x \to \frac{x^j_i - \mu_{\text{LN}_i}(x)}{\sigma_{\text{LN}_i}(x) \cdot \sqrt{d}}, \quad n_{\text{Id}} : x \to x,$$
$$\mu^j_{\text{BN}} : x \to \frac{1}{B}\sum_{i=1}^{B} x^j_i, \quad \sigma^j_{\text{BN}} : x \to \sqrt{\frac{1}{B}\sum_{i=1}^{B}\left(x^j_i\right)^2 - \mu^j_{\text{BN}}(x)^2},$$
$$\mu_{\text{LN}_i} : x \to \frac{1}{d}\sum_{j=1}^{d} x^j_i, \quad \sigma_{\text{LN}_i} : x \to \frac{\|x_i - \mu_{\text{LN}_i}(x)\|_2}{\sqrt{d}}$$

完全不归一化时，投影 $\ell_2$ 范数在前100轮迅速增大，随后稳定于 $3 \cdot 10^6$ 附近，见图7。尽管如此，不归一化仍取得较好性能（$67.4\%$）。$\ell_2$ 归一化表现最好。

## G 使用较小批量训练

第4节结果使用批量4096，分布在512个 TPU 核心上。由于稳健性更高， BYOL 也能用较小批量训练，而不显著降低性能。采用相同线性评估设置， BYOL 以批量512、64个 TPU 核心训练1000轮（约4天），得到73.7%的 top-1 准确率。该设置复用第3节配置，但基础学习率为0.4（再按批量大小缩放），使用 $\tau_{\text{base}} = 0.9995$ 及相同的权重衰减系数 $1.5 \cdot 10^{-6}$。

## H 第3.2节式5的细节

本节说明为何 BYOL的更新与第3.2节的式5有关：

$$\nabla_\theta \mathbb{E}\left[\left\|q^\star(z_\theta) - z'_\xi\right\|_2^2\right] = \nabla_\theta \mathbb{E}\left[\left\|\mathbb{E}\left[z'_\xi | z_\theta\right] - z'_\xi\right\|_2^2\right] = \nabla_\theta \mathbb{E}\left[\sum_i \text{Var}(z'_{\xi,i} | z_\theta)\right]. \tag{5}$$

回顾 $q^\star$ 的定义：

$$q^\star \triangleq \arg\min_q \mathbb{E}\left[\left\|q(z_\theta) - z'_\xi\right\|_2^2\right], \quad \text{where} \quad q^\star(z_\theta) = \mathbb{E}\left[z'_\xi | z_\theta\right], \tag{4}$$

且隐含依赖 $\theta$ 与 $\xi$，因此应记为 $q^\star(\theta, \xi)$，而不只是 $q^\star$。为简便，将 $q^\star(\theta, \xi)(z_\theta)$ 写作 $q^\star(\theta, \xi, z_\theta)$，表示任意参数 $\theta$、$\xi$和输入 $z_\theta$对应的最优预测器输出。

BYOL 沿式5的梯度更新在线参数，但应用链式法则时，只考虑 $q$ 关于第三个自变量 $z$ 的梯度。若改写为

$$\mathbb{E}\left[\left\|q^\star(\theta, \xi, z_\theta) - z'_\xi\right\|_2^2\right] = \mathbb{E}\left[L(q^\star(\theta, \xi, z_\theta), z'_\xi)\right], \tag{14}$$

该量关于 $\theta$ 的梯度为

$$\frac{\partial}{\partial \theta}\mathbb{E}\left[L(q^\star(\theta, \xi, z_\theta), z'_\xi)\right] = \mathbb{E}\left[\frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial \theta} + \frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial z} \cdot \frac{\partial z_\theta}{\partial \theta}\right], \tag{15}$$

其中 $\frac{\partial q^\star}{\partial \theta}$ 和 $\frac{\partial q^\star}{\partial z}$ 分别为 $q^\star$ 关于第一个和最后一个自变量的梯度。由包络定理以及预测器的最优性条件，项 $\mathbb{E}\left[\frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial \theta}\right] = 0$。因此，剩余项$\mathbb{E}\left[\frac{\partial L}{\partial q} \cdot \frac{\partial q^\star}{\partial z} \cdot \frac{\partial z_\theta}{\partial \theta}\right]$ 只通过预测器输入反向传播梯度，恰好就是 BYOL所沿的方向。

## I 近似最优预测器的重要性

本节延续第3.2节中保持预测器近似最优的重要性这一思路。具体地，我们表明，只要预测器足够好，便可去除 BYOL目标网络的指数移动平均（直接将在线权重复制到目标），而不导致表征坍塌。

### I.1 Predictor learning rate

In this setup, we remove the exponential moving average (*i.e.*, set $\tau = 0$ over the full training in Eq. 1), and multiply the learning rate of the predictor by a constant $\lambda$ compared to the learning rate used for the rest of the network; all other hyperparameters are unchanged. As shown in Table 21, using sufficiently large values of $\lambda$ provides a reasonably good level of performance and the performance sharply decreases with $\lambda$ to $0.01\%$ top-1 accuracy (no better than random) for $\lambda = 0$.

To show that this effect is directly related to a change of behavior in the predictor, and not only to a change of learning rate in any subpart of the network, we perform a similar experiment by using a multiplier $\lambda$ on the *predictor*'s learning rate, and a different multiplier $\mu$ for the *projector*. In Table 22, we show that the representation typically collapses or performs poorly when the predictor learning rate is lower or equal to that of the projector. As mentioned in Section 3.2, we further hypothesize that one of the contributions of the target network is to maintain a near optimal predictor at all times.

### I.2 Optimal linear predictor in closed form

Similarly, we can get rid of the slowly moving target network if we use a closed form optimal predictor on the batch, instead of a learned, non-optimal one. In this case we restrict ourselves to a linear predictor,

$$q^\star = \arg\min_Q \left\| Z_\theta Q - Z'_\xi \right\|_2^2 = (Z_\theta^\top Z_\theta)^{-1} Z_\theta^\top Z'_\xi$$

with $\|\cdot\|_2$ being the Frobenius norm, $Z_\theta$ and $Z'_\xi$ of shape $(B, F)$ respectively the online and target projections, where $B$ is the batch size and $F$ the number of features; and $q^\star$ of shape $(F, F)$, the optimal linear predictor for the batch.

At 300 epochs, when using the closed form optimal predictor, and directly hard copying the weights of the online network into the target, we obtain a top-1 accuracy of $fill$.

| $\lambda$ | Top-1 |
|---|---|
| 0 | 0.01 |
| 1 | 5.5 |
| 2 | 62.8±1.5 |
| 10 | 66.6 |
| 20 | 66.3±0.3 |
| Baseline | 72.5 |

Table 21: Top-1 accuracy at 300 epochs when removing the slowly moving target network, directly hard copying the weights of the online network into the target network, and applying a multiplier to the predictor learning rate.

| | | $\lambda_{pred}$ | | | | |
|---|---|---|---|---|---|---|
| | | 1 | 1.5 | 2 | 5 | 10 |
| $\mu_{proj}$ | 1 | 3.2±2.9 | 25.7±6.6 | 60.8±2.9 | 66.7±0.4 | 66.9 |
| | 1.5 | 1.4±2.0 | 9.2±7.0 | 55.2±5.8 | 61.5±0.6 | 66.0±0.3 |
| | 2 | 2.0±2.8 | 5.3±1.9 | 15.8±13.4 | 60.9±0.8 | 66.3 |
| | 5 | 1.5±0.9 | 2.5±1.5 | 2.5±1.4 | 20.5±2.0 | 60.5±0.6 |
| | 10 | 0.1 | 2.1±0.3 | 1.9±0.8 | 2.8±0.4 | 8.3±6.8 |

Table 22: Top-1 accuracy at 300 epochs when removing the slowly moving target network, directly hard copying the weights of the online network in the target network, and applying a multiplier $\mu$ to the projector and $\lambda$ to the predictor learning rate. The predictor learning rate needs to be higher than the projector learning rate in order to successfully remove the target network. This further suggests that the learning dynamic of predictor is central to BYOL's stability.



### I.1 预测器学习率

该设置去除指数移动平均（即式1中整个训练过程均设 $\tau = 0$ ），将预测器学习率设为其余网络学习率的常数 $\lambda$ 倍；其余超参数不变。表21显示，足够大的 $\lambda$ 可取得较好的性能，而性能随 $\lambda$ 下降而急剧降低，$\lambda = 0$时 top-1 仅$0.01\%$（不优于随机）。

为证明该现象直接来自预测器行为的变化，而不只是任意网络部分学习率的变化，我们进行类似实验：预测器学习率乘 $\lambda$ ，投影器学习率乘另一个系数 $\mu$ 。表22显示，预测器学习率小于或等于投影器时，表征通常坍塌或表现较差。如第3.2节所述，我们进一步推测，目标网络的作用之一是始终维持近似最优的预测器。

### I.2 最优线性预测器的闭式解

类似地，若使用批次上有闭式解的最优预测器，而非学习得到、非最优的预测器，也能去掉缓慢移动的目标网络。此时仅考虑线性预测器：

$$q^\star = \arg\min_Q \left\| Z_\theta Q - Z'_\xi \right\|_2^2 = (Z_\theta^\top Z_\theta)^{-1} Z_\theta^\top Z'_\xi$$

其中 $\|\cdot\|_2$为 Frobenius 范数； $Z_\theta$与 $Z'_\xi$分别为在线和目标投影，形状均为（$B$， $F$）， $B$是批量大小、$F$是特征数； $q^\star$形状为（$F$， $F$），是该批次的最优线性预测器。

训练300轮时，采用闭式最优预测器并直接将在线权重复制到目标，top-1 准确率为 $fill$。

| $\lambda$ | Top-1 |
|---|---|
| 0 | 0.01 |
| 1 | 5.5 |
| 2 | 62.8±1.5 |
| 10 | 66.6 |
| 20 | 66.3±0.3 |
| 基线 | 72.5 |

表21：训练300轮的 top-1 准确率。去掉缓慢移动的目标网络，直接将在线权重复制到目标网络，并对预测器学习率乘以一个系数。

| | | $\lambda_{pred}$ | | | | |
|---|---|---|---|---|---|---|
| | | 1 | 1.5 | 2 | 5 | 10 |
| $\mu_{proj}$ | 1 | 3.2±2.9 | 25.7±6.6 | 60.8±2.9 | 66.7±0.4 | 66.9 |
| | 1.5 | 1.4±2.0 | 9.2±7.0 | 55.2±5.8 | 61.5±0.6 | 66.0±0.3 |
| | 2 | 2.0±2.8 | 5.3±1.9 | 15.8±13.4 | 60.9±0.8 | 66.3 |
| | 5 | 1.5±0.9 | 2.5±1.5 | 2.5±1.4 | 20.5±2.0 | 60.5±0.6 |
| | 10 | 0.1 | 2.1±0.3 | 1.9±0.8 | 2.8±0.4 | 8.3±6.8 |

表22：训练300轮的 top-1 准确率。去掉缓慢移动的目标网络，直接将在线权重复制到目标网络；投影器和预测器学习率分别乘 $\mu$ 和 $\lambda$ 。要成功去掉目标网络，预测器学习率必须高于投影器。这进一步表明，预测器的学习动力学是 BYOL稳定性的关键。

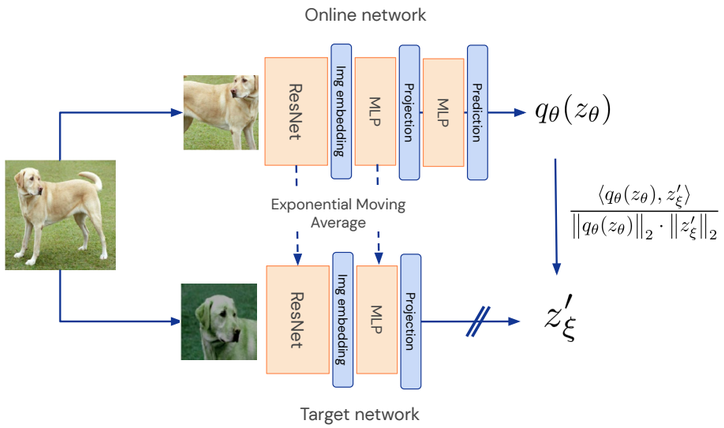



Figure 8: BYOL sketch summarizing the method by emphasizing the neural architecture.



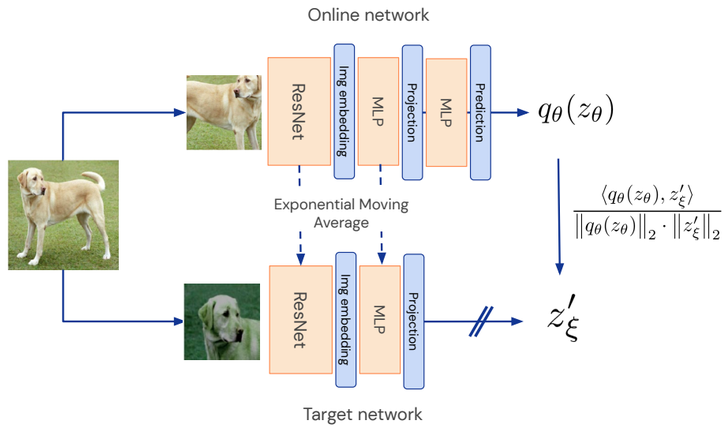



图8：BYOL 示意图，突出神经网络架构以概括方法。

## J BYOL pseudo-code in JAX

### J.1 Hyper-parameters

```
HPS = dict(
    max_steps=int(1000. * 1281167 / 4096),  # 1000 epochs
    batch_size=4096,
    mlp_hidden_size=4096,
    projection_size=256,
    base_target_ema=4e-3,
    optimizer_config=dict(
        optimizer_name='lars',
        beta=0.9,
        trust_coef=1e-3,
        weight_decay=1.5e-6,
        # As in SimCLR and official implementation of LARS, we exclude bias
        # and batchnorm weight from the Lars adaptation and weightdecay.
        exclude_bias_from_adaption=True),
    learning_rate_schedule=dict(
        # The learning rate is linearly increase up to
        # its base value * batchisze / 256 after warmup_steps
        # global steps and then anneal with a cosine schedule.
        base_learning_rate=0.2,
        warmup_steps=int(10. * 1281167 / 4096),
        anneal_schedule='cosine'),
    batchnorm_kwargs=dict(
        decay_rate=0.9,
        eps=1e-5),
    seed=1337,
    )
```

### J.2 Network definition

```
def network(inputs):
  """Build the encoder, projector and predictor."""
  embedding = ResNet(name='encoder', configuration='ResNetV1_50x1')(inputs)
  proj_out = MLP(name='projector')(embedding)
  pred_out = MLP(name='predictor')(proj_out)
  return dict(projection=proj_out, prediction=pred_out)


class MLP(hk.Module):
  """Multi Layer Perceptron, with normalization."""

  def __init(self, name):
    super().__init__(name=name)

  def __call__(self, inputs):
    out = hk.Linear(output_size=HPS['mlp_hidden_size'])(inputs)
    out = hk.BatchNorm(**HPS['batchnorm_kwargs'])(out)
    out = jax.nn.relu(out)
    out = hk.Linear(output_size=HPS['projection_size'])(out)
    return out

# For simplicity, we omit BatchNorm related states.
# In the actual code, we use hk.transform_with_state. The corresponding
# net_init function outputs both a params and a state variable,
# with state containing the moving averages computed by BatchNorm
net_init, net_apply = hk.transform(network)
```



## J BYOL的 JAX 伪代码

### J.1 超参数

```
HPS = dict(max_steps=int(1000. * 1281167 / 4096),  # 1000 epochs
batch_size=4096,mlp_hidden_size=4096,projection_size=256,
base_target_ema=4e-3,optimizer_config=dict(optimizer_name='lars',beta=0.9,
trust_coef=1e-3,weight_decay=1.5e-6,
# As in SimCLR and official implementation of LARS, we exclude bias
# and batchnorm weight from the Lars adaptation and weightdecay.
exclude_bias_from_adaption=True),learning_rate_schedule=dict(
# The learning rate is linearly increase up to
# its base value * batchisze / 256 after warmup_steps
# global steps and then anneal with a cosine schedule.
base_learning_rate=0.2,warmup_steps=int(10. * 1281167 / 4096),
anneal_schedule='cosine'),batchnorm_kwargs=dict(decay_rate=0.9,eps=1e-5),
seed=1337,)
```

### J.2 网络定义

```
def network(inputs):"""Build the encoder, projector and predictor."""
embedding = ResNet(name='encoder', configuration='ResNetV1_50x1')(inputs)
proj_out = MLP(name='projector')(embedding)
pred_out = MLP(name='predictor')(proj_out)
return dict(projection=proj_out, prediction=pred_out)


class MLP(hk.Module):"""Multi Layer Perceptron,
 with normalization."""def __init(self, name):
super().__init__(name=name)



  def __call__(self, inputs):
    out = hk.Linear(output_size=HPS['mlp_hidden_size'])(inputs)
    out = hk.BatchNorm(**HPS['batchnorm_kwargs'])(out)
    out = jax.nn.relu(out)
    out = hk.Linear(output_size=HPS['projection_size'])(out)
    return out

# For simplicity, we omit BatchNorm related states.# In the actual code,
 we use hk.transform_with_state. The corresponding
# net_init function outputs both a params and a state variable,
# with state containing the moving averages computed by BatchNormnet_init,
 net_apply = hk.transform(network)
```

### J.3 Loss function

```
def loss_fn(online_params, target_params, image_1, image_2):
  """Compute BYOL's loss function.

  Args:
    online_params: parameters of the online network (the loss is later
      differentiated with respect to the online parameters).
    target_params: parameters of the target network.
    image_1: first transformation of the input image.
    image_2: second transformation of the input image.

  Returns:
    BYOL's loss function.
  """

  online_network_out_1 = net_apply(params=online_params, inputs=image_1)
  online_network_out_2 = net_apply(params=online_params, inputs=image_2)
  target_network_out_1 = net_apply(params=target_params, inputs=image_1)
  target_network_out_2 = net_apply(params=target_params, inputs=image_2)

  def regression_loss(x, y):
    norm_x, norm_y = jnp.linalg.norm(x, axis=-1), jnp.linalg.norm(y, axis=-1)
    return -2. * jnp.mean(jnp.sum(x * y, axis=-1) / (norm_x * norm_y))

  # The stop_gradient is not necessary as we explicitly take the gradient with
  # respect to online parameters only. We leave it to indicate that gradients
  # are not backpropagated through the target network.
  loss = regression_loss(online_network_out_1['prediction'],
                         jax.lax.stop_gradient(target_network_out_2['projection']))
  loss += regression_loss(online_network_out_2['prediction'],
                          jax.lax.stop_gradient(target_network_out_1['projection']))
  return loss
```

### J.4 Training loop

```
def main(dataset):
  """Main training loop."""

  rng = jax.random.PRNGKey(HPS['seed'])
  rng, rng_init = jax.random.split(rng, num=2)
  dataset = dataset.batch(HPS['batch_size'])
  dummy_input = dataset.next()
  byol_state = init(rng_init, dummy_input)

  for global_step in range(HPS['max_steps']):
    inputs = dataset.next()

    rng, rng1, rng2 = jax.random.split(rng, num=3)
    image_1 = simclr_augmentations(inputs, rng1, image_number=1)
    image_2 = simclr_augmentations(inputs, rng2, image_number=2)
    byol_state = update_fn(
        **byol_state,
        global_step=global_step,
        image_1=image_1,
        image_2=image_2)

  return byol_state['online_params']
```



### J.3 损失函数

```
def loss_fn(online_params, target_params, image_1, image_2):
  """Compute BYOL's loss function.

  Args:
    online_params: parameters of the online network (the loss is later
      differentiated with respect to the online parameters).
    target_params: parameters of the target network.
    image_1: first transformation of the input image.
    image_2: second transformation of the input image.

  Returns:
    BYOL's loss function.
  """

  online_network_out_1 = net_apply(params=online_params, inputs=image_1)
  online_network_out_2 = net_apply(params=online_params, inputs=image_2)
  target_network_out_1 = net_apply(params=target_params, inputs=image_1)
  target_network_out_2 = net_apply(params=target_params, inputs=image_2)

  def regression_loss(x, y):norm_x, norm_y = jnp.linalg.norm(x, axis=-1),
   jnp.linalg.norm(y, axis=-1)
  return -2. * jnp.mean(jnp.sum(x * y, axis=-1) / (norm_x * norm_y))

  # The stop_gradient is not necessary as we explicitly take the gradient with
  # respect to online parameters only. We leave it to indicate that gradients
  # are not backpropagated through the target network.
  loss = regression_loss(online_network_out_1['prediction'],
                         jax.lax.stop_gradient(target_network_out_2['projection']))
  loss += regression_loss(online_network_out_2['prediction'],
                          jax.lax.stop_gradient(target_network_out_1['projection']))
  return loss
```

### J.4 训练循环

```
def main(dataset):
  """Main training loop."""

  rng = jax.random.PRNGKey(HPS['seed'])rng,
   rng_init = jax.random.split(rng, num=2)
  dataset = dataset.batch(HPS['batch_size'])
  dummy_input = dataset.next()
  byol_state = init(rng_init, dummy_input)

  for global_step in range(HPS['max_steps']):
    inputs = dataset.next()

    rng, rng1, rng2 = jax.random.split(rng, num=3)
    image_1 = simclr_augmentations(inputs, rng1, image_number=1)
    image_2 = simclr_augmentations(inputs, rng2, image_number=2)
    byol_state = update_fn(**byol_state,global_step=global_step,
    image_1=image_1,image_2=image_2)


  return byol_state['online_params']
```

### J.5 Update function

```
optimizer = Optimizer(**HPS['optimizer_config'])


def update_fn(online_params, target_params, opt_state, global_step, image_1,
              image_2):
  """Update online and target parameters.

  Args:
    online_params: parameters of the online network (the loss is  differentiated
      with respect to the online parameters only).
    target_params: parameters of the target network.
    opt_state: state of the optimizer.
    global_step: current training step.
    image_1: first transformation of the input image.
    image_2: second transformation of the input image.

  Returns:
    Dict containing updated online parameters, target parameters and
    optimization state.
  """
  # update online network
  grad_fn = jax.grad(loss_fn, argnums=0)
  grads = grad_fn(online_params, target_params, image_1, image_2)
  lr = learning_rate(global_step, **HPS['learning_rate_schedule'])
  updates, opt_state = optimizer(lr).apply(grads, opt_state, online_params)
  online_params = optix.apply_updates(online_params, updates)

  # update target network
  tau = target_ema(global_step, base_ema=HPS['base_target_ema'])
  target_params = jax.tree_multimap(lambda x, y: x + (1 - tau) * (y - x),
                                    target_params, online_params)
  return dict(
      online_params=online_params,
      target_params=target_params,
      opt_state=opt_state)


def init(rng, dummy_input):
  """BYOL's state initialization.

  Args:
    rng: random number generator used to initialize parameters.
    dummy_input: a dummy image, used to compute intermediate outputs shapes.

  Returns:
    Dict containing initial online parameters, target parameters and
    optimization state.
  """
  online_params = net_init(rng, dummy_input)
  target_params = net_init(rng, dummy_input)
  opt_state = optimizer(0).init(online_params)
  return dict(
      online_params=online_params,
      target_params=target_params,
      opt_state=opt_state)
```



### J.5 更新函数

```
optimizer = Optimizer(**HPS['optimizer_config'])


def update_fn(online_params, target_params, opt_state, global_step, image_1,
              image_2):
  """Update online and target parameters.

  Args:
    online_params: parameters of the online network (the loss is  differentiated
      with respect to the online parameters only).
    target_params: parameters of the target network.
    opt_state: state of the optimizer.
    global_step: current training step.
    image_1: first transformation of the input image.
    image_2: second transformation of the input image.

  Returns:
      Dict containing updated online parameters,
    target parameters and optimization state."""

  # update online network
  grad_fn = jax.grad(loss_fn, argnums=0)
  grads = grad_fn(online_params, target_params, image_1, image_2)
  lr = learning_rate(global_step, **HPS['learning_rate_schedule'])updates,
   opt_state = optimizer(lr).apply(grads, opt_state, online_params)
  online_params = optix.apply_updates(online_params, updates)

  # update target network
  tau = target_ema(global_step, base_ema=HPS['base_target_ema'])
  target_params = jax.tree_multimap(lambda x, y: x + (1 - tau) * (y - x),target_params,
   online_params)
  return dict(
  online_params=online_params,
  target_params=target_params,
  opt_state=opt_state)


def init(rng, dummy_input):
  """BYOL's state initialization.

  Args:rng: random number generator used to initialize parameters.
   dummy_input: a dummy image, used to compute intermediate outputs shapes.


  Returns:
    Dict containing initial online parameters, target parameters and
    optimization state.
  """
  online_params = net_init(rng, dummy_input)
  target_params = net_init(rng, dummy_input)
  opt_state = optimizer(0).init(online_params)
  return dict(online_params=online_params,
  target_params=target_params,
  opt_state=opt_state)
```